

> **EslamOs**

Linux 101

Complete Tutorial

From Fundamentals to Production

Twenty-two deep-dive guides • RH124 course notes • Hands-on labs • Command reference

22 Deep Guides	~35k Words	300+ Commands	Print Ready A4
--------------------------	----------------------	-------------------------	--------------------------

Eslam Nabawy • NTI Linux Course (RH124) • Open Source • GitHub Pages
August 2026 • v1.0 • linux101

A single, well-organized, print-ready consolidation of the entire site.
Stripped of navigation chrome, cookie banners & boilerplate — just the tutorial content.
<https://eslamnabawy.github.io/linux101>

How to Use This Guide

This PDF mirrors the website's structure but is optimized for linear reading and printing. Everything you see on the site — minus navigation bars, footers, search widgets and cookie banners — is here, in one place.

- **Structure.** Four parts follow the site's track order: **Foundations** → **System Ops** → **Networking & Security** → **Automation**. Each chapter is a standalone guide; read in order or jump via the clickable TOC.
- **Code is monospace.** All terminal commands, file paths and code blocks use a fixed-width font on a shaded background — clearly distinguished from prose. Copy-paste ready.
- **Tables & callouts.** Reference tables have dark headers and alternating row shading. Tips (blue) and warnings (amber/red) appear as colored boxes.
- **Print.** A4 with generous margins, header/footer on every page, page numbers bottom-center. Links are clickable on-screen and URLs are shown for print.
- **Editions.** This is a static snapshot. The live site may have newer updates — keep this PDF as a desk reference and check the site for the latest.

TIP

Tip: In your PDF reader, open the **Bookmarks** pane (Ctrl+Shift+B / Cmd+Shift+B) — every chapter and section is bookmarked for instant navigation, just like the site's sidebar.

Conventions

Style	Meaning
<code>monospace</code>	Command, path, code or value to type exactly
<code>placeholder</code>	Replace with your own value (e.g. myapp)
\$	Terminal prompt — do not type the \$ itself
#	Root prompt — command needs sudo or root

Table of Contents

Click any entry to jump — page numbers are generated at build time. Headings also appear as PDF bookmarks.

How to Use This Guide

2

Conventions

2

At a Glance — 22 Guides • 36,359 words • 4 Parts

25

Part I — Start Here: Foundations

25

Linux System & Command Rules — Reference Guide

25

Introduction

25

Core Mental Model

25

Filesystem Hierarchy Rules (FHS)

25

Path Rules

26

Command Syntax Rules

26

Permissions Rules

26

Users & Groups Rules

27

6.1 Checking who you currently are

27

6.2 Step-by-Step: Becoming Root — Method 1 (`su`)

27

6.3 Step-by-Step: Becoming Root — Method 2 (`sudo`)

28

6.4 Which Method Should You Actually Use?

29

Process Rules

29

I/O Redirection & Pipe Rules

29

Wildcards & Globbing Rules

30

Environment & Shell Rules

30

Package Management Rules

30

Service Management Rules (systemd)

31

Networking Rules

31

13.1 Viewing & Configuring Network Interfaces — Two Ways

31

13.2 Command Equivalence Table

33

Golden Rules & Common Gotchas

33

Important Files Worth Generating on a Linux / Dev System

34

Introduction

34

Shell Configuration	34
Editor Configuration	34
Git Configuration	34
SSH Configuration	35
Environment & Secrets	35
System-Level Config (root-owned — edit with care, see your failure-scenarios guide)	35
Service / Automation Files	36
Project-Level Dev Files	36
AIOps / Automation-Specific	37
Quick Priority Summary	37
Vim / Vi & Nano — The Complete Guide	37
The Core Idea: Modal Editing	37
Starting & Exiting	38
Normal Mode — Navigation	38
<i>Basic movement</i>	38
<i>Word-wise movement</i>	38
<i>Screen movement</i>	39
<i>Character search on current line</i>	39
Insert Mode — Entering It	39
Editing Commands (Normal Mode)	39
<i>Core operators</i>	39
<i>Combine with a motion</i>	40
<i>Single-key shortcuts (no motion needed)</i>	40
<i>Paste</i>	40
<i>Undo / redo</i>	40
Text Objects	41
Visual Mode	41
Search & Replace	41
<i>Searching</i>	41
<i>Find & replace (Ex command)</i>	41
Registers (Clipboard System)	42
Marks	42

Macros	42
Buffers, Windows & Tabs	42
Ex (Command-Line) Commands & Ranges	43
Folding	43
Configuration — `~/.vimrc`	43
Vim Quick Cheat Sheet	44
Philosophy	44
Starting Nano	44
File Operations	44
Navigation	44
Editing	45
Search & Replace	45
Multiple Buffers (Files)	46
Display Options	46
Configuration — `~/.nanorc`	46
Nano Quick Cheat Sheet	46
PART 3: Which One Should You Use?	46
Bash Scripting Deep Dive	48
Introduction	48
Script Anatomy	48
Variables	48
Arrays	48
<i>Indexed arrays</i>	48
<i>Associative arrays (key-value, bash 4+)</i>	49
<i>Common array pitfalls</i>	49
Loops	49
`for` loop — three styles	49
`while` loop	49
`until` loop (opposite of while — runs until the condition becomes true)	50
`break` and `continue`	50
Conditionals	50

<i>`case` statement — cleaner than a long if/elif chain</i>	51
Functions	51
Error Handling — The Part Most Scripts Skip	52
<i>The essential header for almost every script you write</i>	52
<i>`set -e` gotchas — it doesn't catch everything</i>	52
<i>Debug/trace mode</i>	52
<i>`trap` — running cleanup code no matter how the script exits</i>	52
<i>Checking exit codes explicitly</i>	53
Writing Scripts That Don't Silently Fail	53
String Manipulation	54
Input & Output	54
A Hardened Script Template	54

Part II — System Operations 57

Disk & Storage Management — Partitions, LVM, Mounting & fstab	57
Introduction	57
The Storage Hierarchy	57
Viewing Disks & Partitions	57
Partitioning	57
<i>MBR vs GPT — know which you're working with</i>	57
<i>Partitioning with `fdisk` (MBR-friendly, works on GPT too)</i>	58
<i>Partitioning with `parted` (better for GPT, scriptable)</i>	58
Creating a Filesystem	58
Mounting — Temporary (Manual)	59
LVM — Logical Volume Management	59
<i>Core concepts</i>	59
<i>Step-by-step LVM setup</i>	59
<i>Resizing — the actual payoff of using LVM</i>	60
<i>Snapshots — point-in-time copies for safe experimentation/backups</i>	60
<i>/etc/fstab` — The Real Deep Dive</i>	60
<i>Anatomy of a line</i>	60
<i>Field 1: Device identifier — and why UUID beats `/dev/sdX`</i>	61
<i>Field 2: Mount point</i>	61

Field 3: Filesystem type	61
Field 4: Mount options — the field most people leave at `defaults` and never learn	61
Field 5: Dump flag	62
Field 6: fsck pass order	62
Testing fstab changes safely — never just reboot and hope	62
Recovering from a broken fstab that already caused a boot failure	62
Swap in fstab	63
Checking & Repairing Filesystems	63
Ongoing Disk Usage Monitoring	63
Quick Reference Summary	63
The Linux Boot Process — BIOS/UEFI → GRUB → systemd	64
Introduction	64
The Full Chain, At a Glance	64
Stage 1: BIOS / UEFI Firmware	64
BIOS vs UEFI	64
What happens in this stage	64
Symptoms that mean the failure is HERE	65
What to check	65
Stage 2: GRUB (Bootloader)	65
GRUB's own stages (relevant mainly for legacy BIOS)	65
Key files	65
Common settings in `/etc/default/grub`	65
Interacting with GRUB live at boot	66
Symptoms that mean the failure is HERE	66
Manually booting from a `grub rescue>` prompt (get in, then fix it properly)	66
Stage 3: Kernel & initramfs	66
What initramfs is for	66
Symptoms that mean the failure is HERE	66
What to check	67
Stage 4: systemd (PID 1)	67
Targets — the systemd replacement for old-style "runlevels"	67
Symptoms that mean the failure is HERE	67

<i>Diagnosing systemd-stage boot problems</i>	67
<i>Recovering from emergency/rescue mode</i>	67
Full Diagnostic Flowchart	68
Reinstalling GRUB (When It's Genuinely Broken)	68
Quick Reference	69
systemd Deep Dive — Unit Files & Timers (The Modern Cron Replacement)	70
Introduction	70
Unit Basics	70
<i>Unit types you'll actually use</i>	70
<i>Where unit files live — and why location matters</i>	70
Anatomy of a Service Unit File	70
`[Unit]` section	71
`[Service]` section	71
`[Install]` section	72
Service Types — Picking the Right One	72
Restart Policies & Preventing Restart Loops	72
Managing Your Unit	72
Overriding a Package-Provided Unit (Without Editing It Directly)	73
systemd Timers — The Modern Cron Replacement	73
<i>The pairing convention</i>	73
`OnCalendar=` syntax — <i>more readable than cron once you know it</i>	74
<i>Monotonic timers — relative to boot/activation instead of a fixed clock time</i>	74
`Persistent=true` — <i>the feature cron simply doesn't have</i>	74
Timers vs Cron — Direct Comparison	74
Practical Example — A Health-Check Job (AIOps-Flavored)	75
Debugging Units	76
Quick Reference	76
Process & Performance Monitoring — Diagnosing Slow or Crashing Services	77
Introduction	77
The Mental Model	77
`top` — The Universal Starting Point	77
<i>Reading the header (first 5 lines)</i>	77

<i>Reading the process table columns</i>	78
<i>Interactive commands (press while `top` is running)</i>	78
`htop` — Same Data, Much Easier to Read	78
`vmstat` — Memory, Swap & CPU in One Glance	79
`iostat` — Disk I/O Specifically	79
`strace` — Seeing Exactly What a Process Is Doing	80
<i>Attach to an already-running process</i>	80
<i>Trace a command from the start</i>	80
<i>Filter to a specific category of syscalls</i>	80
<i>Summary mode — which syscalls are eating the most time</i>	80
<i>The single most common real-world use</i>	80
`journalctl` — Actually Diagnosing a Service	81
<i>The essentials</i>	81
<i>Filtering by severity</i>	81
<i>System-wide, not just one service</i>	81
<i>Diagnosing a crashed service — the actual workflow</i>	81
Putting It Together — Two Common Investigations	82
<i>"The server feels slow"</i>	82
<i>"A service crashed or won't start"</i>	82
Quick Reference	82
Linux Failure Scenarios — What Actually Causes Errors & File Corruption	83
Introduction	83
Shell Redirection Traps	83
<i>Reading and writing the same file in one command</i>	83
`>` vs `>>` confusion	83
<i>Piping into `sudo` incorrectly</i>	83
Killing a Process Mid-Write	84
Editor Swap Files & Crash Recovery	84
<i>Vim `.swp` conflict</i>	84
<i>Nano's lack of swap-file locking</i>	84
Destructive Command Mistakes	85
`rm -rf` with a wrong or expanded path	85

<i>Wildcard expansion including files you didn't intend</i>	85
<i>`dd` with the wrong `of=` target</i>	85
Permission & Ownership Mistakes	85
<i>Recursive chmod on the wrong directory</i>	85
<i>Recursive chown breaking service accounts</i>	85
Filesystem-Level Corruption	86
<i>Unsafe shutdown / power loss during a write</i>	86
<i>Unmounting a drive while it's still in use</i>	86
<i>Disk full during a write</i>	86
Text Encoding & Line-Ending Corruption	86
<i>Windows line endings breaking a script</i>	86
<i>Mixing character encodings</i>	87
<i>Editing a binary file as if it were text</i>	87
Symlink & Hardlink Pitfalls	87
<i>Deleting through a symlink with a trailing slash</i>	87
<i>Broken symlinks after moving the target</i>	87
Package Manager Corruption	87
<i>Interrupting `apt`/`dnf` mid-install</i>	87
<i>Running two package managers at once</i>	87
Concurrent Access Race Conditions	87
Quick Prevention Checklist	88
Log Management — syslog vs journald, logrotate & Where Logs Actually Live	89
Introduction	89
The Core Confusion: syslog vs journald	89
journald — Configuration & Persistence	89
<i>Is journald even keeping logs across reboots?</i>	89
<i>Enabling persistent journal storage</i>	89
<i>Key config: `/etc/systemd/journal.conf`</i>	90
<i>Manually cleaning the journal</i>	90
Where Every Service Actually Writes Its Logs	90
<i>systemd-managed services (the majority on a modern distro)</i>	90
<i>Classic system-wide log files (`/var/log/`)</i>	90

Application-specific logs (each has its own convention — worth knowing the common ones)	91
Finding a log location you don't already know	91
<code>logrotate</code> — Keeping Log Files From Growing Forever	91
How it works, conceptually	91
Where the config lives	91
Anatomy of a <code>logrotate</code> config	91
Testing a <code>logrotate</code> config without waiting for the schedule	92
Writing your own rule for a custom app	92
Practical Cheat Sheet	92

Part III — Networking & Security 95

Networking Troubleshooting Toolkit — "Why Can't This Reach That?" 95

Introduction	95
The Mental Model — Where Connections Actually Break	95
<code>ping</code> — Is It Reachable At All?	95
Reading the output	95
What <code>ping</code> tells you — and what it doesn't	96
<code>traceroute</code> — Where Does the Path Actually Break?	96
How it works, briefly	96
Reading the output	96
<code>mtr</code> — the better version for ongoing diagnosis	96
<code>dig</code> / <code>nslookup</code> — Is DNS the Problem?	96
<code>dig</code> — the more detailed, modern tool	96
Reading <code>dig</code> 's output	97
<code>nslookup</code> — older, still common, simpler output	97
Checking your actual configured resolver	97
Checking a Specific Port — Is the Service Actually Listening?	97
<code>tcpdump</code> — See the Actual Packets	98
A genuinely common diagnostic pattern	98
Reading basic <code>tcpdump</code> output	98
<code>curl -v</code> — When It's an Application-Layer Problem, Not a Network One	98
The Full Diagnostic Workflow	98
Quick Reference	99

SSH Remote Access From Outside Your Network — Complete Guide 100

Introduction 100

Two Ways to Do This 100

Install & Enable the SSH Server 100

Find Your IPs 100

Forward the Port on Your Router 101

Open the Port on the Server's Own Firewall 101

Connect From Outside 101

Connecting as Root vs. as a Regular User — Two Ways In 101

6.1 Method 1 — Connect as a Regular User, Then Elevate (Recommended) 101

6.2 Method 2 — Connect Directly as Root 102

6.3 Which One Should You Actually Use? 102

Your Public IP Keeps Changing — Fix It With Dynamic DNS 103

Security Hardening — Don't Skip This 103

Switch to key-based authentication, then disable passwords entirely 103

Disable root login 103

Change the default port 104

Restrict which users can SSH in at all 104

Install fail2ban 104

Rate-limit connections at the firewall too 104

Keep the system patched 104

The Safer Alternative: Skip Port Forwarding Entirely 104

Tailscale (recommended) 104

Cloudflare Tunnel 104

Reverse SSH tunnel via a cheap VPS 104

Troubleshooting 105

Quick Setup Checklist 105

Firewall Deep Dive — ufw vs iptables vs nftables 106

Introduction 106

How These Three Actually Relate 106

`ufw` — Beyond Just `allow` 106

<i>The basics, quickly</i>	106
<i>Real rule-writing</i>	106
`iptables` — The Legacy Standard, In Real Depth	108
<i>Core concepts</i>	108
<i>Rule syntax anatomy</i>	108
<i>Real rule-writing</i>	108
<i>Viewing & deleting rules</i>	109
<i>Making rules persistent (they're wiped on reboot otherwise)</i>	110
`nftables` — The Modern Replacement	110
<i>Core concepts — similar shape, different words</i>	110
<i>Basic setup</i>	110
<i>Writing rules — interactive vs. file-based</i>	110
<i>Sets — nftables' real advantage for larger rule sets</i>	111
<i>Viewing & managing rules</i>	111
Same Scenario, All Three Tools — Direct Comparison	112
Which One Should You Actually Use?	112
Quick Reference	113
General Server Hardening — Beyond SSH	114
Introduction	114
The Hardening Mental Model	114
Minimizing Attack Surface	114
<i>Audit what's actually running and listening</i>	114
<i>Disable services you don't need</i>	114
<i>Bind services to localhost when they don't need external access</i>	114
<i>Remove unused packages, don't just disable them</i>	114
<i>Close ports at the firewall too (defense in depth with the previous point)</i>	115
`unattended-upgrades` — Automatic Security Patching	115
<i>Install and configure</i>	115
<i>Key config file: `/etc/apt/apt.conf.d/50unattended-upgrades`</i>	115
<i>Whether to enable automatic reboots</i>	115
<i>Confirm it's actually enabled and working</i>	115
<i>Check what it's actually done</i>	115

<i>Dry-run test (see what it WOULD do, right now, without applying anything)</i>	115
Tuning `fail2ban` Properly	116
<i>Understanding the two files that matter</i>	116
<i>A properly tuned `jail.local`</i>	116
<i>Escalating bans for repeat offenders</i>	116
<i>Checking fail2ban's actual state</i>	116
<i>Testing a jail's regex against your actual logs before trusting it</i>	117
`auditd` — Knowing What Actually Happened	117
<i>Install and enable</i>	117
<i>Writing audit rules — `/etc/audit/rules.d/audit.rules`</i>	117
<i>Actually querying the audit log — this is the part that makes it useful</i>	117
Putting It Together — A Baseline Hardening Checklist	118
Linux User & Group Management — Complete Guide	119
Introduction	119
PART 1: Core Concepts	119
<i>How Linux Tracks Users</i>	119
<i>User Types</i>	119
<i>Primary vs. Supplementary Groups</i>	119
PART 2: User Commands	120
<i>Viewing User Info</i>	120
<i>Creating Users</i>	120
<i>Modifying Users — `usermod` (Deep Dive)</i>	120
<i>Deleting Users</i>	121
<i>Password & Account Aging</i>	121
PART 3: Group Commands	121
<i>Creating & Managing Groups</i>	121
<i>Managing Group Membership</i>	122
PART 4: File Permissions Tied to Users & Groups	122
<i>The Permission Triad</i>	122
<i>Special Permissions (Beyond rwx)</i>	122
<i>Access Control Lists (ACLs) — Beyond Owner/Group/Other</i>	123
PART 5: `sudo` — Privileged Access Without Full Root	123

PART 6: Practical Walkthrough — Common Scenarios	124
<i>Scenario 1: Onboard a new developer with sudo + Docker access</i>	124
<i>Scenario 2: Shared team directory where everyone's files land in the same group automatically</i>	124
<i>Scenario 3: Give one extra user read access to a file without changing its group</i>	124
<i>Scenario 4: Temporarily disable a departing employee's account without deleting anything</i>	124
Quick Reference Card	124

Linux Directory Permissions — Complete Guide 126

Introduction	126
PART 1: What `rwX` Actually Means on a Directory	126
<i>The classic gotcha</i>	126
<i>The other classic gotcha — write on a directory is powerful</i>	126
PART 2: Directory Permission Recipes	126
PART 3: Special Bits on Directories	127
<i>SGID on a Directory — Group Inheritance</i>	127
<i>Sticky Bit on a Directory — Restricted Deletion</i>	127
<i>Combining Both</i>	128
PART 4: `umask` — Default Permissions for New Directories & Files	128
PART 5: Auditing & Troubleshooting Directory Permissions	128
<i>Common real-world directory permissions (for reference)</i>	128
Quick Reference Card	129

Part IV — Servers & Automation 131

Docker & Containers — Full Guide 131

Introduction	131
What a Container Actually Is (Briefly)	131
Installing Docker	131
Images — Pulling, Building & Understanding Layers	131
<i>Pulling existing images</i>	131
<i>Building your own image — Dockerfile anatomy</i>	132
<i>Layers — why instruction order matters for build speed</i>	132
<i>`.dockerignore` — keep the build context small</i>	132
Containers — Running & Managing	133

<i>Restart policies</i>	133
<i>Interacting with a running container</i>	133
Volumes — Persisting Data Beyond a Container's Lifetime	133
<i>Three types</i>	133
Networks — How Containers Talk to Each Other and the World	134
<i>Default network types</i>	134
<i>Publishing a port — making a container reachable from outside</i>	134
<i>Custom networks — the right way for multi-container communication</i>	134
Docker Compose — Multi-Container Stacks Declaratively	135
<i>Anatomy of a `docker-compose.yml`</i>	135
<i>Core commands</i>	135
<i>The `depends_on` gotcha</i>	136
Practical AIOps Example — A Monitoring Stack via Compose	136
Debugging & Common Issues	137
Cleanup & Maintenance	137
Quick Reference	137
Ansible Basics — Write Once, Apply to Every Server	139
Introduction	139
What Makes Ansible Different	139
Installation	139
Inventory — Defining What "Every Server" Means	139
<i>Basic static inventory (INI-style)</i>	139
<i>YAML inventory (more common in real-world setups now)</i>	139
<i>Testing connectivity to your inventory</i>	140
Ad-Hoc Commands — One-Off Tasks Without a Playbook	140
Playbooks — The Real Unit of Work	140
<i>Anatomy of a basic playbook</i>	140
<i>Reading the structure</i>	141
<i>The idempotency payoff, made concrete</i>	141
Variables	141
<i>Inline in a playbook</i>	141
<i>`group_vars` and `host_vars` — the scalable pattern for real projects</i>	141

<i>Variable precedence (simplified, most to least specific wins)</i>	142
Templates — Dynamic Config Files with Jinja2	142
Handlers — Restart Only When Something Actually Changed	142
Conditionals & Loops	143
<i>Conditionals</i>	143
<i>Loops</i>	143
<i>Combining both</i>	143
Roles — Reusable, Shareable Structure	143
Ansible Vault — Handling Secrets Properly	144
Practical Example — A Real Hardening Playbook	144
Common Modules Reference	145
Running, Checking & Debugging	146
Quick Reference	146
Monitoring Stack Setup — Prometheus + Grafana From Zero	147
Introduction	147
Architecture — How the Pieces Fit	147
Two Installation Paths	147
Path A: Docker Compose Setup	147
<i>Step 1 — Directory structure</i>	147
<i>Step 2 — Prometheus config</i>	147
<i>Step 3 — Compose file</i>	148
<i>Step 4 — Launch it</i>	148
Path B: Native systemd Install (No Containers)	148
<i>Step 1 — Create a dedicated user (services shouldn't run as root)</i>	148
<i>Step 2 — Download and install Prometheus</i>	149
<i>Step 3 — Same `prometheus.yml` as above, placed at `/etc/prometheus/prometheus.yml`</i>	149
<i>Step 4 — systemd unit (direct application of your systemd deep-dive guide)</i>	149
<i>Step 5 — node_exporter, same pattern</i>	149
<i>Step 6 — Grafana (native)</i>	150
Verifying Prometheus Is Actually Collecting Data	150
Connecting Grafana to Prometheus	150
Basic PromQL — Enough to Be Dangerous	150

Alerting — Prometheus Rules + Alertmanager	151
Step 1 — Define alert rules	151
Step 2 — Alertmanager, for actual notifications	152
Securing the Stack	152
Quick Reference	152
tmux & screen — Sessions That Survive a Disconnect	154
Introduction	154
Why This Matters, Concretely	154
tmux — The Modern Standard	154
Core concepts	154
The prefix key	154
Starting & basic session management	154
Detaching (the whole point)	154
Windows (tabs within a session)	155
Panes (splits within a window)	155
Copy mode — scrolling back and copying text	155
Configuration — ~/.tmux.conf	155
screen — The Older Alternative, Still Worth Knowing	156
Basic session management	156
Detaching	156
Windows	156
Basic splits (screen's split support is more limited than tmux's)	157
tmux vs screen — Direct Comparison	157
Practical Workflow — Tying This Into Your Existing Guides	157
Quick Reference	158
Linux for Data Engineers — Comprehensive Overview	159
Introduction	159
1 ■■■ ■■■■■ ■■■■■■■ ■■■■■■■ ■■■■■■■	159
2 ■■■ ■■■■■ ■■■■■■■■■: ■■ ■■■■■■■ ■■■■■■■ ■■■ ■■■ ■■■■■ ■■■■■	159
■. ■■■■■ ■■■■■ ■■■■■■■ "■■■■■■ ■■■" (Bell Labs)	159
■. ■■■■■■■ ■■■■■■■ ■■■■■ ■■■■■■■	159
■. ■■■■■ ■■■■■ (Linux Kernel)	160

<i>Getting IP & SSH</i>	167
Minimum Requirements for RHEL 9	168
Access the Command Line	168
<i>Running Commands</i>	168
<i>Example</i>	168
<i>Command Syntax: Right vs. Wrong</i>	168
<i>Shell Shortcuts</i>	169
Manage Files from the Command Line	169
<i>The File System Hierarchy</i>	169
<i>Important Directories</i>	169
<i>File Types</i>	170
<i>Rules for Naming Files</i>	170
<i>Absolute Path vs. Relative Path</i>	170
Inodes, Hard Links, and Soft Links	171
<i>What Is an Inode?</i>	171
<i>Metadata Stored in an Inode</i>	171
<i>Soft Link vs. Hard Link</i>	171
Pattern Matching (Regex)	172
<i>Shell Pattern-Matching Symbols</i>	172
<i>Regular Expressions</i>	172
Key Takeaways	173
Users & Privileges (Preview)	173
Day 2 — Get Help, Manage Text Files, and Administer Local Users	174
Learning Objectives	174
Get Help in Red Hat Enterprise Linux	174
<i>Manual Page Sections</i>	174
<i>Structure of a Manual Page</i>	174
Create, View, and Edit Text Files	175
<i>Input-Output Redirection</i>	175
<i>Pipelines</i>	175

Redirection Operators

175

Example

175

The vim Text Editor

175

Variables in Linux

176

Word Count & Pipelines Deep Dive

177

What it does

177

Getting just ONE of these numbers

177

Combining wc with other commands

177

Manage Local Users and Groups

178

What Is a User?

178

UID Ranges

178

The /etc/passwd File

178

The /etc/group File

178

Primary Group and Supplementary Group

178

Superuser Privilege: su vs. sudo

179

The /etc/shadow File

179

Configuring Password Aging with chage

179

Example

179

Key Takeaways

180

Switching Users — su

180

Comparison

180

visudo — Safely Managing Sudo Permissions

181

The file behind sudo

181

Why not just edit it with vim directly?

181

Vim Scripting & External Commands

181

Vim Script: Find and Replace

181

Execute External Command in Vim

181

Vim Options

181

Variables in Shell

182

Mathematics in Shell

182

Day 3 — Coming Soon

183

Coming Soon	183
Part VI — Hands-On Labs & Exercises	184
Lab 1 — Solved	184
Linux Lab 1 — Solved Exercises	184
1. less vs more	184
<i>Example</i>	<i>184</i>
2. cat vs more	184
3. rm vs rmdir	184
<i>Examples</i>	<i>185</i>
4. Create and manipulate the hierarchy	185
<i>Create it</i>	<i>185</i>
4a. Remove dir11 in one step	185
<i>What happened?</i>	<i>185</i>
4b. Remove dir12 using rmdir -p	185
4c. Absolute and relative paths for mycv	185
5. Copy /etc/passwd to the home directory as mypasswd	186
6. Rename mypasswd to oldpasswd	186
7. Four ways to go from /usr/bin to the home directory	186
<i>Method 1 — cd with no argument</i>	<i>186</i>
<i>Method 2 — Use ~</i>	<i>186</i>
<i>Method 3 — Use \$HOME</i>	<i>186</i>
<i>Method 4 — Use the absolute path</i>	<i>186</i>
8. List commands in /usr/bin beginning with w	187
9. Display the first 4 lines of /etc/passwd	187
10. Display the last 7 lines of /etc/passwd	187
Quick Command Reference	187
Key Takeaways	188
Lab 2 — Solved	189
Linux Lab 2 — Solved Exercises	189
1. Display the passwd command and file manual pages sequentially	189

2. Display the manual page for the passwd file	189
3. Find commands whose manual pages contain passwd	189
4. Create a user account	189
<i>Create the account</i>	189
<i>Set the password</i>	190
<i>Verify</i>	190
5. Search manual pages mentioning password	190
6. Create notes.txt using Vim	190
7. Display notes.txt	190
8. Append AIOPS without opening an editor	191
<i>Why >>?</i>	191
9. Redirect ls -l /etc into etc_listing.txt	191
<i>What happened?</i>	191
10. Safely inspect which users have sudo privileges	191
11. Switch to root with su -	192
12. Create steps.txt in Vim and save without exiting	192
13. Search for the Vim line and replace raining with Course	193
14. Copy the current line, paste it below, then save and exit	193
<i>Combined command sequence</i>	194

Vim Quick Reference **194**

Linux Command Reference **194**

Key Takeaways **195**

Part VII — Appendix: Command Reference Bank **196**

<i>Categories at a Glance</i>	196
---	-----

File & Directory Navigation **196**

File Manipulation **197**

Viewing & Editing Text **197**

System & Process Management **198**

Networking & Permissions **198**

Archiving & Compression 198

Version Control (Git) 199

Containers (Docker) 199

Alphabetical Bank (excerpt) 199

At a Glance — 22 Guides • 36,359 words • 4 Parts

Part	Guides
Part I — Start Here: Foundations Core concepts every Linux user must know before touching a server.	4 guides
Part II — System Operations Disks, boot, systemd, processes & logs — keep the machine healthy.	6 guides
Part III — Networking & Security From ping to SSH to firewalls and user management.	6 guides
Part IV — Servers & Automation Containers, automation & monitoring at scale.	6 guides

Part I — Start Here: Foundations

Core concepts every Linux user must know before touching a server. — 4 guides

Linux System & Command Rules — Reference Guide

The mental model behind Linux — everything is a file, one tree, plain-text config. Read this first.

Level: Beginner • Words: 2,385 • Sections: 15
Commands: apt, chgrp, chmod, chown, echo, find +12 more

Introduction

A reference for how the Linux system is built and how commands are structured — not a command dump, but the underlying rules that make the rest predictable.

Core Mental Model

- **Everything is a file.** Devices, processes, sockets, even kernel interfaces (`/proc`, `/sys`) are represented as files.
- **Single unified tree.** There's one root (`/`) — no drive letters. Everything, including other disks/partitions, gets mounted into this tree.
- **Case-sensitive.** `File.txt` ≠ `file.txt` ≠ `FILE.txt`.
- **Extensions are cosmetic.** `.sh`, `.txt`, `.py` are conventions for humans — the OS determines file type by content/permissions, not name.
- **Multi-user by design.** Every process runs as a user; every file is owned by a user and group.
- **Configuration is text.** Almost every system setting lives in a plain-text file (usually under `/etc`), not a binary registry.

Filesystem Hierarchy Rules (FHS)

Path	Purpose
<code>/</code>	Root of everything
<code>/bin</code> , <code>/usr/bin</code>	Essential user command binaries
<code>/sbin</code> , <code>/usr/sbin</code>	System admin binaries (often need root)
<code>/etc</code>	System-wide configuration files

Path	Purpose
<code>/home</code>	Personal user directories
<code>/root</code>	Root user's home (separate from <code>/home</code>)
<code>/var</code>	Variable data: logs, caches, spool files
<code>/tmp</code>	Temporary files, usually cleared on reboot
<code>/opt</code>	Optional/third-party software
<code>/lib, /usr/lib</code>	Shared libraries
<code>/dev</code>	Device files (disks, terminals, etc.)
<code>/proc</code>	Virtual filesystem exposing kernel/process info live
<code>/sys</code>	Virtual filesystem exposing kernel/device info live
<code>/mnt, /media</code>	Mount points for filesystems/removable media
<code>/boot</code>	Kernel, bootloader files

TIP

Rule of thumb: if it's a log → `/var/log`; if it's a config → `/etc`; if it's a live kernel stat → `/proc` or `/sys`.

Path Rules

- **Absolute path:** starts with `/` — always resolved from root (`/home/solo/project`).
- **Relative path:** resolved from your current directory (`./project`, `./script.sh`).
- `.` = current directory, `..` = parent directory, `~` = current user's home directory.
- Spaces and special characters (`$`, `*`, `&`, `!`) must be escaped (`\`) or the whole path quoted (`"my file.txt"`).
- Hidden files/directories start with `.` (e.g. `.bashrc`) — invisible to plain `ls`, shown with `ls -a`.

Command Syntax Rules

General anatomy:

```
command [options] [arguments]
```

- **Short options:** single dash, single letter — `-a`, `-l`. Can often be combined: `-la` = `-l -a`.
- **Long options:** double dash, full word — `--all`, `--verbose`.
- `--` signals "end of options" — everything after is treated as a literal argument, even if it looks like a flag. Useful for filenames starting with `-`.
- **Case matters** in flags too: `-R` (recursive, e.g. `chmod`) is not the same as `-r`.
- Every standard command supports `--help`; every installed command has a manual: `man <command>`.
- Exit codes: `0` = success, non-zero = failure. Check with `echo $?` right after a command.

Permissions Rules

Every file/directory has three permission sets: **owner (u)**, **group (g)**, **others (o)** — each with **read (r)**, **write (w)**, **execute (x)**.

```
-rwxr-xr-- 1 solo devs 4096 Aug 21 file.sh
■■■■■■■■■■
■ u g o
■ file type (- file, d directory, l symlink)
```

TIP**Numeric (octal) values:**

Permission	Value
read	4
write	2
execute	1

Sum them per group: `rwX` = 7, `rw-` = 6, `r-X` = 5, `r--` = 4.

```
chmod 755 script.sh      # rwxr-xr-x
chmod u+x script.sh      # symbolic: add execute for owner
chown solo:devs file      # change owner:group
chgrp devs file           # change group only
```

TIP**Special bits:**

- **setuid** (`4000`) — run binary as the file's owner, not the caller.
- **setgid** (`2000`) — run as file's group; on a directory, new files inherit that group.
- **sticky bit** (`1000`) — in a shared directory (e.g. `/tmp`), only the file's owner can delete/rename it.

Default permissions: governed by `umask` — the value subtracted from the system default (666 for files, 777 for dirs).

Users & Groups Rules

- **root (UID 0)** — unrestricted; every other user is `UID ≥ 1000` (varies by distro).
- Identity lives in `/etc/passwd` (users), `/etc/shadow` (password hashes), `/etc/group` (groups).
- **sudo** runs a single command as root (or another user), logged and password-gated.
- **su** switches your entire shell session to another user.
- Every user has one **primary group** and can belong to multiple **secondary groups** — this is what group-based permission checks use.

6.1 Checking who you currently are

Before switching anything, know where you stand:

```
whoami      # prints your current username
id          # prints your UID, GID, and every group you belong to
echo $UID   # 0 = root, anything else = regular user
```

Your shell prompt is also a live indicator: it typically ends in `$` for a regular user and `#` for root.

6.2 Step-by-Step: Becoming Root — Method 1 (`su`)

`su` ("switch user") drops you into a full new shell session as another user — you stay in that session until you explicitly exit it.

TIP**Step 1 — Run the command:**

```
su -
```

The trailing `-` matters: it loads root's own environment (its `$PATH`, its home directory `/root`, its shell config) instead of keeping your current user's environment. Without the `-`, you become root but keep your old user's environment, which

can cause confusing `$PATH`/permission mismatches.

Step 2 — Enter root's password (not your own password — `su` asks for the target account's password):

```
Password: *****
```

TIP

Step 3 — Confirm you're root:

```
whoami    # should now print: root
```

TIP

Step 4 — Exit back to your normal user when done:

```
exit  
# or press Ctrl+D
```

TIP

Downside: `su` requires knowing root's actual password, which many modern distros (Ubuntu included) disable by default — root has no password set at all out of the box, so plain `su -` will fail until you explicitly set one with `sudo passwd root`.

6.3 Step-by-Step: Becoming Root — Method 2 (`sudo`)

`sudo` is the modern, more common approach — it uses your own password (not root's) and only works if your user is listed in `/etc/sudoers` or belongs to the `sudo/wheel` group.

TIP

Step 1 — Confirm you have sudo rights:

```
groups    # look for "sudo" (Debian/Ubuntu) or "wheel" (RHEL/Fedora) in the output
```

TIP

Step 2a — Run a single command as root (most common day-to-day use):

```
sudo apt update
```

You'll be prompted for **your own** password, not root's. `sudo` caches that authorization for a few minutes, so repeated `sudo` commands in a row won't keep re-prompting.

TIP

Step 2b — Get a full root shell instead of one command at a time:

```
sudo -i  
# or equivalently:  
sudo su -
```

`sudo -i` behaves like `su -` — full root environment, root's home directory — but authenticated with your own password instead of root's.

TIP**Step 3 — Confirm:**

```
whoami    # root
```

TIP**Step 4 — Exit back to your normal user:**

```
exit
```

6.4 Which Method Should You Actually Use?

Situation	Use
One-off admin command (install a package, edit a config)	<code>sudo <command></code>
Need several root commands in a row	<code>sudo -i</code> (or <code>sudo su -</code>)
Root has a set password and you specifically want classic <code>su</code> behavior	<code>su -</code>
Best practice on any modern personal/server system	<code>sudo</code> — it's logged (check <code>/var/log/auth.log</code>), tied to your own identity, and doesn't require a separate root password to exist at all

TIP

Golden rule: don't stay logged in as root longer than you need to. Run what you need, then `exit` back to your normal user — staying in a root shell out of convenience is how a typo turns into a system-wide disaster (see the `chmod -R 777 /` and `rm -rf` scenarios in the failure-scenarios guide).

Process Rules

- Every process has a **PID** (process ID) and a **PPID** (parent process ID) — Linux processes form a tree, rooted at PID 1 (`systemd` on modern distros).
- States:** running, sleeping, stopped, zombie (finished but not yet reaped by its parent).
- Foreground vs background:** `&` at the end backgrounds a job; `jobs` lists them; `fg/bg` bring them forward/resume them; `Ctrl+Z` suspends the current one.
- Signals** are how processes are told to do something — not killed by force by default:

Signal	Meaning
<code>SIGHUP</code> (1)	Hangup / reload config
<code>SIGINT</code> (2)	Interrupt (Ctrl+C)
<code>SIGTERM</code> (15)	Polite "please exit" (default for <code>kill</code>)
<code>SIGKILL</code> (9)	Force-kill, cannot be caught or ignored

```
kill -15 1234    # ask nicely
kill -9 1234     # force it, last resort
pkill -f "process_name"
```

I/O Redirection & Pipe Rules

Every process starts with three open streams:

FD	Stream	Symbol
0	stdin	<
1	stdout	>, >>
2	stderr	2>

```
command > out.txt      # overwrite stdout to file
command >> out.txt     # append stdout to file
command 2> err.txt    # redirect only errors
command &> all.txt     # redirect stdout AND stderr
command < input.txt   # feed a file in as stdin
```

Pipes (|) chain commands — stdout of the left becomes stdin of the right:

```
ps aux | grep python | awk '{print $2}'
```

xargs bridges commands that expect arguments, not stdin:

```
find . -name "*.log" | xargs rm
```

Wildcards & Globbing Rules

Pattern	Matches
*	Any number of characters (not hidden files unless combined with <code>.</code>)
?	Exactly one character
[abc]	Any one of a, b, or c
[0-9]	Any digit
{a,b,c}	Brace expansion — literal set: <code>file{1,2,3}.txt</code> → <code>file1.txt file2.txt file3.txt</code>

Globbing is expanded **by the shell**, before the command ever sees it — not by the command itself.

Environment & Shell Rules

- **Environment variables** are key=value pairs available to a process and its children: `export VAR=value`.
- `$PATH` is an ordered, colon-separated list of directories the shell searches for executables — first match wins. `which <cmd>` shows which one gets used.
- **Shell startup file load order** (bash, login shell): `/etc/profile` → `~/.bash_profile` (or `~/.profile`) → for interactive non-login shells: `~/.bashrc`.
- **Aliases** are shell-only shortcuts (`alias ll='ls -la'`) — they don't exist for scripts run by other programs.
- Quoting matters: `"$VAR"` expands the variable; `'$VAR'` treats it as a literal string.

Package Management Rules

Family	Distros	Low-level	High-level
Debian	Ubuntu, Debian	<code>dpkg</code>	<code>apt</code> , <code>apt-get</code>
Red Hat	RHEL, Fedora, CentOS	<code>rpm</code>	<code>dnf</code> , <code>yum</code>

- High-level tools (`apt`, `dnf`) resolve **dependencies** automatically; low-level tools (`dpkg`, `rpm`) install a single package file and expect dependencies to already be present.
- Always `apt update` (refresh package index) before `apt install` (actually install) — these are two different, sequential steps.

Service Management Rules (systemd)

- Modern distros use **systemd** (PID 1) to manage services via **unit files**, typically in `/etc/systemd/system/` or `/lib/systemd/system/`.

```
systemctl start nginx      # start now
systemctl stop nginx      # stop now
systemctl enable nginx     # start automatically on boot
systemctl status nginx     # current state
journalctl -u nginx -f    # live-tail that service's logs
```

- enable** and **start** are independent — enabling doesn't start it now, starting doesn't survive a reboot. Do both if you want "on now, and on every boot."

Networking Rules

- Ports below **1024** are "privileged" — binding to them requires root (or a capability grant).
- ss -tulnp** — show listening ports and the process using them (modern replacement for **netstat**).
- ssh user@host** — remote shell; key-based auth (`~/.ssh/id_ed25519`) is the standard over passwords.
- curl / wget** — fetch over HTTP(S); **curl** is better for APIs (headers, methods), **wget** for straight downloads.

13.1 Viewing & Configuring Network Interfaces — Two Ways

There are two tool families for this: the legacy **net-tools** package (**ifconfig**, **route**) and the modern **iproute2** package (**ip**). Modern distros ship **ip** by default; **ifconfig** often needs manual install now. Both are shown here since you'll run into either depending on the machine.

TIP

Check which you have installed:

```
which ifconfig    # empty output = not installed
which ip          # this one is almost always present
sudo apt install net-tools -y    # installs ifconfig if it's missing
```

Method 1 — **ifconfig** (legacy, still common on older systems)

TIP

Step 1 — View all active interfaces:

```
ifconfig
```

TIP

Step 2 — View all interfaces, including inactive ones:

```
ifconfig -a
```

TIP

Step 3 — View just one interface:

```
ifconfig eth0
```

TIP

Step 4 — Bring an interface up or down:

```
sudo ifconfig eth0 up
sudo ifconfig eth0 down
```

TIP**Step 5 — Manually assign an IP address:**

```
sudo ifconfig eth0 192.168.1.50 netmask 255.255.255.0
```

TIP**Step 6 — Check the routing table (companion command):**

```
route -n
```

Method 2 — **ip** (modern, the current standard)

TIP**Step 1 — View all interfaces and addresses:**

```
ip a
# or the more explicit form:
ip addr show
```

TIP**Step 2 — View just one interface:**

```
ip addr show eth0
```

TIP**Step 3 — Bring an interface up or down:**

```
sudo ip link set eth0 up
sudo ip link set eth0 down
```

TIP**Step 4 — Manually assign an IP address (note: CIDR notation, not a separate netmask):**

```
sudo ip addr add 192.168.1.50/24 dev eth0
```

TIP**Step 5 — Remove an assigned IP:**

```
sudo ip addr del 192.168.1.50/24 dev eth0
```

TIP**Step 6 — Check the routing table:**


```
ip route
```

TIP

Important distinction for both methods: changes made this way are **temporary** — they're wiped on reboot or interface restart. For a permanent change, edit your distro's network config instead (Netplan's YAML files on modern Ubuntu, `/etc/network/interfaces` on older Debian-based systems, or `nmcli/nmtui` if NetworkManager is running the show).

13.2 Command Equivalence Table

Task	<code>ifconfig</code> (legacy)	<code>ip</code> (modern)
Show all interfaces	<code>ifconfig</code>	<code>ip a</code>
Show one interface	<code>ifconfig eth0</code>	<code>ip addr show eth0</code>
Bring interface up	<code>ifconfig eth0 up</code>	<code>ip link set eth0 up</code>
Bring interface down	<code>ifconfig eth0 down</code>	<code>ip link set eth0 down</code>
Assign an IP	<code>ifconfig eth0 <ip> netmask <mask></code>	<code>ip addr add <ip>/<cidr> dev eth0</code>
View routing table	<code>route -n</code>	<code>ip route</code>

TIP

Which to actually learn: default to `ip` — it's the maintained tool and what you'll find on nearly every current server. Recognize `ifconfig` syntax for when you land on an older box that still has it, but don't build new habits around it.

Golden Rules & Common Gotchas

- **`rm -rf` has no undo.** Double-check `pwd` and the target path before running anything destructive with `-rf`, especially as root.
- **Never run untrusted scripts with `sudo`** without reading them first.
- **Quote your variables** in scripts ("`$var`") — unquoted variables break on spaces/globs.
- **Symlinks vs hardlinks:** a symlink (`ln -s`) is a pointer to a path and breaks if the target moves; a hardlink (`ln`) points to the same inode and survives the original being deleted.
- **A command that "isn't found"** is almost always a `$PATH` issue, not a missing install — check with `which` or `type`.
- **Trailing slashes matter** for some commands (notably `rsync`) — `dir/` vs `dir` changes whether the directory itself or its contents are the target.

Important Files Worth Generating on a Linux / Dev System

/etc, /var, /proc — the key files every admin reads and maintains, and what breaks when they vanish.

Level: Beginner • Words: 754 • Sections: 11
Commands: alias, export

Introduction

A priority-ranked list of config files that are worth deliberately creating rather than leaving at system defaults — organized by what they control.

TIP

Priority key: ● Essential essential · ● Recommended strongly recommended · ■ Situational situational

Shell Configuration

File	Priority	Purpose
<code>~/.bashrc</code>	● Essential	Aliases, functions, <code>\$PATH</code> additions, prompt customization — loaded for every interactive non-login shell
<code>~/.bash_profile</code> or <code>~/.profile</code>	● Recommended	Loaded once at login — usually just sources <code>.bashrc</code> and sets login-only environment vars
<code>~/.bash_aliases</code>	■ Situational	Optional split-out file for aliases, sourced from <code>.bashrc</code> to keep it clean
<code>~/.zshrc</code>	■ Situational	Same role as <code>.bashrc</code> , if you're on zsh instead

```
# ~/.bashrc essentials
export PATH="$HOME/.local/bin:$PATH"
alias ll='ls -la'
alias gs='git status'
export EDITOR=vim
```

Editor Configuration

File	Priority	Purpose
<code>~/.vimrc</code>	● Essential	Vim defaults — line numbers, indentation, search behavior (full config in your vim-nano guide)
<code>~/.nanorc</code>	● Recommended	Nano defaults — syntax highlighting, tab behavior

Git Configuration

File	Priority	Purpose
<code>~/.gitconfig</code>	● Essential	Global identity, default editor, aliases — set once per machine
<code>.gitignore</code> (per project)	● Essential	Prevents build artifacts, secrets, and local config from ever being committed
<code>.gitattributes</code> (per project)	● Recommended	Normalizes line endings across contributors, marks files as binary, controls diff behavior
<code>~/.gitignore_global</code>	■ Situational	OS/editor cruft (<code>.DS_Store</code> , <code>.vscode/</code>) ignored across all your repos, not just one

```
# ~/.gitconfig
[user]
  name = SOLO
  email = your@email.com
[core]
  editor = vim
[init]
  defaultBranch = main
```

```
# .gitignore – Flutter project baseline
.dart_tool/
.packages
build/
.env
*.log
```

SSH Configuration

File	Priority	Purpose
<code>~/.ssh/config</code>	● Essential	Per-host shortcuts (<code>ssh myserver</code> instead of a long flag string), key selection, connection reuse
<code>~/.ssh/authorized_keys</code>	● Essential	On a server — which public keys are allowed to log in as that user
<code>~/.ssh/id_ed25519 (+ .pub)</code>	● Essential	Your actual keypair — generate with <code>ssh-keygen -t ed25519</code> , never share the private half

```
# ~/.ssh/config
Host myserver
  HostName 192.168.1.100
  User solo
  IdentityFile ~/.ssh/id_ed25519
  Port 22
```

Environment & Secrets

File	Priority	Purpose
<code>.env</code> (per project)	● Essential	API keys, DB URLs, local secrets — must be in <code>.gitignore</code> , never committed
<code>.env.example</code>	● Recommended	Committed template showing which variables are needed, with placeholder values
<code>~/.netrc</code>	■ Situational	Auto-login credentials for tools like <code>curl/ftp</code> — keep permissions at <code>600</code>

System-Level Config (root-owned — edit with care, see your failure-scenarios guide)

File	Priority	Purpose
<code>/etc/hosts</code>	● Recommended	Manual hostname → IP mappings, useful for local dev domains
<code>/etc/fstab</code>	● Recommended	Defines what gets mounted at boot — get this wrong and the system can fail to boot
<code>/etc/hostname</code>	■ Situational	The machine's hostname

File	Priority	Purpose
<code>/etc/environment</code>	■ Situational	System-wide environment variables (not shell-specific, applies to all users)
<code>/etc/ssh/sshd_config</code>	● Recommended	Server-side SSH daemon config — disable password auth here once keys work
<code>/etc/sudoers</code>	● Essential (edit only via <code>visudo</code>)	Who can run what as root — a syntax error here can lock you out of <code>sudo</code> entirely

Service / Automation Files

File	Priority	Purpose
<code>/etc/systemd/system/myapp.service</code>	● Recommended	Defines a custom background service — start on boot, auto-restart on crash
Crontab (<code>crontab -e</code>)	● Recommended	Scheduled recurring jobs — not a manually-created file path, edited through the command
<code>~/.config/systemd/user/*.service</code>	■ Situational	User-level (no root needed) systemd services

```
# /etc/systemd/system/myapp.service
[Unit]
Description=My App
After=network.target

[Service]
ExecStart=/usr/bin/python3 /opt/myapp/main.py
Restart=on-failure
User=solo

[Install]
WantedBy=multi-user.target
```

Project-Level Dev Files

File	Priority	Purpose
<code>README.md</code>	● Essential	What the project is, how to run it — the first thing anyone (including future you) reads
<code>.gitignore</code>	● Essential	(see above)
<code>LICENSE</code>	● Recommended	Legal terms for reuse — matters the moment a repo goes public
<code>CHANGELOG.md</code>	■ Situational	Human-readable version history, especially valuable once you're shipping releases
<code>.editorconfig</code>	● Recommended	Enforces consistent indentation/line-endings across every editor/IDE a contributor might use
<code>Dockerfile</code>	● Recommended	Reproducible build/run environment for the project
<code>docker-compose.yml</code>	● Recommended	Multi-service local dev stack (app + DB + cache in one command)
<code>.dockerignore</code>	● Recommended	Keeps build context small — mirrors <code>.gitignore</code> 's role but for Docker builds
<code>pubspec.yaml</code>	● Essential (Flutter)	Dependencies, assets, and metadata for any Flutter project
<code>analysis_options.yaml</code>	● Recommended (Flutter/Dart)	Lint rules — catches style/bug issues before they ship

AIOps / Automation-Specific

Relevant given your current track:

File	Priority	Purpose
<code>prometheus.yml</code>	● Recommended	Defines what Prometheus scrapes and how often — the backbone of most monitoring setups
Grafana provisioning YAML (<code>dashboards.yml</code> , <code>datasources.yml</code>)	■ Situational	Version-controls your dashboards instead of clicking them into existence by hand
Ansible inventory (<code>inventory.ini</code>) + playbooks (<code>*.yaml</code>)	■ Situational	Declarative server config — the "generate once, apply everywhere" pattern AIOps leans on heavily
n8n workflow export (<code>workflow.json</code>)	■ Situational	Lets you version-control and re-import your automation flows instead of them living only in the n8n UI

Quick Priority Summary

TIP

Set these up on day one of any new machine: `.bashrc`, `.vimrc`, `.gitconfig`, `~/.ssh/config` + keypair, `/etc/sudoers` awareness (don't need to edit it, just know it exists).

TIP

Set these up on day one of any new project: `README.md`, `.gitignore`, `.env` + `.env.example`.

TIP

Set these up the moment you deploy something that should survive a reboot or crash: a systemd unit file, and if it needs monitoring, `prometheus.yml`.

Vim / Vi & Nano — The Complete Guide

Survive and then thrive in vim and nano — the editors you will find on every server you ever SSH into.

Level: Beginner • Words: 2,151 • Sections: 27

Commands: nano, nano/vim, type, vim

The Core Idea: Modal Editing

Vim is not like other editors — it has **modes**, and the same key does completely different things depending on which mode you're in. This is the single most important concept to internalize before anything else makes sense.

Mode	Purpose	How to enter
Normal	Navigate, delete, copy, run commands (default mode)	<code>Esc</code> from any other mode
Insert	Type text like a normal editor	<code>i</code> , <code>a</code> , <code>o</code> , etc. from Normal
Visual	Select text (char/line/block)	<code>v</code> , <code>V</code> , <code>Ctrl+v</code> from Normal
Command-line	Run <code>:</code> commands (save, quit, search/replace)	<code>:</code> from Normal
Replace	Overtyping existing text	<code>R</code> from Normal

TIP

Golden rule: `Esc` always brings you back to Normal mode. When lost, hit `Esc`.

Starting & Exiting

```
vim file.txt          # open (or create) a file
vim +42 file.txt      # open at line 42
vim +/pattern file.txt # open and jump to first match of pattern
vimdiff file1 file2   # open two files side-by-side, highlighting differences
```

TIP

Exiting (all typed in Normal mode, starting with `:`):

Command	Effect
<code>:w</code>	Save (write)
<code>:w newname.txt</code>	Save as a new file
<code>:q</code>	Quit (fails if unsaved changes)
<code>:q!</code>	Quit, discard changes
<code>:wq</code> or <code>:x</code>	Save and quit
<code>ZZ</code>	Save and quit (Normal mode shortcut, no <code>:</code>)
<code>ZQ</code>	Quit without saving
<code>:wqa</code>	Save and quit all open buffers/windows

Normal Mode — Navigation

Basic movement

Key	Moves
<code>h j k l</code>	left / down / up / right
<code>0</code>	start of line (column 0, ignores indentation)
<code>^</code>	first non-blank character of line
<code>\$</code>	end of line
<code>gg</code>	first line of file
<code>G</code>	last line of file
<code>42G</code> or <code>:42</code>	go to line 42

Word-wise movement

Key	Moves
<code>w</code>	start of next word
<code>W</code>	start of next WORD (whitespace-delimited, ignores punctuation splits)
<code>b</code>	back to start of previous word
<code>B</code>	back to start of previous WORD
<code>e</code>	end of current/next word
<code>E</code>	end of current/next WORD

Screen movement

Key	Moves
Ctrl+f	page forward
Ctrl+b	page backward
Ctrl+d	half page down
Ctrl+u	half page up
H	top of visible screen
M	middle of visible screen
L	bottom of visible screen
zz	center screen on cursor line

Character search on current line

Key	Effect
f x	jump to next x on this line
F x	jump to previous x on this line
t x	jump to just before next x
T x	jump to just after previous x
;	repeat last f/F/t/T
,	repeat last f/F/t/T reversed

TIP

Count prefix rule: almost any movement can be prefixed with a number to repeat it — **5j** moves down 5 lines, **3w** moves 3 words forward.

Insert Mode — Entering It

Key	Effect
i	insert before cursor
a	insert after cursor (append)
I	insert at start of line
A	insert at end of line
o	open new line below, enter insert mode
O	open new line above, enter insert mode
Esc	return to Normal mode

Editing Commands (Normal Mode)

Vim editing commands follow a grammar: **[count] operator [count] motion**. Once you know the operators and motions, they combine freely.

Core operators

Operator	Meaning
d	delete
c	change (delete + enter insert mode)
y	yank (copy)
> / <	indent / un-indent

Combine with a motion

```

dw → delete to next word
d$ → delete to end of line
d0 → delete to start of line
dd → delete whole line (doubling the operator = whole line)
cw → change to next word (delete + insert)
cc → change whole line
yy → yank whole line
y$ → yank to end of line
3dd → delete 3 lines

```

Single-key shortcuts (no motion needed)

Key	Effect
x	delete character under cursor
X	delete character before cursor
D	delete to end of line (= d\$)
C	change to end of line (= c\$)
Y	yank whole line (= yy)
s	delete char, enter insert (substitute)
S	delete line, enter insert
r x	replace single char under cursor with x
R	enter Replace mode (overtyping until Esc)
J	join current line with next
~	toggle case of character under cursor

Paste

Key	Effect
p	paste after cursor / below line
P	paste before cursor / above line

Undo / redo

Key	Effect
u	undo
Ctrl+r	redo
.	repeat last change — extremely powerful, use it constantly

Text Objects

Text objects let operators target a logical unit rather than a raw motion — this is what separates fluent Vim use from beginner Vim use.

Pattern: **operator + i/a + object**

- **i** = "inner" (just the content)
- **a** = "around" (content + surrounding delimiter/whitespace)

Example	Effect
diw	delete inner word (word only)
daw	delete a word (word + trailing space)
ci"	change inside double quotes
ca(	change including the parentheses
di{	delete inside { }
dip	delete inner paragraph
dap	delete a paragraph (incl. trailing blank line)
dit	delete inside an HTML/XML tag

Visual Mode

Key	Enters
v	character-wise visual
V	line-wise visual
Ctrl+v	block-wise visual (select a rectangular column)

Once in visual mode, move the cursor to extend the selection, then apply an operator:

```
v3w d    → select 3 words, delete them
V j j d  → select 3 lines, delete them
Ctrl+v, move down, I, type text, Esc → insert text at same column on multiple lines
```

gv reselects the last visual selection.

Search & Replace

Searching

Command	Effect
/pattern	search forward
?pattern	search backward
n	repeat search, same direction
N	repeat search, opposite direction
*	search forward for word under cursor
#	search backward for word under cursor

Find & replace (Ex command)

```

:s/old/new/      → replace first match on current line
:s/old/new/g     → replace all matches on current line
:%s/old/new/g    → replace all matches in entire file
:%s/old/new/gc   → same, but confirm each replacement
:5,10s/old/new/g → replace only within lines 5-10
:%s/\<foo\>/bar/g → replace whole word "foo" only (word boundaries)

```

Vim search patterns support regex — `.` any char, `*` zero-or-more, `\d` digit, `^/$` line anchors, `\(...\)` grouping.

Registers (Clipboard System)

Vim has multiple named "clipboards" called registers.

Register	Contents
<code>""</code>	default/unnamed register — used automatically by <code>y/d/p</code>
<code>"0</code>	last yank (specifically — not overwritten by delete)
<code>"1-"9</code>	rolling history of deletes
<code>"a-"z</code>	manually named registers you control
<code>"+</code>	system clipboard (cross-application copy/paste)

```

"ayy → yank current line into register a
"ap  → paste from register a
"+y  → yank into system clipboard
"+p  → paste from system clipboard

```

Marks

Marks let you bookmark a position and jump back to it.

```

ma → set mark "a" at cursor position
`a → jump to exact position of mark a
'a → jump to start of line containing mark a
`` → jump to position before last jump (great for "go back")

```

Macros

Record a sequence of keystrokes and replay it — ideal for repetitive edits across many lines.

```

qa → start recording into register a
... → do your edits
q → stop recording
@a → replay the macro once
5@a → replay it 5 more times
@@ → repeat the last macro played

```

Buffers, Windows & Tabs

These are three different concepts in Vim and are often confused:

- **Buffer** = a file loaded in memory
- **Window** = a viewport showing a buffer (you can have several windows showing different — or the same — buffers)
- **Tab** = a collection of windows

Command	Effect
<code>:e file.txt</code>	open file into a new buffer

Command	Effect
<code>:ls</code> or <code>:buffers</code>	list open buffers
<code>:bn</code> / <code>:bp</code>	next / previous buffer
<code>:b 3</code>	switch to buffer number 3
<code>:split</code> or <code>:sp</code>	horizontal window split
<code>:vsplit</code> or <code>:vsp</code>	vertical window split
<code>Ctrl+W</code> then <code>h/j/k/l</code>	move between windows
<code>Ctrl+W</code> then <code>q</code>	close current window
<code>:tabnew file.txt</code>	open file in a new tab
<code>gt</code> / <code>gT</code>	next / previous tab

Ex (Command-Line) Commands & Ranges

Any `:` command can be prefixed with a **range** telling it which lines to act on:

Range	Meaning
<code>:5</code>	line 5
<code>:5,10</code>	lines 5 through 10
<code>:\$</code>	current line to end of file
<code>:%</code>	whole file (shorthand for <code>1,\$</code>)
<code>:'<,'></code>	the last visual selection (auto-filled after selecting in Visual mode, then <code>:</code>)

Common Ex commands:

```

:set number      → show line numbers
:set nonumber    → hide them
:noh             → clear search highlighting
:!ls             → run a shell command without leaving vim
:r !date         → insert output of a shell command into the buffer

```

Folding

```

zf a{           → create a fold over "a{" text object (e.g. a code block)
zo              → open fold under cursor
zc              → close fold under cursor
za              → toggle fold
zR              → open all folds
zM              → close all folds

```

Configuration — `~/.vimrc`

Vim reads `~/.vimrc` on startup. A solid practical starting config:

```

set number           " show line numbers
set relativenumber   " line numbers relative to cursor – makes counts easy
set expandtab         " use spaces instead of tabs
set tabstop=4        " a tab is 4 spaces wide
set shiftwidth=4     " indent width
set autoindent       " keep indentation on new lines
set incsearch        " search as you type
set hlsearch         " highlight all matches
set ignorecase       " case-insensitive search...
set smartcase        " ...unless the pattern has a capital letter
set clipboard=unnamedplus " use system clipboard by default
syntax on            " syntax highlighting

```

Vim Quick Cheat Sheet

```

NAVIGATE      h j k l    0 ^ $    gg G    w b e    f{char}
INSERT        i a I A o O
EDIT          x dd dw d$    cc cw c$    yy yw    p P    u Ctrl+r    .
VISUAL        v V Ctrl+v
SEARCH        /pat ?pat  n N    :%s/old/new/g
SAVE/QUIT     :w :q :wq ZZ :q!

```

Philosophy

Nano is a straightforward, modeless text editor — what you see is what you get, no Normal/Insert distinction. Every shortcut is shown at the bottom of the screen, which makes it the default choice when you just need to edit a file right now without a learning curve.

TIP

Notation used everywhere in Nano's own docs:

- **^** means **Ctrl** — **^X** = **Ctrl+X**
- **M-** means **Alt/Meta** — **M-U** = **Alt+U**

Starting Nano

```

nano file.txt          # open (or create) a file
nano +42 file.txt      # open at line 42
nano -l file.txt       # force line numbers on
sudo nano /etc/hosts   # edit a root-owned file

```

File Operations

Shortcut	Effect
Ctrl+O	Write out (save) — you'll be prompted to confirm the filename
Ctrl+X	Exit (prompts to save if there are unsaved changes)
Ctrl+R	Read/insert another file's contents at the cursor
Ctrl+S	Save (in newer nano versions), without the write-prompt

Navigation

Shortcut	Effect
Arrow keys	Move cursor

Shortcut	Effect
Ctrl+F / Ctrl+B	Forward / back one character
Ctrl+P / Ctrl+N	Previous / next line
Ctrl+A	Go to start of line
Ctrl+E	Go to end of line
Ctrl+Y	Page up
Ctrl+V	Page down
Ctrl+_ then a number	Go to a specific line (and column)
M-\\	Go to first line of file
M-/	Go to last line of file

Editing

Shortcut	Effect
Ctrl+K	Cut current line (or selected region) into the clipboard
Ctrl+U	Paste (uncut) whatever was last cut
M-6	Copy current line (or selection) without cutting it
Ctrl+Shift+6 or start selection with M-A	Begin a text selection (then move cursor to extend it)
Ctrl+D	Delete character under cursor
Backspace	Delete character before cursor
M-U	Undo
M-E	Redo
Ctrl+T	Run the spell-checker (if installed)
M-J	Justify the current paragraph (wrap text neatly)

TIP

Selecting and cutting a block: move to the start, **Ctrl+^** (or **M-A**) to mark the start, move the cursor to the end of the region, then **Ctrl+K** cuts the whole selection at once (instead of line by line).

Search & Replace

Shortcut	Effect
Ctrl+W	Search forward
Ctrl+W then Ctrl+R , or Ctrl+\\	Search and replace
Inside a search prompt, Alt+C	Toggle case sensitivity
Inside a search prompt, Alt+R	Toggle regular-expression mode
Alt+W	Repeat the last search

TIP

Replace workflow:

```
Ctrl+\    → opens "Search (to replace):" prompt
type pattern, Enter
type replacement, Enter
→ Nano asks Yes/No/All/Cancel for each match
```

Multiple Buffers (Files)

Shortcut	Effect
Ctrl+R then a filename	Insert another file into the current buffer
M-< / M->	Switch to previous / next open buffer (if nano was started with multiple files or -F /multibuffer mode enabled)
nano -F file1 file2	Open multiple files as separate buffers in one session

Display Options

Shortcut	Effect
Ctrl+C	Show current cursor line/column position
M-N	Toggle line numbers
M-P	Toggle display of whitespace characters
M-\$	Toggle "soft wrap" of long lines
M-Y	Toggle syntax highlighting on/off

Configuration — `~/.nanorc`

```
set linenumbers      # always show line numbers
set tabsize 4        # tab width
set tabstospaces     # convert tabs to spaces
set autoindent       # keep indentation on new lines
set mouse            # enable mouse support
include "/usr/share/nano/*.nanorc" # enable syntax highlighting for common languages
```

Nano Quick Cheat Sheet

```
SAVE/EXIT  ^O (write out)  ^X (exit)
NAVIGATE   arrows  ^A (line start)  ^E (line end)  ^Y/^V (page up/down)
CUT/PASTE  ^K (cut line)  ^U (paste)  M-6 (copy)
SEARCH     ^W (search)  ^\ (replace)  M-W (repeat search)
UNDO/REDO  M-U / M-E
```

PART 3: Which One Should You Use?

Situation	Use
Quick edit of a config file over SSH	Nano — zero learning curve, shortcuts on-screen
You edit code/config for hours daily	Vim — the modal grammar pays for itself fast at that volume
Editing on a minimal/embedded system	Whichever is installed — Vi (the original, always present) is the fallback if neither Vim nor Nano exists
Scripted, repetitive text transformations	Vim — macros and Ex-command ranges do in seconds what takes minutes by hand

Situation	Use
You just SSH'd into a server for one line change	Nano

TIP

One thing worth knowing: almost every minimal Linux install ships with `vi` even when `vim` isn't installed — so knowing at least `i` (insert), `Esc`, and `:wq` (save+quit) in bare Vi is a genuine survival skill for server work, even if Nano is your daily driver.

Bash Scripting Deep Dive

Functions, arrays, loops, and error handling for writing robust bash scripts that fail loudly.

Level: Intermediate • Words: 1,856 • Sections: 12

Commands: cat, chmod, echo, env, grep, rm

Introduction

Functions, arrays, loops, and — the part most tutorials skip — how to write scripts that fail loudly instead of silently corrupting something three steps later.

Script Anatomy

```
#!/usr/bin/env bash
# ^ shebang – tells the OS which interpreter runs this file.
# #!/usr/bin/env bash is more portable than #!/bin/bash (finds bash wherever it's
# installed)

echo "Hello, SOLO"
```

```
chmod +x script.sh      # make it executable
./script.sh             # run it
bash script.sh          # or run it explicitly through bash without chmod
```

Variables

```
name="SOLO"             # NO spaces around = – "name = SOLO" is a syntax error, not an
assignment              # always quote variables when reading them
echo "$name"            # constant – reassigning it later throws an error
readonly PI=3.14         # delete a variable
unset name

local x=5               # only valid INSIDE a function – scopes the variable to that
function
```

TIP

Quoting rules that actually matter:

```
var="hello world"
echo $var              # WRONG in most cases – unquoted, this splits on whitespace: prints as
2 args
echo "$var"            # RIGHT – preserves it as one string, including internal spaces
echo '$var'            # single quotes = literal, no expansion at all – prints the text $var
```

Arrays

Indexed arrays


```

fruits=("apple" "banana" "cherry")

echo "${fruits[0]}"          # apple – array indices start at 0
echo "${fruits[@]}"          # apple banana cherry – all elements
echo "${#fruits[@]}"         # 3 – length of the array

fruits+=("mango")            # append an element
fruits[1]="blueberry"        # overwrite index 1

for f in "${fruits[@]}"; do   # correct way to iterate – quoted, with @
    echo "$f"
done

unset 'fruits[0]'             # remove one element (index 0), leaves a gap

```

Associative arrays (key-value, bash 4+)

```

declare -A ages
ages["solo"]=25
ages["ali"]=30

echo "${ages[solo]}"         # 25
echo "${!ages[@]}"           # solo ali – all keys
echo "${ages[@]}"            # 25 30 – all values

for key in "${!ages[@]}"; do  # iterate keys, look up values
    echo "$key is ${ages[$key]}"
done

```

Common array pitfalls

```

echo "${fruits[@]}"          # correct – each element stays separate, even with spaces inside
echo "${fruits[*]}"          # joins ALL elements into ONE string – rarely what you want
echo $fruits                 # WRONG – without an index, this only gives element 0

```

Loops

`for` loop — three styles

```

# 1. Iterate a list
for fruit in apple banana cherry; do
    echo "$fruit"
done

# 2. C-style
for ((i=0; i<5; i++)); do
    echo "Number: $i"
done

# 3. Range
for i in {1..5}; do
    echo "$i"
done

# Iterate command output (careful – see "reading files" below for a safer pattern)
for file in *.txt; do
    echo "Found: $file"
done

```

`while` loop

```
count=0
while [ "$count" -lt 5 ]; do
    echo "Count: $count"
    ((count++))
done
```

The correct way to read a file line-by-line (avoids classic pitfalls with spaces/globbering):

```
while IFS= read -r line; do
    echo "Line: $line"
done < file.txt
```

`until` loop (opposite of while — runs until the condition becomes true)

```
count=0
until [ "$count" -ge 5 ]; do
    echo "$count"
    ((count++))
done
```

`break` and `continue`

```
for i in {1..10}; do
    [ "$i" -eq 5 ] && break          # exit the loop entirely
    [ $((i % 2)) -eq 0 ] && continue # skip to next iteration
    echo "$i"
done
```

Conditionals

```
if [ "$1" == "yes" ]; then
    echo "Confirmed"
elif [ "$1" == "no" ]; then
    echo "Denied"
else
    echo "Unclear"
fi
```

TIP

[] vs [[]]: always prefer **[[]]** in bash — it's safer (no word-splitting/glob surprises on unquoted variables) and supports **&&**, **||**, and pattern matching directly inside it.

```
[[ "$name" == "SOLO" ]]      # string equality
[[ "$name" == S* ]]          # pattern match — only works inside [[ ]]
[[ -f "$file" && -r "$file" ]] # file exists AND is readable
```

TIP

Common test operators:

Test	Meaning
-eq -ne -lt -le -gt -ge	numeric comparisons
== !=	string equality/inequality
-z "\$s"	string is empty
-n "\$s"	string is non-empty

Test	Meaning
<code>-f file</code>	file exists and is a regular file
<code>-d dir</code>	directory exists
<code>-r -w -x file</code>	readable / writable / executable
<code>-e path</code>	path exists (any type)

`case` statement — cleaner than a long if/elif chain

```
case "$1" in
  start)
    echo "Starting..."
    ;;
  stop)
    echo "Stopping..."
    ;;
  restart|reload)
    echo "Restarting..."
    ;;
  *)
    echo "Usage: $0 {start|stop|restart}"
    exit 1
    ;;
esac
```

Functions

```
greet() {
  local name="$1"      # $1 = first argument passed to the function – always
  'local' it
  echo "Hello, $name"
}

greet "SOLO"           # call it like a command

# All positional parameters inside a function:
show_args() {
  echo "First: $1"
  echo "All: $@"        # each argument as a separate word
  echo "Count: $#"      # number of arguments passed
}
show_args a b c
```

TIP

Return values — the part that trips people up: bash functions don't "return" data the way most languages do. `return` only sets an **exit code** (0–255). To get actual data out, either `echo` it and capture with `$()`, or write into a variable.

```
add() {
  local result=$(( $1 + $2 ))
  echo "$result"        # "return" data by printing it
}
sum=$(add 3 4)          # capture it via command substitution
echo "$sum"             # 7

is_even() {
  (( $1 % 2 == 0 ))     # return TRUE/FALSE as an exit code, not printed text
}
if is_even 4; then
  echo "even"
fi
```

Error Handling — The Part Most Scripts Skip

By default, bash **keeps going after a failed command** — this is the single biggest source of scripts that silently do the wrong thing.

The essential header for almost every script you write

```
set -euo pipefail
```

Effect		
zero exit code)		
hes typos like \$FIL instead of \$FILE)		
	b \	c) fails if any co

TIP

Why pipefail matters specifically: without it, `bad_command | grep something` only checks `grep`'s exit code — if `bad_command` itself failed, the pipeline still reports success as long as `grep` didn't error. This hides real failures constantly.

```
false | echo "done"           # WITHOUT pipefail: exit code 0 – looks successful!
set -o pipefail
false | echo "done"           # WITH pipefail: exit code reflects false's failure
```

`set -e` gotchas — it doesn't catch everything

```
set -e
if some_command; then      # commands inside an if CONDITION don't trigger set -e – this
is intentional
    echo "ok"
fi

some_command || true       # explicitly suppresses set -e for that one line
some_command && echo "ok"   # also won't trigger set -e on failure of some_command
```

Debug/trace mode

```
set -x    # print every command before running it – invaluable when a script does
something unexpected
set +x    # turn it back off
```

Run a whole script in trace mode without editing it:

```
bash -x script.sh
```

`trap` — running cleanup code no matter how the script exits

`trap` catches signals or the script's own exit and runs a function — essential for cleaning up temp files, releasing locks, or logging what happened, even when the script crashes or gets killed.

```
cleanup() {
    echo "Cleaning up..."
    rm -f "$TMPFILE"
}
trap cleanup EXIT          # runs on ANY exit – success, error, or Ctrl+C

trap 'echo "Interrupted!"; exit 1' INT      # specifically catch Ctrl+C (SIGINT)
trap 'echo "Error on line $LINENO"' ERR    # runs whenever a command fails (with set -e active)
```

TIP**Practical temp-file pattern:**

```
TMPFILE=$(mktemp)
trap 'rm -f "$TMPFILE"' EXIT      # guarantees cleanup even if the script dies halfway through
echo "some data" > "$TMPFILE"
```

Checking exit codes explicitly

```
some_command
if [ $? -ne 0 ]; then          # $? = exit code of the LAST command run
    echo "some_command failed" >&2
    exit 1
fi

# Cleaner, more idiomatic version of the same thing:
if ! some_command; then
    echo "some_command failed" >&2
    exit 1
fi
```

Writing Scripts That Don't Silently Fail

A checklist, not just theory:

```
``bash if [ $# -lt 1 ]; then echo "Usage: $0 <filename>" >&2 exit 1 fi FILE="$1" if [ ! -f "$FILE" ];
then echo "Error: $FILE does not exist" >&2 exit 1 fi ``

``bash log() { echo "[$(date '+%Y-%m-%d %H:%M:%S')] $*"; } log "Starting backup..." ``

``bash sudo apt install shellcheck -y shellcheck script.sh ``
```

- **Start every script with `set -euo pipefail`** unless you have a specific reason not to.
- **Validate arguments before using them:**
- **Send errors to stderr (`>&2`), not stdout** — keeps error messages separate from real output, so anything piping your script's output doesn't accidentally consume an error message as data.
- **Use meaningful, distinct exit codes** — `0` for success, and different non-zero codes for different failure types, so callers (or you, six months later) can tell what failed without re-reading the script.
- **Use `trap ... EXIT`** for any script that creates temp files, acquires locks, or opens connections — cleanup should be automatic, not something you remember to add at the bottom (which never runs if the script dies early anyway).
- **Quote every variable expansion** ("`$var`", "`${array[@]}`") — the single most common source of scripts that "work on my machine" and break on filenames with spaces.
- **Never trust `$1/$2` blindly** — a script called with the wrong number of arguments should fail with a clear usage message, not proceed with an empty/unset variable and do something wrong silently.
- **Log with timestamps** for anything long-running or cron-scheduled — when it fails at 3 AM, you want to know when, not just that:

- Run **shellcheck** on every script before trusting it. It's a static analyzer that catches quoting bugs, unreachable code, and dozens of other classic bash mistakes before you ever run the script:

String Manipulation

```
str="Hello, World"

echo "${#str}"           # 12 – length
echo "${str:7}"          # World – substring from index 7
echo "${str:7:3}"        # Wor – substring, 3 chars starting at index 7
echo "${str:World/Bash}" # Hello, Bash – replace first match
echo "${str//o/O}"       # Hello, World – replace ALL matches
echo "${str^^}"          # HELLO, WORLD – uppercase
echo "${str,,}"          # hello, world – lowercase
```

Input & Output

```
read -p "Enter your name: " name      # prompt + capture input
echo "Hi, $name"

result=$(ls -la)                      # command substitution – capture command output
into a variable

cat <<EOF                             # here-doc – multi-line text block
This is line 1
This is line 2, with a variable: $name
EOF
```

A Hardened Script Template

Putting it all together — this is a reasonable default skeleton for any real script:

```
#!/usr/bin/env bash
set -euo pipefail

log() { echo "[$(date '+%Y-%m-%d %H:%M:%S')] $*"; }
error() { echo "[$(date '+%Y-%m-%d %H:%M:%S')] ERROR: $*" >&2; }

cleanup() {
    log "Cleaning up..."
    [ -n "${TMPFILE:-}" ] && rm -f "$TMPFILE"
}
trap cleanup EXIT
trap 'error "Failed on line $LINENO" ERR'

usage() {
    echo "Usage: $0 <input_file>" >&2
    exit 1
}

# --- Validate input ---
[ $# -lt 1 ] && usage
INPUT_FILE="$1"
[ ! -f "$INPUT_FILE" ] && { error "$INPUT_FILE not found"; exit 1; }

# --- Main logic ---
TMPFILE=$(mktemp)
log "Processing $INPUT_FILE..."
grep "pattern" "$INPUT_FILE" > "$TMPFILE"
log "Found $(wc -l < "$TMPFILE") matches"

log "Done."
```

This template: fails fast on any error, catches unset variables, catches pipeline failures, logs with timestamps, cleans up its temp file no matter how it exits, and validates its input before touching it — the combination that turns "script that mostly works" into "script you can actually trust in a cron job."



Part II — System Operations

Disks, boot, systemd, processes & logs — keep the machine healthy. — 6 guides

Disk & Storage Management — Partitions, LVM, Mounting & fstab

Partitions, mounts, LVM and df/du — understand disk layout and never run out of space by surprise.

Level: Intermediate • Words: 1,970 • Sections: 12

Commands: blkid, chmod, df, du, fdisk, lsblk +8 more

Introduction

How storage actually fits together on Linux: disk → partition → filesystem → mount point — and the file that ties it all together at boot.

The Storage Hierarchy

```
Physical disk (/dev/sda)
  ■■ Partition (/dev/sda1)
    ■■ Filesystem (ext4, xfs, etc. — formatted onto the partition)
      ■■ Mount point (/ , /home, /mnt/data — where it becomes accessible in the
tree)
```

A raw disk is useless until it's partitioned; a partition is useless until it has a filesystem; a filesystem is inaccessible until it's mounted somewhere in the directory tree. Every storage task on Linux is really just moving down (or up) this chain.

Viewing Disks & Partitions

```
lsblk          # tree view of all block devices and their partitions — start
here, always
lsblk -f       # same, plus filesystem type and UUID
sudo fdisk -l  # detailed partition table info, needs root
sudo parted -l # same idea, better for GPT disks
blkid         # list filesystem UUIDs and types for every device
df -h         # disk usage of currently MOUNTED filesystems
du -sh /path/to/dir # disk usage of a specific directory (sums recursively)
```

TIP

Reading **lsblk** output:

NAME	SIZE	TYPE	MOUNTPOINT
sda	500G	disk	
■■sda1	512M	part	/boot/efi
■■sda2	50G	part	/
■■sda3	449.5G	part	/home

disk = the whole physical device; **part** = a partition on it. The indentation shows the parent-child relationship.

Partitioning

MBR vs GPT — know which you're working with

	MBR (older)	GPT (modern)
Max partitions	4 primary (or 3 + extended)	128
Max disk size	2TB	Effectively unlimited
Boot mode	BIOS legacy	UEFI (also supports legacy)
Use today	Only for compatibility with very old systems	Default choice for anything new

Partitioning with `fdisk` (MBR-friendly, works on GPT too)

TIP

Step 1 — Enter fdisk on the target disk (never a mounted/in-use one):

```
sudo fdisk /dev/sdb
```

TIP

Step 2 — Inside the fdisk prompt, key commands:

```
n    → create a new partition
p    → print the current partition table
d    → delete a partition
t    → change a partition's type
w    → write changes to disk and exit (nothing is applied until this)
q    → quit WITHOUT saving – your escape hatch if you're unsure
```

TIP

Step 3 — Creating a partition, walked through:

```
Command: n
Partition type: p (primary) or e (extended)
Partition number: (accept default, e.g. 1)
First sector: (accept default – start right after the previous partition)
Last sector: +50G (or accept default to use all remaining space)
Command: w    → commits it
```

Partitioning with `parted` (better for GPT, scriptable)

```
sudo parted /dev/sdb
(parted) mklabel gpt          # initialize a GPT partition table – WARNING: wipes
existing partition table
(parted) mkpart primary ext4 0% 50%  # create a partition spanning the first half of
the disk
(parted) print                # verify
(parted) quit
```

TIP

After creating any partition, the kernel needs to notice it:

```
sudo partprobe /dev/sdb    # re-reads the partition table without a reboot
```

Creating a Filesystem

A fresh partition is just empty space until it's formatted:

```

sudo mkfs.ext4 /dev/sdb1      # format as ext4 — the common general-purpose default
sudo mkfs.xfs /dev/sdb1      # xfs — strong for large files, used a lot in server/DB
contexts
sudo mkfs.vfat -F 32 /dev/sdb1  # FAT32 — for cross-compatibility with Windows/
embedded devices

```

Filesystem	Good for
ext4	General-purpose default on most Linux distros — mature, reliable
xfs	Large files, high-throughput workloads (common on RHEL-family servers)
btrfs	Snapshots, built-in RAID-like features, copy-on-write
vfat/exfat	Cross-OS compatibility (USB drives you'll also plug into Windows)

This step is destructive — formatting erases whatever was previously on that partition. Always confirm the device name with `lsblk` immediately before running `mkfs.*`.

Mounting — Temporary (Manual)

```

sudo mkdir -p /mnt/data      # the mount point must already exist as a directory
sudo mount /dev/sdb1 /mnt/data  # attach the filesystem there
mount | grep sdb1           # confirm it's mounted, and see the options it
mounted with
sudo umount /mnt/data        # detach it — always do this before physically
removing the device

```

TIP

If `umount` refuses ("target is busy"):

```

sudo lsof /mnt/data          # find which process still has a file open there
sudo fuser -m /mnt/data      # alternative way to find the culprit process

```

Never force-unmount a device that's actively being written to — see the file-corruption guide for why.

Important: manual mounts like this vanish on reboot. For anything that needs to persist, it belongs in `/etc/fstab` — which is the rest of this guide.

LVM — Logical Volume Management

LVM adds a flexible layer between raw partitions and filesystems, letting you resize, combine, and snapshot storage without being locked into a partition's original fixed size.

Core concepts

```

Physical Volume (PV) → a raw disk or partition, initialized for LVM use
Volume Group (VG)   → a pool combining one or more PVs into one big space
Logical Volume (LV)  → a resizable "virtual partition" carved out of a VG

```

Think of the VG as a shared pool of storage, and LVs as flexible allocations from that pool — you can grow an LV later by just giving it more of the pool, without the rigid "adjacent free space" constraint raw partitions have.

Step-by-step LVM setup

TIP

Step 1 — Turn a raw partition/disk into a Physical Volume:

```
sudo pvcreate /dev/sdb1
sudo pvdisplay      # verify
```

TIP**Step 2 — Create a Volume Group from one or more PVs:**

```
sudo vgcreate data_vg /dev/sdb1
sudo vgdisplay      # verify – shows total/free space in the group
```

TIP**Step 3 — Carve out a Logical Volume from the group:**

```
sudo lvcreate -L 20G -n data_lv data_vg      # 20GB LV named "data_lv" inside "data_vg"
sudo lvdisplay                                # verify
```

TIP**Step 4 — Format and mount it like any normal partition:**

```
sudo mkfs.ext4 /dev/data_vg/data_lv
sudo mkdir -p /mnt/data
sudo mount /dev/data_vg/data_lv /mnt/data
```

Resizing — the actual payoff of using LVM

```
sudo lvextend -L +10G /dev/data_vg/data_lv      # grow the LV by 10GB (needs free space
in the VG)
sudo resize2fs /dev/data_vg/data_lv              # grow the ext4 filesystem to fill
the new LV size
# for xfs, the equivalent is:
sudo xfs_growfs /mnt/data
```

TIP

Note: shrinking is riskier and filesystem-dependent (ext4 supports it offline; xfs does not support shrinking at all) — always back up before attempting a shrink.

Snapshots — point-in-time copies for safe experimentation/backups

```
sudo lvcreate -L 5G -s -n data_snap /dev/data_vg/data_lv      # create a snapshot
# ... do risky work on the original LV, or back up the snapshot ...
sudo lvremove /dev/data_vg/data_snap                          # remove the snapshot
when done
```

`/etc/fstab` — The Real Deep Dive

This is the file that tells the system what to mount automatically at every boot. Get it wrong and the machine can fail to boot entirely — so understanding every field matters.

Anatomy of a line

```
UUID=1234-5678 /mnt/data ext4 defaults 0 2
■          ■          ■          ■          ■ ■
■          ■          ■          ■          ■ ■ fsck order (pass)
■          ■          ■          ■          ■ ■ dump flag
■          ■          ■          ■          ■ ■ mount options
■          ■          ■          ■          ■ ■ filesystem type
■          ■          ■          ■          ■ ■ mount point
■          ■          ■          ■          ■ ■ device identifier
```

Six fields, always space/tab-separated, always in this order.

Field 1: Device identifier — and why UUID beats `/dev/sdX`

```
/dev/sdb1                                # device path – WORKS but is unstable
UUID=1234-5678-ABCD                     # stable identifier tied to the
filesystem itself
LABEL=mydata                             # a human-assigned label, if you set
one
```

TIP

Why this matters: `/dev/sdb1` is assigned by detection order at boot — plug in a USB drive, add a new disk, or have a drive fail, and the same physical disk you meant might now be `/dev/sdc1` instead. `/etc/fstab` entries using raw device paths can silently mount the **wrong disk** at the **right mount point** after such a change. UUIDs are generated when the filesystem is created and never change, regardless of detection order.

TIP

Find your UUIDs:

```
sudo blkid
# or
lsblk -f
```

Field 2: Mount point

The directory this filesystem attaches to — must already exist (`mkdir -p` it first if it doesn't).

Field 3: Filesystem type

`ext4`, `xfs`, `vfat`, `ntfs`, `swap`, or `auto` (let the kernel detect it — fine for removable media, not recommended for critical system mounts).

Field 4: Mount options — the field most people leave at ``defaults`` and never learn

Option	Effect
<code>defaults</code>	Shorthand for: <code>rw</code> , <code>suid</code> , <code>dev</code> , <code>exec</code> , <code>auto</code> , <code>nouser</code> , <code>async</code>
<code>ro</code> / <code>rw</code>	Read-only / read-write
<code>noauto</code>	Don't mount automatically at boot — only mounts when you explicitly run <code>mount /mountpoint</code>
<code>nofail</code>	Critical for external/removable drives — if the device isn't present at boot, skip it instead of hanging or dropping into emergency mode
<code>user</code>	Allow non-root users to mount it
<code>nouser</code>	Only root can mount it
<code>exec</code> / <code>noexec</code>	Allow / block executing binaries from this filesystem
<code>suid</code> / <code>nosuid</code>	Honor / ignore <code>setuid</code> bits on this filesystem — <code>nosuid</code> is a real security hardening step for untrusted removable media
<code>async</code> / <code>sync</code>	Buffered (faster) vs immediate (safer, slower) writes

TIP

A very common real mistake: adding a second internal or external drive to `/etc/fstab` without `nofail`. If that drive is ever disconnected, missing, or fails, the **entire boot process hangs** waiting for it — turning a "drive I use sometimes" into "reason the whole server won't come up." Always pair external/optional drives with `nofail` (and usually `noauto` too, if you don't need it mounted every single boot).

```
UUID=1234-5678 /mnt/backup-drive ext4 defaults,nofail 0 2
```

Field 5: Dump flag

Legacy — controls whether the ancient `dump` backup utility includes this filesystem. Almost nobody uses `dump` anymore. Convention: `1` for the root filesystem, `0` for everything else. Safe to always set `0` on modern systems.

Field 6: fsck pass order

Controls filesystem-check order and whether it happens at all:

- `0` — never fsck this filesystem (correct for swap, removable media, network mounts)
- `1` — check first (reserved for the **root filesystem only**)
- `2` — check after root, for everything else that should be checked

Testing fstab changes safely — never just reboot and hope**TIP****Step 1 — Edit the file:**

```
sudo nano /etc/fstab # or vim
```

TIP**Step 2 — Test it WITHOUT rebooting:**

```
sudo mount -a # attempts to mount everything in fstab that isn't already mounted
```

If there's a syntax error or a bad UUID, `mount -a` reports it immediately — on a live system, where you can still fix it. A reboot with a broken fstab, by contrast, can drop you into **emergency mode**, a minimal recovery shell, at boot.

TIP**Step 3 — Verify what actually got mounted:**

```
df -h
mount | grep /mnt/data
```

Recovering from a broken fstab that already caused a boot failure

If you do end up in emergency mode after a bad edit:

```
# emergency mode typically drops you to a root shell automatically
mount -o remount,rw / # root may be mounted read-only in this mode — fix
that first
nano /etc/fstab # fix or comment out the bad line with a leading #
reboot
```

If it won't even reach emergency mode, boot from a live USB, mount the disk manually, and edit `/etc/fstab` from there.

Swap in fstab

TIP

Swap partition:

```
UUID=abcd-1234 none swap sw 0 0
```

TIP

Swap file (common on cloud/VPS instances without a dedicated swap partition):

```
sudo fallocate -l 2G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
```

Then in fstab:

```
/swapfile none swap sw 0 0
```

Checking & Repairing Filesystems

```
sudo fsck /dev/sdb1      # NEVER run on a mounted filesystem – unmount first
sudo fsck -f /dev/sdb1  # force a check even if the filesystem looks clean
```

fsck on the root filesystem specifically requires booting into single-user/rescue mode, since root can't be unmounted while the system is running from it.

Ongoing Disk Usage Monitoring

```
df -h          # free/used space per mounted filesystem
du -sh /var/log # total size of a specific directory
du -h --max-depth=1 /home # size of each immediate subdirectory – good for hunting
what's eating space
ncdu /         # interactive, navigable disk usage explorer (install:
sudo apt install ncdu)
```

Quick Reference Summary

```
VIEW      lsblk  lsblk -f  df -h  blkid
PARTITION fdisk /dev/sdX or parted /dev/sdX
FORMAT    mkfs.ext4 /dev/sdX1
MOUNT (temp) mount /dev/sdX1 /mnt/point
MOUNT (perm) edit /etc/fstab, then: mount -a to test
LVM CHAIN pvcreate → vgcreate → lvcreate → mkfs → mount
FSTAB SAFETY ALWAYS use UUID, ALWAYS use nofail on optional drives, ALWAYS test with
mount -a before reboot
```

The Linux Boot Process — BIOS/UEFI → GRUB → systemd

From BIOS/UEFI to login prompt — every stage of boot, and how to diagnose a machine that won't come up.

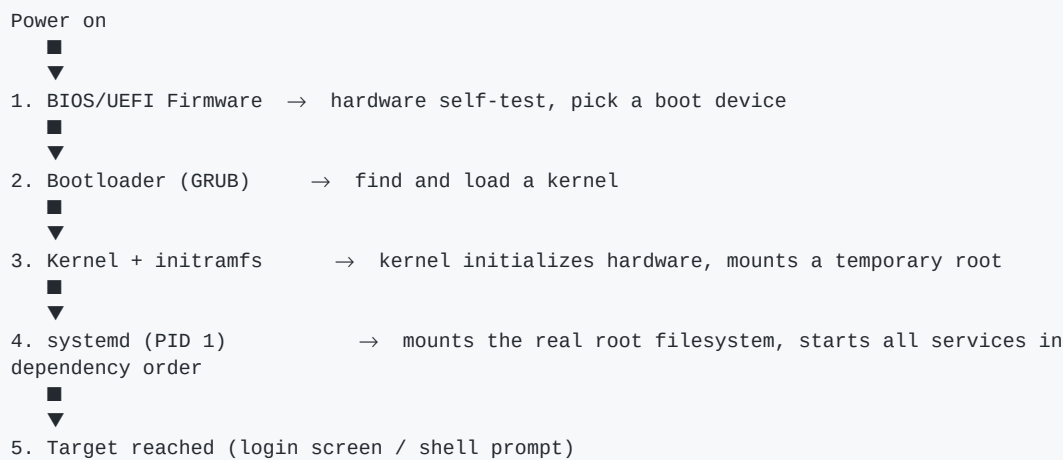
Level: Intermediate • Words: 1,915 • Sections: 9

Commands: cat, journalctl, ls, lsblk, mount, nano +2 more

Introduction

Every stage a machine passes through between "power button pressed" and "login prompt," and — the actual point of this guide — how to tell which stage failed when a machine won't boot.

The Full Chain, At a Glance



Every boot failure lives in exactly one of these stages — the entire diagnostic skill is recognizing which one from what you see on screen.

Stage 1: BIOS / UEFI Firmware

The firmware is code baked into the motherboard, running before any OS exists. It has one job: run hardware checks, then hand control to a bootable device.

BIOS vs UEFI

	Legacy BIOS	UEFI (modern standard)
Age	Original, decades-old standard	Modern replacement, standard on hardware since ~2012+
Disk requirement	MBR partition table	GPT partition table (with an EFI System Partition)
Boot process	Reads the first 512 bytes (MBR) of the boot disk directly	Reads an actual <code>.efi</code> executable file from the ESP
Secure Boot	Not supported	Supported — cryptographically verifies the bootloader before running it
Max disk size for boot drive	2TB	Effectively unlimited

What happens in this stage

- **POST** (Power-On Self-Test) — checks RAM, CPU, attached devices are present and functional.
- Firmware consults its **boot order** — an ordered list of devices to try (disk, USB, network/PXE, etc).

- For UEFI: it looks for a bootloader `.efi` file on the **EFI System Partition (ESP)** — commonly mounted at `/boot/efi` on a running Linux system — typically `/EFI/<distro>/grubx64.efi` or similar.
- For legacy BIOS: it loads the first 512 bytes (the Master Boot Record) of the selected disk, which contains a tiny bit of bootloader code (GRUB Stage 1).

Symptoms that mean the failure is HERE

- No display output at all, no beep codes, fans spin but nothing happens → hardware/POST failure, not a Linux problem
- `No bootable device found / Boot device not found` → firmware can't find anything to boot at all — wrong boot order, disk not detected, or the ESP/MBR itself is missing or corrupted
- Machine goes straight into the firmware setup screen instead of booting → boot order is misconfigured, or it's failing every device in the list and falling through to setup

What to check

- Enter the firmware setup (usually `Del`, `F2`, `F10`, or `Esc` at power-on, varies by motherboard) and confirm:
- The boot drive is actually detected/listed
- Boot order has the correct drive first
- Secure Boot isn't blocking an unsigned bootloader (a real issue if you've installed a custom kernel or third-party GRUB build)

Stage 2: GRUB (Bootloader)

GRUB's job: find the actual Linux kernel and initramfs, load them into memory, and hand off control.

GRUB's own stages (relevant mainly for legacy BIOS)

```
Stage 1 → tiny code in the MBR, just enough to load Stage 1.5/2
Stage 1.5 → filesystem drivers, lets GRUB read the actual /boot partition
Stage 2 → the full GRUB environment – reads grub.cfg, shows the boot menu
```

On UEFI systems this is simplified — the `.efi` file loaded directly from the ESP largely is the equivalent of Stage 2.

Key files

```
/boot/grub/grub.cfg      # the ACTUAL config GRUB reads at boot – auto-generated,
don't hand-edit
/etc/default/grub        # the file you SHOULD edit – human-readable settings
/etc/grub.d/             # scripts that generate grub.cfg's contents
```

TIP

Critical rule: never hand-edit `/boot/grub/grub.cfg` directly — it's regenerated automatically and your changes will be silently lost. Edit `/etc/default/grub`, then regenerate:

```
sudo update-grub          # Debian/Ubuntu
sudo grub2-mkconfig -o /boot/grub2/grub.cfg # RHEL/Fedora
```

Common settings in `/etc/default/grub`

```
GRUB_DEFAULT=0           # which menu entry boots by default (0 = first)
GRUB_TIMEOUT=5           # seconds the menu is shown before auto-booting
GRUB_CMDLINE_LINUX_DEFAULT="quiet splash" # kernel boot parameters
```

Interacting with GRUB live at boot

- Hold **Shift** (BIOS) or repeatedly tap **Esc** (UEFI) during boot to force the GRUB menu to appear if it's hidden/too fast to catch.
- At the GRUB menu, press **e** to **edit** a boot entry's parameters for this boot only (doesn't persist) — useful for adding **single** or removing **quiet splash** to see what's actually happening.
- Press **c** to drop into the **GRUB command line** — a minimal shell for manually specifying a kernel to boot when the configured entries are broken.

Symptoms that mean the failure is HERE

- **error: no such partition / error: file not found** at a GRUB prompt → grub.cfg is pointing at a partition/UUID that's changed or doesn't exist (very common after resizing partitions, or a UUID mismatch — same root cause as the fstab issues from your storage guide)
- Dropped into a bare **grub rescue>** prompt → GRUB's own core files can't be found — the ESP/boot partition itself may be damaged or GRUB's install is broken
- GRUB menu shows, but every entry fails the same way → likely a kernel/initramfs problem (next stage), not GRUB itself

Manually booting from a `grub rescue>` prompt (get in, then fix it properly)

```
grub rescue> ls                                # list available drives/partitions
grub rescue> ls (hd0,gpt2)/                    # inspect a partition's contents to find the
right one
grub rescue> set root=(hd0,gpt2)
grub rescue> set prefix=(hd0,gpt2)/boot/grub
grub rescue> insmod normal
grub rescue> normal                            # attempts to load the full GRUB environment
from here
```

Stage 3: Kernel & initramfs

Once GRUB hands off, the actual Linux kernel loads into memory and starts running.

What initramfs is for

The kernel alone often can't mount the real root filesystem yet — it might need drivers for RAID, LVM, encrypted disks, or unusual storage controllers, which aren't compiled directly into the kernel. **initramfs** (initial RAM filesystem) is a small temporary root filesystem, loaded into memory alongside the kernel, that has just enough tools/drivers to find and mount the real root filesystem — after which the kernel switches over (**pivot_root**) and initramfs's job is done.

```
/boot/vmlinuz-<version>    # the kernel image itself
/boot/initrd.img-<version>  # the initramfs
```

Regenerating initramfs (needed after certain driver/LVM/encryption config changes):

```
sudo update-initramfs -u    # Debian/Ubuntu
sudo dracut --force         # RHEL/Fedora
```

Symptoms that mean the failure is HERE

- **Kernel panic** — the screen fills with a wall of technical output ending in **kernel panic - not syncing: ...** → often a missing/corrupt initramfs, an incompatible kernel module, or (on real hardware) a driver issue for the boot storage controller
- **Gave up waiting for root device** → initramfs can't find/mount the real root filesystem — frequently a UUID mismatch in the kernel command line or initramfs not having the right storage driver
- Boot hangs indefinitely with no error, just a blinking cursor after the kernel starts loading → often a graphics driver issue; try removing **quiet splash** from the GRUB kernel parameters (press **e** at the GRUB menu) to see

actual boot messages instead of a blank splash screen

What to check

```
# once booted (or from a live USB, chrooted into the broken system):
ls -la /boot/                # confirm vmlinuz and initrd.img actually exist and
                              # aren't zero-byte
cat /etc/default/grub | grep CMDLINE  # check kernel parameters for anything obviously
wrong
```

Stage 4: systemd (PID 1)

Once the real root filesystem is mounted, the kernel starts the very first userspace process — on virtually every modern distro, that's **systemd**, always PID 1. From here, systemd's job is to bring the system up to a target state by starting units (services, mounts, sockets, etc.) in dependency order.

Targets — the systemd replacement for old-style "runlevels"

Target	Roughly equivalent to	Purpose
poweroff.target	runlevel 0	shutdown
rescue.target	runlevel 1	single-user mode — minimal, root shell, most services not started
multi-user.target	runlevel 3	full multi-user system, no GUI
graphical.target	runlevel 5	multi-user + display manager/GUI
emergency.target	(no old equivalent)	absolute minimum — even less than rescue, drops you to a root shell with almost nothing mounted, used when something as fundamental as fstab is broken

```
systemctl get-default          # what target boots by default
sudo systemctl set-default multi-user.target  # e.g. disable booting to a GUI
```

Symptoms that mean the failure is HERE

- Machine boots, kernel loads fine, but **drops into emergency mode** with a message like **You are in emergency mode** → almost always a bad **/etc/fstab** entry (see your fstab deep-dive guide) or a critical mount failing
- Boots to a login prompt eventually, but takes a very long time → a specific unit is hanging waiting on a dependency (commonly network-related units waiting for a network that never comes up, especially on machines with unreliable NICs or missing **nofail/noauto** on fstab entries)
- One specific service fails repeatedly but the rest of the system is fine → that's a service-level problem, not a boot-chain problem — see your process/journalctl guide for that investigation

Diagnosing systemd-stage boot problems

```
systemd-analyze                # total boot time breakdown, by stage (firmware/
loader/kernel/userspace)
systemd-analyze blame          # every unit, sorted by how long it took to start
- find the slow one fast
systemd-analyze critical-chain  # shows the actual dependency chain that
determined total boot time
journalctl -b                  # full log of the current boot
journalctl -b -1                # full log of the PREVIOUS boot — essential
after an unexpected reboot/crash
```

Recovering from emergency/rescue mode

```
# you're typically dropped into a root shell automatically here
mount -o remount,rw /          # root is often mounted read-only in this mode – fix
that first
nano /etc/fstab                # find and fix (or comment out with #) the
offending line
systemctl daemon-reload       # if you changed a unit file instead
reboot
```

Full Diagnostic Flowchart

```
No display, no beeps at all
→ hardware/PSU/RAM issue, not a boot-chain problem – check physical hardware first

Firmware setup screen, or "No bootable device"
→ Stage 1 (BIOS/UEFI) – check boot order, disk detection, Secure Boot setting

"grub rescue>" prompt, or "error: no such partition"
→ Stage 2 (GRUB) – grub.cfg pointing at a wrong/missing partition, reinstall GRUB (
below)

Kernel panic wall of text
→ Stage 3 (kernel/initramfs) – corrupt initramfs, bad kernel param, missing driver

Boots into "emergency mode"
→ Stage 4 (systemd) – almost always fstab; check /etc/fstab first, always

Boots but a specific service is broken/slow
→ not a boot-chain issue at all – this is a service-level investigation
(journalctl -u <service>, systemd-analyze blame)
```

Reinstalling GRUB (When It's Genuinely Broken)

If GRUB itself is damaged (common after a Windows reinstall overwrites the MBR/ESP, or a failed update), you need to boot from a live USB and repair it from there.

Step 1 — Boot a live USB (Ubuntu/Debian live environment works fine for this even on other distros' broken installs).

TIP

Step 2 — Identify and mount your actual root partition:

```
lsblk          # find your real root partition, e.g. /dev/sda2
sudo mount /dev/sda2 /mnt
sudo mount /dev/sda1 /mnt/boot/efi # only if UEFI – mount the ESP too
```

TIP

Step 3 — Bind the necessary system directories and chroot in:

```
for dir in /dev /dev/pts /proc /sys /run; do sudo mount --bind $dir /mnt$dir; done
sudo chroot /mnt
```

TIP

Step 4 — Reinstall GRUB from inside the chroot:

```
grub-install /dev/sda          # legacy BIOS – target the whole disk, not a
partition
# or, for UEFI:
grub-install --target=x86_64-efi --efi-directory=/boot/efi --bootloader-id=GRUB
update-grub
```

TIP**Step 5 — Exit and reboot:**

```
exit
sudo reboot
```

Quick Reference

STAGE 1 (firmware) detection	No display / "no bootable device"	→ check boot order, disk
STAGE 2 (GRUB) reinstall GRUB	"grub rescue>" prompt	→ grub.cfg broken,
STAGE 3 (kernel) param	Kernel panic wall of text	→ bad initramfs/kernel
STAGE 4 (systemd) FIRST	"emergency mode"	→ check /etc/fstab
DIAGNOSTIC COMMANDS		
systemd-analyze	→ total boot time, by stage	
systemd-analyze blame	→ slowest units, sorted	
journalctl -b -1	→ previous boot's full log	
journalctl -k	→ kernel messages only	

systemd Deep Dive — Unit Files & Timers (The Modern Cron Replacement)

Units, targets, journalctl and timers — run services properly on any modern distro.

Level: Intermediate • Words: 1,758 • Sections: 12
Commands: env, journalctl, systemctl

Introduction

Writing your own systemd units properly, and why timers beat cron for anything you actually want to monitor and trust.

Unit Basics

Unit types you'll actually use

Extension	Purpose
<code>.service</code>	a long-running or one-shot process
<code>.timer</code>	schedules another unit to run — the cron replacement
<code>.socket</code>	socket-activated services (start on first connection, not boot)
<code>.mount</code> / <code>.automount</code>	filesystem mounts as managed units
<code>.target</code>	a named grouping/synchronization point (like <code>multi-user.target</code>)
<code>.path</code>	trigger a unit when a file/directory changes

Where unit files live — and why location matters

Location	Priority	Use for
<code>/etc/systemd/system/</code>	Highest — overrides everything below	Your own custom units go here
<code>/run/systemd/system/</code>	Runtime-only, not persisted across reboot	Rarely used manually
<code>/lib/systemd/system/</code> (or <code>/usr/lib/systemd/system/</code>)	Lowest — package-installed defaults	Managed by <code>apt/dnf</code> — never hand-edit these directly

TIP

Rule: always create your own units in `/etc/systemd/system/`. If you need to modify a package-provided unit instead of writing a new one, use an override (see section 6) rather than editing the package's file directly — your edit would be silently overwritten on the next package update.

Anatomy of a Service Unit File

```
# /etc/systemd/system/myapp.service

[Unit]
Description=My Application
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=myappuser
Group=myappuser
WorkingDirectory=/opt/myapp
ExecStart=/usr/bin/python3 /opt/myapp/main.py
Restart=on-failure
RestartSec=5
Environment=APP_ENV=production
EnvironmentFile=/opt/myapp/.env
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

[Unit] section

Directive	Meaning
Description	human-readable label, shown in systemctl status
After=	pure ordering — start this unit after the listed ones, but doesn't force them to be pulled in as dependencies by itself
Before=	the reverse of After=
Requires=	a hard dependency — if the required unit fails, this one is stopped too
Wants=	a soft dependency — pulls in the listed unit if possible, but doesn't fail this one if it doesn't start
Conflicts=	this unit cannot run at the same time as the listed one

TIP

The single most common mistake: using only **After=network.target** for something that needs actual internet access. **network.target** just means "networking subsystem initialized," not "network is actually up and has connectivity" — for anything that makes outbound connections at startup, use **After=network-online.target** and **Wants=network-online.target** together, as shown above.

[Service] section

Directive	Meaning
Type=	how systemd should treat this process (see section 3)
ExecStart=	the command that actually runs — must be an absolute path
ExecStop=	optional custom stop command (systemd sends SIGTERM by default if omitted)
ExecReload=	optional command for systemctl reload (e.g. send SIGHUP instead of a full restart)
Restart=	when to auto-restart (see section 4)
RestartSec=	delay before a restart attempt
User= / Group=	run as this user instead of root — always set this unless you specifically need root
WorkingDirectory=	the directory the process starts in — matters for relative file paths in your app
Environment=	inline environment variables (repeat the line for multiple)
EnvironmentFile=	load environment variables from a file (your .env pattern from the important-files guide)

`[Install]` section

Directive	Meaning
<code>WantedBy=multi-user.target</code>	which target pulls this in when enabled — <code>multi-user.target</code> is the standard choice for a normal background service

TIP

Note: `[Install]` only matters for `systemctl enable` — it has zero effect on `systemctl start`. A unit with no `[Install]` section can still be started manually, it just can't be enabled to start automatically at boot.

Service Types — Picking the Right One

Type	Use when
<code>simple</code> (default)	the process started by <code>ExecStart</code> stays in the foreground and IS the service — the overwhelming majority of custom scripts/apps
<code>forking</code>	a traditional daemon that forks into the background and the parent exits — systemd needs <code>PIDFile=</code> to track the actual running process
<code>oneshot</code>	a task that runs once and exits, not a long-running process — pair with <code>RemainAfterExit=yes</code> if later units should treat it as "active" even after it exits
<code>notify</code>	the app itself tells systemd when it's actually ready via <code>sd_notify()</code> — more precise startup ordering, requires app-level support
<code>idle</code>	delays execution until other jobs have finished — mostly cosmetic, avoids interleaving output on boot

For your own scripts (Python, Node, Dart/Flutter backend processes, etc.), `simple` is almost always correct unless the process explicitly daemonizes itself.

Restart Policies & Preventing Restart Loops

```
Restart=on-failure
RestartSec=5
StartLimitIntervalSec=60
StartLimitBurst=3
```

`Restart=` value	Restarts when...
<code>no</code> (default)	never auto-restarts
<code>on-success</code>	only if it exited cleanly (0) — unusual, rarely what you want
<code>on-failure</code>	exited with a non-zero code, or was killed by a signal — the common choice
<code>on-abnormal</code>	crashed, was killed, or timed out — but not a clean non-zero exit
<code>always</code>	restarts no matter how it exited, including clean shutdowns — use carefully

TIP

`StartLimitIntervalSec/StartLimitBurst` matters more than it looks: without a limit, a genuinely broken service (crashes instantly every time) will restart in an infinite tight loop, burning CPU and flooding your logs. The example above says "if it fails 3 times within 60 seconds, stop trying and give up" — check `systemctl status` afterward, it'll show `Failed to start` and stop attempting further restarts until you manually intervene.

Managing Your Unit


```

sudo systemctl daemon-reload      # REQUIRED after creating or editing any unit file
— systemd caches unit definitions
sudo systemctl start myapp
sudo systemctl stop myapp
sudo systemctl restart myapp
sudo systemctl reload myapp        # only works if ExecReload= is defined
sudo systemctl enable myapp        # start automatically on boot
sudo systemctl enable --now myapp   # enable AND start in one command
sudo systemctl status myapp
sudo systemctl disable myapp

```

TIP

The mistake that costs the most debugging time: editing a unit file and then just running `systemctl restart` — without `daemon-reload` first, systemd is still running on the old cached definition, and your changes silently don't apply.

Overriding a Package-Provided Unit (Without Editing It Directly)

```
sudo systemctl edit nginx
```

This opens an editor for a small override snippet (stored separately, under `/etc/systemd/system/nginx.service.d/override.conf`) — your changes layer on top of the original unit instead of replacing it, and survive package updates cleanly.

```

# only need to specify the directives you're actually changing
[Service]
Restart=always
RestartSec=10

```

systemd Timers — The Modern Cron Replacement

A timer is a separate unit that triggers another unit (almost always a same-named `.service`) on a schedule — cron's job, but integrated into systemd with real dependency awareness, native logging, and missed-run recovery.

The pairing convention

```

myjob.service    ← what to actually run (Type=oneshot, no [Install] needed)
myjob.timer      ← when to run it

```

TIP

The service half:

```

# /etc/systemd/system/myjob.service
[Unit]
Description=My scheduled job

[Service]
Type=oneshot
ExecStart=/opt/scripts/myjob.sh

```

TIP

The timer half:

```
# /etc/systemd/system/myjob.timer
[Unit]
Description=Run myjob every 15 minutes

[Timer]
OnCalendar=*:0/15
Persistent=true

[Install]
WantedBy=timers.target
```

Enable the TIMER, not the service — the timer is what needs to be enabled/started; it triggers the service on schedule:

```
sudo systemctl daemon-reload
sudo systemctl enable --now myjob.timer
```

`OnCalendar=` syntax — more readable than cron once you know it

```
OnCalendar=*-*-* 04:00:00      # every day at 4:00 AM
OnCalendar=Mon *-*-* 09:00:00 # every Monday at 9:00 AM
OnCalendar=*:0/15             # every 15 minutes
OnCalendar=hourly             # shorthand — every hour
OnCalendar=daily              # shorthand — every day at midnight
OnCalendar=weekly             # shorthand — every Monday at midnight
```

TIP

Test what a calendar expression actually resolves to, before trusting it:

```
systemd-analyze calendar "Mon *-*-* 09:00:00"
```

This prints the next several actual trigger times — an easy way to sanity-check your syntax without waiting for it to fire.

Monotonic timers — relative to boot/activation instead of a fixed clock time

```
[Timer]
OnBootSec=10min      # 10 minutes after boot
OnUnitActiveSec=1h   # 1 hour after this timer last activated its service — good
for "run every hour, but not at a fixed clock time"
```

`Persistent=true` — the feature cron simply doesn't have

If the machine is **off** when a scheduled run would have happened (laptop asleep, server rebooting for updates), **Persistent=true** makes systemd run the missed job as soon as the system is back up, instead of just silently skipping it until the next scheduled time. This alone is a strong reason to prefer timers over cron for anything that genuinely needs to run — backups, certificate renewal checks, monitoring scripts.

Timers vs Cron — Direct Comparison

	cron	systemd timers
Logging	Job output isn't captured by default — you must manually redirect (>> <code>logfile.log 2>&1</code>) or it vanishes	Captured in <code>journalctl -u myjob.service</code> automatically, no extra config
Missed runs (machine was off)	Silently skipped, no recovery	Persistent=true catches up automatically
Dependency awareness	None — cron doesn't know if the network/a mount/another service is ready	Full systemd dependency system — After= , Requires= , etc. all apply

	cron	systemd timers
Manual testing	Have to wait for the schedule, or manually run the underlying command yourself	<code>systemctl start myjob.service</code> runs it immediately, on demand, independent of the timer
Resource control	None built-in	Can apply the same resource limits as any systemd service (<code>MemoryMax=</code> , <code>CPUQuota=</code> , etc.)
Per-user jobs	<code>crontab -e</code> per user, separate mechanism	<code>systemctl --user</code> timers, same unit file format
Syntax	Terse, easy to get subtly wrong (* * * * *) ordering is easy to misremember)	More verbose but more explicit, and testable with <code>systemd-analyze calendar</code>

TIP

When cron still makes sense: extremely simple personal scripts on a system where you don't care about missed-run recovery or structured logging, and you already know cron syntax cold. For anything you'd actually call "AIOps" — health checks, backups, monitoring, log rotation triggers — timers are the better default going forward.

Practical Example — A Health-Check Job (AIOps-Flavored)

A script that checks a service's health every 5 minutes, logs structured output to the journal, and would be trivial to wire into an alerting pipeline later.

```
#!/usr/bin/env bash
# /opt/scripts/health-check.sh
set -euo pipefail

log() { echo "[$(date '+%Y-%m-%d %H:%M:%S')] $*"; }

if curl -sf http://localhost:8080/health > /dev/null; then
    log "OK: service healthy"
else
    log "FAIL: service health check failed"
    exit 1      # non-zero exit – this is what lets systemd/monitoring tools recognize
failure
fi
```

```
# /etc/systemd/system/health-check.service
[Unit]
Description=Health check for myapp

[Service]
Type=oneshot
ExecStart=/opt/scripts/health-check.sh
```

```
# /etc/systemd/system/health-check.timer
[Unit]
Description=Run health-check every 5 minutes

[Timer]
OnCalendar=*:0/5
Persistent=true

[Install]
WantedBy=timers.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now health-check.timer
```

TIP

Now every run's outcome is queryable with standard tools:

```
journalctl -u health-check.service --since today      # full history, timestamped,
structured
journalctl -u health-check.service -p err            # ONLY the failures
systemctl list-timers health-check.timer              # confirm next scheduled
run time
```

This is the exact advantage over a cron-based version of the same script — no separate log file to manage, no missing timestamps, and `journalctl -p err` gives you a pre-filtered failure history for free.

Debugging Units

```
sudo systemd-analyze verify /etc/systemd/system/myapp.service  # catches syntax
errors BEFORE you try to start it
sudo systemctl status myapp                                     # current state +
last few log lines
journalctl -u myapp -f                                          # live tail
journalctl -u myapp --since "10 min ago"
systemctl list-timers --all                                     # every timer,
and its next scheduled run
systemctl list-units --failed                                   # everything
currently in a failed state, system-wide
```

Quick Reference

```
CREATE A UNIT          /etc/systemd/system/<name>.service
AFTER ANY EDIT         sudo systemctl daemon-reload    (always, no exceptions)
ENABLE + START NOW     sudo systemctl enable --now <name>
CHECK STATUS           systemctl status <name>
LIVE LOGS              journalctl -u <name> -f

TIMER PAIR             <name>.service (Type=oneshot, no [Install])
                      <name>.timer  (OnCalendar=..., Persistent=true,
WantedBy=timers.target)
ENABLE THE TIMER       sudo systemctl enable --now <name>.timer
TEST A SCHEDULE EXPRESSION  systemd-analyze calendar "<expression>"
SEE ALL SCHEDULED TIMERS   systemctl list-timers --all
```

Process & Performance Monitoring — Diagnosing Slow or Crashing Services

ps, top, htop and load averages — find the process eating your CPU and read performance signals with confidence.

Level: Intermediate • Words: 1,844 • Sections: 10

Commands: apt, dmesg, htop, iostat, journalctl, systemctl +3 more

Introduction

The toolkit for answering two questions that come up constantly: **"why is this slow?"** and **"why did this crash?"** — each tool below answers a different slice of that, and the real skill is knowing which one to reach for first.

The Mental Model

Symptom	Tool to reach for first
System feels sluggish overall	<code>top / htop</code> — get the big picture fast
Suspect memory pressure/swapping	<code>vmstat</code>
Suspect disk is the bottleneck	<code>iostat</code>
Need to see exactly what a process is doing right now	<code>strace</code>
Service crashed or won't start	<code>journalctl -u <service></code>

They're not competing tools — a real investigation usually chains 2–3 of these together.

`top` — The Universal Starting Point

```
top
```

Reading the header (first 5 lines)

```
top - 14:32:01 up 3 days, 4:12, 2 users, load average: 1.85, 2.10, 1.95
Tasks: 210 total, 2 running, 208 sleeping, 0 stopped, 0 zombie
%Cpu(s): 12.3 us, 3.1 sy, 0.0 ni, 83.9 id, 0.5 wa, 0.0 hi, 0.2 si, 0.0 st
MiB Mem : 15872 total, 2103 free, 9821 used, 3948 buff/cache
MiB Swap: 2048 total, 1200 free, 848 used. 4501 avail Mem
```

Load average (`1.85, 2.10, 1.95`) — the 1/5/15-minute average of how many processes were runnable (running or waiting for CPU). **This number is meaningless without knowing your core count.** Check cores with `nproc`. A load of `4.0` is fine on an 8-core machine (half-utilized) and a serious problem on a 2-core machine (2x oversubscribed).

TIP

`%Cpu(s)` breakdown:

Field	Meaning
<code>us</code>	user-space CPU time (your applications)
<code>sy</code>	kernel/system CPU time
<code>id</code>	idle
<code>wa</code>	iowait — CPU sitting idle waiting on disk I/O. High <code>wa</code> = disk is the bottleneck, not CPU
<code>si/hi</code>	soft/hard interrupt handling
<code>st</code>	stolen time — a VM waiting on its hypervisor (relevant on cloud instances)

Reading the process table columns

Column	Meaning
PID	process ID
USER	owning user
PR / NI	priority / niceness (lower NI = higher priority, range -20 to 19)
VIRT	total virtual memory the process has mapped (often much larger than actual usage — not a red flag by itself)
RES	resident memory — actual physical RAM in use, the number that usually matters
S	state: R running, S sleeping, D uninterruptible sleep (usually waiting on I/O — worth noting), Z zombie
%CPU / %MEM	current usage share
TIME+	total accumulated CPU time since the process started

Interactive commands (press while `top` is running)

Key	Effect
P	sort by %CPU (default)
M	sort by %MEM
T	sort by running time
k	kill a process (prompts for PID, then signal)
r	renice a process (change its priority)
1	toggle per-core CPU breakdown instead of the aggregate
c	toggle showing the full command path/args
q	quit

TIP

D state is worth specifically knowing: a process stuck in **D** (uninterruptible sleep) can't even be killed with **-9** until whatever I/O it's blocked on resolves — a strong signal of a disk or NFS problem, not an application bug.

`htop` — Same Data, Much Easier to Read

```
sudo apt install htop -y
htop
```

TIP

What htop adds over top:

- Color-coded, per-core CPU and memory meters at the top — no mental math needed
- **Tree view** (**F5**) — shows parent/child process relationships visually, immediately clarifies "what spawned what"
- **Search** (**F3**) and **filter** (**F4**) by process name
- Mouse support — click a process, click a column header to sort
- **F9** to kill (lets you pick the signal from a menu, not just guess **-9**)
- **F2** for setup — customize which columns/meters are shown

TIP

Practical habit: use **F5** (tree view) whenever you suspect a runaway process is spawning children — it's far faster to spot in the tree than by scanning a flat PID list.

`vmstat` — Memory, Swap & CPU in One Glance

```
vmstat 1 5      # sample every 1 second, 5 times total
```

```
procs -----memory----- --swap-- -----io---- -system-- -----cpu-----
 r  b   swpd   free   buff  cache   si   so    bi   bo    in   cs us sy id wa st
 2   0   86000 210400  98200 3800000    0    0    45   120   850  1200 12  3 84  1  0
```

Column	Meaning
r	processes waiting for CPU time right now — sustained high values mean CPU is genuinely the bottleneck
b	processes blocked, waiting on I/O
swpd	amount of memory swapped to disk
free	free memory
buff / cache	memory used for buffers/page cache — this is not wasted memory, the kernel reclaims it instantly when needed
si / so	swap in / swap out (per second) — any sustained non-zero value here is a real problem , it means the system is actively thrashing to disk
bi / bo	blocks in / out from block devices — raw disk I/O activity
in	interrupts per second
cs	context switches per second — very high values can indicate excessive process/thread switching overhead
us/sy/id/wa/st	same meaning as top's CPU breakdown

TIP

Diagnostic pattern to remember: **si/so** staying above zero across multiple samples = the system doesn't have enough RAM for its workload and is actively swapping — this alone can explain "everything feels slow" better than any CPU metric will.

`iostat` — Disk I/O Specifically

```
sudo apt install sysstat -y      # provides iostat
iostat -x 1 5                    # extended stats, every 1 second, 5 times
```

```
Device    r/s    w/s   rkB/s   wkB/s  avgqu-sz  await  %util
sda        2.00   45.00  128.00  5400.00    0.85   12.30   38.5
```

Column	Meaning
r/s / w/s	reads/writes per second
rkB/s / wkB/s	throughput
avgqu-sz	average queue length — I/O requests waiting their turn. Consistently above ~1 means the disk can't keep up with demand
await	average time (ms) an I/O request waits, including queue time — the number that best reflects what an application actually experiences

Column	Meaning
%util	percentage of time the device was busy servicing requests — not the same as "percentage of capacity used"; a single-queue device can hit 100% util while still having room, but sustained high util alongside rising await is a genuine bottleneck

TIP

Diagnostic pattern: if `top/vmstat` shows high **wa** (iowait), `iostat -x` is the next stop — it tells you which device and how badly.

`strace` — Seeing Exactly What a Process Is Doing

strace traces every system call a process makes — the actual requests it sends to the kernel (open a file, read, write, connect a socket, etc). This is the tool for "why is this hanging" or "why is this failing" when the application's own error message isn't enough.

```
sudo apt install strace -y
```

Attach to an already-running process

```
sudo strace -p <PID>
```

Watch what it's doing live — if it's stuck, the **last line printed** is almost always the syscall it's blocked on (commonly `read()`, `connect()`, or `futex()` for a lock).

Trace a command from the start

```
strace ./myprogram
strace -f ./myprogram      # -f follows child processes too – essential for multi-
                             process programs
```

Filter to a specific category of syscalls

```
strace -e trace=network ./myprogram      # only network-related syscalls (connect, send,
recv...)
strace -e trace=file ./myprogram          # only file-related syscalls (open, read,
write, close...)
strace -e trace=open,openat ./myprogram   # just file-open attempts – great for "why
can't it find this file"
```

Summary mode — which syscalls are eating the most time

```
strace -c ./myprogram
```

```
% time      seconds  usecs/call   calls   syscall
-----
 62.15      0.004821    120         40  read
 21.30      0.001652     82         20  write
```

This immediately tells you whether a slow program is spending its time on file I/O, network calls, or something else entirely — without guessing.

The single most common real-world use

```
sudo strace -f -e trace=open,openat -p <PID> 2>&1 | grep ENOENT
```


Catches "file not found" errors live — invaluable for chasing down a service that's failing because it can't find a config file or dependency, when the application's own error message is too vague to say which file.

TIP

Note: attaching `strace` to a live process does add real overhead — it can noticeably slow the process down. Fine for debugging, not something to leave running on a production service under load.

`journalctl` — Actually Diagnosing a Service

Every systemd-managed service's output goes into the journal — this is where you go once `top/vmstat/iostat` have told you something's wrong and you need to know specifically what a service is doing.

The essentials

```
journalctl -u nginx          # all logs for the nginx service, oldest first
journalctl -u nginx -f       # live tail, like tail -f
journalctl -u nginx --since "10 min ago"
journalctl -u nginx --since "2026-08-20 09:00" --until "2026-08-20 10:00"
journalctl -u nginx -n 50    # just the last 50 lines
journalctl -u nginx -r       # reverse order, newest first – good for "what
just happened"
```

Filtering by severity

```
journalctl -u nginx -p err    # only error-level and worse
journalctl -u nginx -p warning # warning and worse
```

Priority levels, worst to least severe: `emerg`, `alert`, `crit`, `err`, `warning`, `notice`, `info`, `debug`.

System-wide, not just one service

```
journalctl -b                # everything since the current boot
journalctl -b -1              # everything from the PREVIOUS boot – crucial after an
unexpected reboot/crash
journalctl -k                  # kernel messages only (equivalent to dmesg, but
persistent and filterable)
journalctl -x                  # adds extra explanatory context to messages that
support it
```

Diagnosing a crashed service — the actual workflow

TIP

Step 1 — Check current status first, it often tells you the exit code directly:

```
systemctl status nginx
```

```
Active: failed (Result: exit-code) since ...
Process: 1234 ExecStart=/usr/sbin/nginx (code=exited, status=1/FAILURE)
```

TIP

Step 2 — Pull the logs right around the crash:

```
journalctl -u nginx --since "5 min ago" -r
```

TIP**Step 3 — Check if the kernel killed it (Out-Of-Memory killer is a very common silent cause):**

```
journalctl -k | grep -i "killed process"
# or
dmesg | grep -i oom
```

An OOM kill often shows up as the service just "disappearing" with no application-level error at all — the kernel terminated it to save the system, and the only trace is in the kernel log, not the service's own log.

TIP**Step 4 — If it's a crash loop (keeps restarting), check the restart history:**

```
systemctl status nginx      # look at "Main PID" changing between checks, and restart
count
journalctl -u nginx | grep -i "start|stop|fail"
```

TIP**Step 5 — If logs alone aren't enough, reproduce it under strace:**

```
sudo strace -f -o trace.log /usr/sbin/nginx -g "daemon off;"
```

Running the service in the foreground under **strace** (instead of through systemd) captures the exact syscall it dies on — the highest-resolution view available short of attaching a debugger.

Putting It Together — Two Common Investigations

"The server feels slow"

```
1. top / htop          → is it CPU-bound, memory-bound, or mostly idle-but-slow?
2. vmstat 1 5          → check `wa` (iowait) and `si`/`so` (swapping)
3. iostat -x 1 5       → if wa is high, confirm which device and how saturated
4. strace -c -p <PID>   → if one process is the culprit, see what it's actually
   spending time on
```

"A service crashed or won't start"

```
1. systemctl status <service>      → immediate exit code / failure reason
2. journalctl -u <service> --since ... → logs right before the failure
3. journalctl -k | grep -i killed    → rule out OOM kill
4. strace -f <the command it runs>   → if still unclear, watch it fail live
```

Quick Reference

BIG PICTURE	top / htop	
MEMORY/SWAP	vmstat 1 5	→ watch si/so and wa
DISK I/O	iostat -x 1 5	→ watch await and %util
SYSCALL TRACE	strace -p <PID> or strace -f -c <command>	
SERVICE LOGS	journalctl -u <service> -f	
CRASH HISTORY	journalctl -u <service> --since "10 min ago" -r	
OOM CHECK	journalctl -k grep -i "killed process"	

Linux Failure Scenarios — What Actually Causes Errors & File Corruption

Disk full at 3am? Server unresponsive? Walk through common failure scenarios and their rescue playbooks.

Level: Advanced • Words: 1,746 • Sections: 12

Commands: apt, chmod, chown, dd, echo, rm +3 more

Introduction

A catalog of the specific wrong moves that break files, systems, or data on Linux — why each one happens, and how to avoid or recover from it.

Shell Redirection Traps

Reading and writing the same file in one command

```
sort file.txt > file.txt      # WRONG – truncates file.txt to empty BEFORE sort ever
                              reads it
```

TIP

Why it corrupts: the shell opens `file.txt` for writing (truncating it to 0 bytes) before the command on the left even runs. By the time `sort` tries to read it, there's nothing left to read.

TIP

Fix:

```
sort file.txt -o file.txt     # many tools support in-place output flags
# or
sort file.txt > tmp && mv tmp file.txt
```

`>` vs `>>` confusion

```
echo "new log line" > app.log  # WRONG if you meant to append – wipes the entire log
file first
```

`>` always truncates first. `>>` appends. This single character mistake is one of the most common ways people accidentally delete logs or config history.

Piping into `sudo` incorrectly

```
sudo echo "text" > /etc/somefile  # FAILS silently or with permission denied
```

TIP

Why: `sudo` only elevates the `echo` — the `>` redirection is still performed by your unprivileged shell, which can't write to a root-owned file.

TIP

Fix:

```
echo "text" | sudo tee /etc/somefile
echo "text" | sudo tee -a /etc/somefile  # append version
```

Killing a Process Mid-Write

If you `kill -9` (SIGKILL) a process — or the process crashes, or the power dies — while it's in the middle of writing a file, the write can be left **partially applied**. SIGKILL cannot be caught or handled, so the process gets zero chance to close the file cleanly or finish its write.

TIP

High-risk moments:

- Editors saving a file (`Ctrl+O` in nano, `:w` in vim) — interrupted mid-save
- Database writes, especially without journaling/WAL enabled
- `dd`, `cp`, `rsync` of large files interrupted partway
- Package manager mid-install (`apt/dpkg` writing package metadata)

TIP

Result: truncated files, half-written binaries that won't execute, or — for structured files like `/etc/passwd`-style configs — a corrupted line that breaks every tool that parses it afterward.

TIP

Mitigation:

- Prefer `SIGTERM` (`kill -15`, or plain `kill`) — gives the process a chance to finish and exit cleanly. Only escalate to `-9` if it's truly hung.
- For critical files, write to a temp file and `mv` it into place atomically (a `mv` on the same filesystem is atomic — readers never see a partial file).

Editor Swap Files & Crash Recovery

Vim `.swp` conflict

Opening a file that's already open in another Vim session (or one that crashed) triggers:

```
E325: ATTENTION
Found a swap file by the name ".file.txt.swp"
```

TIP

If you ignore this and choose "Edit anyway" while the original session is still active: you now have two Vim instances able to save over each other — whichever saves last wins, silently discarding the other session's edits.

TIP

Correct response to that prompt:

- `(O)pen Read-Only` if you just want to look
- `(Q)uit` if another session is genuinely still editing it
- `(D)elete` the swap file only if you're sure no other session is active (e.g. after a crash) — then reopen normally
- `(R)ecover` to recover unsaved changes from a crash, via `vim -r file.txt`

Nano's lack of swap-file locking

Nano (by default) does **not** warn you if the same file is open in two nano sessions — both can save independently, and the second save simply overwrites the first, with no merge and no warning. If you're editing the same file from two terminals, this is a silent data-loss trap Vim would have caught and Nano won't.

Destructive Command Mistakes

`rm -rf` with a wrong or expanded path

```
rm -rf $DIR/*      # if $DIR is unset/empty, this becomes: rm -rf /*
```

TIP

Why it's catastrophic: an unset variable expands to nothing, silently turning `$DIR/*` into `/*` — deleting from root. This is one of the most infamous Linux disaster patterns.

TIP

Mitigation:

```
set -u                # in scripts: error out on unset variables instead of
silently expanding empty
rm -rf "${DIR:?DIR not set}/*" # fails loudly instead of running against /
```

Wildcard expansion including files you didn't intend

```
rm -- -rf      # a file literally named "-rf" needs -- to avoid being parsed as flags
chmod 644 *    # if a filename starts with a dash, it can be misread as an option
```

`dd` with the wrong `of=` target

```
dd if=image.iso of=/dev/sda    # WRONG target – wipes an entire disk, not a partition
```

TIP

Why it's dangerous: `dd` has no "are you sure" prompt and no undo. Getting `/dev/sda` (whole disk) vs `/dev/sdb1` (a specific partition) wrong destroys everything on that device, silently and immediately.

TIP

Mitigation: always run `lsblk` immediately before a `dd` command to confirm the exact device name, and double/triple check `of=` before hitting Enter.

Permission & Ownership Mistakes

Recursive `chmod` on the wrong directory

```
chmod -R 777 /      # catastrophic – breaks permission-dependent security across the
entire OS
```

This breaks SSH (which refuses to work if key/config permissions are too open), `sudo` (which requires specific binary permissions), and most system services that check permission bits before trusting a file.

Recursive `chown` breaking service accounts

```
chown -R $USER:$USER /var/www    # if a webserver process expects www-data ownership,
it can no longer read its own files
```

TIP

General rule: never run `-R` permission/ownership changes against a directory you haven't fully scoped in your head first — target the narrowest path that actually needs the change.

Filesystem-Level Corruption

Unsafe shutdown / power loss during a write

If the system loses power (or is force-powered-off) while the filesystem has pending writes in its cache, files can end up with:

- Zero-length or truncated content
- Metadata pointing to data that was never actually written
- In severe cases, filesystem structure damage requiring `fsck` to repair

TIP

Mitigation: always `sudo shutdown -h now` or `sync && sudo shutdown -h now` rather than cutting power directly. `sync` flushes pending writes from memory to disk before shutdown.

Unmounting a drive while it's still in use

```
umount /mnt/usb      # fails or corrupts data if a process still has open file handles
on it
```

If a file is still being written when the device is yanked or force-unmounted, the write is left incomplete. Use `lsof /mnt/usb` first to check nothing's still using it, or `umount -l` (lazy unmount) only as a last resort — it detaches the mount point immediately but lets in-flight I/O finish in the background, which is safer than yanking hardware outright.

Disk full during a write

Writing to a filesystem that fills up mid-write leaves a truncated file with no error surfaced to some tools (depends on how the writing program checks return codes). Always check `df -h` before large writes/copies, especially into `/`, `/var`, or `/tmp`, which fill up fastest from logs and package caches.

Text Encoding & Line-Ending Corruption

Windows line endings breaking a script

```
./script.sh
# bash: ./script.sh: /bin/bash^M: bad interpreter: No such file or directory
```

TIP

Why: the file has Windows-style `\r\n` line endings instead of Unix `\n`. The shebang line (`#!/bin/bash`) ends up with an invisible `\r` (^M) appended, and the kernel can't find an interpreter called `bash\r`.

TIP**Fix:**

```
dos2unix script.sh
# or
sed -i 's/\r$//' script.sh
```

Mixing character encodings

Editing a UTF-8 file with a tool/locale expecting Latin-1 (or vice versa) can silently mangle non-ASCII characters (accents, non-Latin scripts) into garbled bytes — often visible as ■ or seemingly random characters. Check with `file filename` (reports detected encoding) and set your editor/locale (`echo $LANG`) to match the file's actual encoding.

Editing a binary file as if it were text

Opening a binary (executable, image, compiled object) in Vim/Nano and saving it can corrupt it — text editors may "normalize" line endings or encoding on save, which mangles binary data that was never meant to be interpreted as text. If you must inspect binary content, use `xxd file` or `hexdump -C file`, not a text editor's save function.

Symlink & Hardlink Pitfalls

Deleting through a symlink with a trailing slash

```
rm -rf symlink_to_dir/    # the trailing slash makes some tools follow the link and
                           delete the TARGET's contents, not just the link
```

Behavior here varies by tool and trailing-slash handling — the safe habit is to `rm` the link itself (no trailing slash) unless you specifically intend to touch the target's contents.

Broken symlinks after moving the target

Moving or renaming the file/directory a symlink points to leaves the symlink dangling — it still exists, but any read through it fails. `ls -la` shows dangling links in a distinct color in most terminals; `find -xtype l` finds all broken symlinks under a path.

Package Manager Corruption

Interrupting `apt`/`dnf` mid-install

Killing the terminal, losing power, or force-closing during a package install can leave the package database in a `dpkg` `was interrupted` state — subsequent `apt` commands refuse to run until it's repaired:

```
sudo dpkg --configure -a    # resume/repair an interrupted dpkg state
sudo apt --fix-broken install
```

Running two package managers at once

Running `apt install` in two terminals simultaneously causes a lock contention error (`Could not get lock /var/lib/dpkg/lock`) — and if the lock itself is stale from a crashed process, forcibly deleting the lock file without confirming no process actually holds it can let two installs interleave and corrupt the package database.

Concurrent Access Race Conditions

Two processes (or two terminal sessions, or a script and a human) writing to the same file at the same time — without any locking — will have the **last write win**, discarding whatever the other one wrote. This isn't Linux-specific but is extremely common in:

- Editing the same config file from two SSH sessions
- A cron job and a manual edit touching the same file simultaneously
- A script appending to a log file while log rotation is renaming/truncating it

TIP

Mitigation: for anything script-driven, use `flock` to take an exclusive lock before writing:

```
flock /tmp/mylock.lock -c "echo 'safe write' >> shared_file.txt"
```

Quick Prevention Checklist

- Never redirect (`>`) output back into a file you're still reading as input in the same command.
- Use `SIGTERM` before `SIGKILL` when stopping a process that might be mid-write.
- Write critical files to a temp path, then atomically `mv` them into place.
- Check `lsdf` before force-unmounting or removing a device.
- Always confirm `dd`'s `of=` target with `lsblk` immediately before running it.
- Quote variables and use `set -u` in scripts to catch unset-variable disasters before they run.
- Scope `-R` permission/ownership changes to the narrowest directory that needs it — never run them against `/` or a guess.
- Run `dos2unix` on any script that came from a Windows machine or editor before executing it.
- Use `flock` for any script that might run concurrently with itself or another writer.

Log Management — syslog vs journald, logrotate & Where Logs Actually Live

journald, rsyslog and log rotation — find the signal in /var/log before the noise buries it.

Level: Intermediate • Words: 1,320 • Sections: 6
Commands: cat, find, journalctl, ls, mkdir, systemctl

Introduction

Two logging systems coexist on most modern Linux distros, often confusingly overlapping. This is what each one actually does, how they hand off to each other, how rotation keeps them from filling your disk, and where to actually look for any given service's output.

The Core Confusion: syslog vs journald

	syslog (rsyslog/syslog-ng)	journald (systemd)
What it is	The traditional Unix logging daemon/protocol	systemd's own logging component
Storage format	Plain text files	Binary, indexed format
Location	<code>/var/log/*.log</code> (readable with <code>cat</code> , <code>tail</code> , <code>grep</code>)	<code>/var/log/journal/</code> (only readable through <code>journalctl</code>)
Structure	Just text lines, minimal built-in structure	Rich metadata per entry: PID, UID, unit name, boot ID, etc.
Persistence	Always persistent (it's just files)	Persistent OR volatile depending on config (see below)
Query power	<code>grep/awk</code> on plain text	Native filtering — by service, time range, priority, boot

TIP

Why both exist on the same system: journald is what actually captures logs first on any systemd-based distro (which is nearly everything now — Ubuntu, Debian, Fedora, RHEL, Arch). rsyslog then either reads from journald and writes out the traditional `/var/log/*.log` files, or runs independently for anything not going through systemd. On most modern distros, journald is the source of truth and the classic `/var/log` files are a compatibility layer built on top of it.

TIP

Practical implication: for any service managed by systemd, `journalctl -u <service>` is more complete and more precise than grepping `/var/log/syslog` — you get structured filtering (by exact service, exact boot, exact priority) that plain text grepping can only approximate.

journald — Configuration & Persistence

Is journald even keeping logs across reboots?

```
journalctl --disk-usage      # shows how much space the journal is currently using
ls /var/log/journal/        # if this directory exists and has content, logs are
persistent
```

By default on many distros, journald logs are **volatile** — kept only in `/run/log/journal` (memory-backed, wiped on reboot) unless persistent storage is explicitly enabled.

Enabling persistent journal storage

```
sudo mkdir -p /var/log/journal
sudo systemd-tmpfiles --create --prefix /var/log/journal
sudo systemctl restart systemd-journald
```

After this, `journalctl -b -1` (previous boot's logs) actually works — without it, a reboot wipes your only crash evidence.

Key config: `/etc/systemd/journald.conf`

```
[Journal]
Storage=persistent          # persistent | volatile | auto
SystemMaxUse=500M          # cap total disk space the journal can consume
SystemMaxFileSize=50M      # size of each individual journal file before rotating
MaxRetentionSec=30day      # delete entries older than this, regardless of size
```

Apply changes with:

```
sudo systemctl restart systemd-journald
```

TIP

This matters more than it looks: without `SystemMaxUse`, an unexpectedly chatty service (a crash-looping process logging constantly) can fill `/var/log/journal` and, by extension, your root filesystem — see the disk-full failure scenarios from your earlier guide. Always cap it explicitly.

Manually cleaning the journal

```
sudo journalctl --vacuum-size=200M    # shrink the journal down to 200MB
sudo journalctl --vacuum-time=7d      # delete anything older than 7 days
```

Where Every Service Actually Writes Its Logs

This is the part most guides skip — a real map of what's where.

systemd-managed services (the majority on a modern distro)

```
journalctl -u <service_name>
```

This is the authoritative source. No separate log file exists unless the service also writes its own (many do, for app-specific reasons — see below).

Classic system-wide log files (`/var/log/`)

File	Contents
<code>/var/log/syslog</code> (Debian/Ubuntu) or <code>/var/log/messages</code> (RHEL/Fedora)	General system messages, the traditional catch-all
<code>/var/log/auth.log</code> (Debian/Ubuntu) or <code>/var/log/secure</code> (RHEL/Fedora)	Authentication events — logins, <code>sudo</code> usage, SSH attempts
<code>/var/log/kern.log</code>	Kernel messages (also viewable live via <code>dmesg</code> or <code>journalctl -k</code>)
<code>/var/log/dpkg.log</code>	Package install/remove/upgrade history (Debian/Ubuntu)
<code>/var/log/boot.log</code>	Messages from the boot process
<code>/var/log/faillog</code> / <code>/var/log/lastlog</code>	Failed login attempts / last login per user

Application-specific logs (each has its own convention — worth knowing the common ones)

Service	Log location
nginx	<code>/var/log/nginx/access.log</code> , <code>/var/log/nginx/error.log</code>
Apache	<code>/var/log/apache2/access.log</code> , <code>/var/log/apache2/error.log</code> (Debian) or <code>/var/log/httpd/</code> (RHEL)
MySQL/MariaDB	<code>/var/log/mysql/error.log</code>
PostgreSQL	<code>/var/log/postgresql/postgresql-<version>-main.log</code>
Docker containers	<code>docker logs <container></code> (reads the container's own log driver output, usually JSON files under <code>/var/lib/docker/containers/<id>/</code>)
cron jobs	<code>/var/log/cron.log</code> , or wherever the job's own script redirects output (cron itself only logs that it ran, not the job's stdout, unless you explicitly redirect it)
Fail2ban	<code>/var/log/fail2ban.log</code>
UFW	<code>/var/log/ufw.log</code>

Finding a log location you don't already know

```
sudo find /var/log -name "*<servicename>*"      # search by name pattern
sudo lsof | grep -i "\.log$" | grep <process_name> # find open log files for a running
process, by name
cat /etc/<servicename>/*.conf | grep -i log      # check the service's own config
for a configured log path
```

TIP

A genuinely useful habit: when in doubt for a systemd-managed service, start with `journalctl -u <service>` regardless of whether it also has its own log file — it captures stdout/stderr from the very moment the process starts, before the application has even opened its own log file, which makes it the best source for startup failures specifically.

`logrotate` — Keeping Log Files From Growing Forever

`logrotate` handles the traditional `/var/log/*.log` files — `journal` manages its own size internally (via `SystemMaxUse`, above) and does **not** use `logrotate`.

How it works, conceptually

On a schedule (typically daily, via a cron job or systemd timer), `logrotate` checks each configured log file against rules: if it's grown too large or too old, it gets renamed (`access.log` → `access.log.1`), optionally compressed, and a fresh empty log file is started — with old rotations eventually deleted once a configured count is exceeded.

Where the config lives

```
/etc/logrotate.conf      # global defaults
/etc/logrotate.d/        # per-service config files — this is where most real
rules live
```

```
ls /etc/logrotate.d/      # nginx, apache2, mysql-server, etc — most packages install
their own rule here automatically
```

Anatomy of a logrotate config

```

/var/log/nginx/*.log {
    daily
    missingok
    rotate 14
    compress
    delaycompress
    notifempty
    create 0640 www-data adm
    sharedscripts
    postrotate
        systemctl reload nginx > /dev/null 2>&1 || true
    endscript
}

```

Directive	Meaning
daily / weekly / monthly	how often to rotate (can also be size-based, see below)
size 100M	rotate once the file exceeds this size, instead of/in addition to time-based
rotate 14	keep 14 old rotations before deleting the oldest
compress	gzip old rotations to save space
delaycompress	delay compressing the most recent rotation by one cycle — some apps still have it open briefly after rotation, this avoids a race
missingok	don't error if the log file doesn't exist
notifempty	don't rotate an empty file — no point
create 0640 www-data adm	after rotating, create a fresh file with these permissions/owner, so the service doesn't need to be restarted to start writing again
postrotate / endscript	shell commands to run after rotation — commonly used to signal the service to reopen its log file handle

TIP

Why `postrotate` matters more than it looks: many services keep a log file open by file descriptor, not by name. If you rotate the file (rename it) without telling the service, it keeps writing into the now-renamed old file — the new `access.log` stays empty forever, and you silently lose all future logging until the service is restarted. The `postrotate` hook (commonly `systemctl reload` or a `kill -HUP`) tells the service to close and reopen its log handle by name, picking up the fresh file.

Testing a logrotate config without waiting for the schedule

```

sudo logrotate -d /etc/logrotate.d/nginx    # dry run – shows what WOULD happen,
changes nothing
sudo logrotate -f /etc/logrotate.d/nginx    # force it to actually rotate right now

```

Writing your own rule for a custom app

```

/var/log/myapp/*.log {
    weekly
    rotate 8
    compress
    missingok
    notifempty
    create 0644 myappuser myappuser
}

```

Save this as `/etc/logrotate.d/myapp` — no restart of `logrotate` itself needed, it's read fresh on every scheduled run.

Practical Cheat Sheet

VIEW A SERVICE'S LOGS	<code>journalctl -u <service> -f</code>	
VIEW SYSTEM-WIDE LOGS	<code>tail -f /var/log/syslog</code>	(or <code>/var/log/messages</code> on RHEL-family)
CHECK JOURNAL DISK USAGE	<code>journalctl --disk-usage</code>	
SHRINK THE JOURNAL	<code>journalctl --vacuum-size=200M</code>	
FIND A LOG FILE	<code>sudo find /var/log -name "*<name>*"</code>	
LIST LOGROTATE RULES	<code>ls /etc/logrotate.d/</code>	
TEST A LOGROTATE RULE	<code>sudo logrotate -d /etc/logrotate.d/<name></code>	
FORCE ROTATION NOW	<code>sudo logrotate -f /etc/logrotate.d/<name></code>	

TIP

One-line mental model to keep: `journald` captures everything `systemd` starts, in a queryable binary format that self-manages its size; `logrotate` manages everything that writes plain text files to `/var/log`, on a schedule you configure per-service. Most real systems run both side by side, and knowing which one owns a given log is the fastest way to stop searching in the wrong place.



Part III — Networking & Security

From ping to SSH to firewalls and user management. — 6 guides

Networking Troubleshooting Toolkit — "Why Can't This Reach That?"

ping to tcpdump — a layered toolkit for diagnosing “the network is down”, from cable to DNS.

Level: Intermediate • Words: 1,622 • Sections: 10

Commands: apt, cat, curl, dig, mtr, nc +4 more

Introduction

The tools for isolating exactly where a connection breaks: DNS, routing, a blocked port, or the application itself. Each tool below answers a different link in that chain.

The Mental Model — Where Connections Actually Break

1. DNS → can the name even resolve to an IP?
2. Reachability → can a packet get to that IP at all?
3. Routing → what path is it taking, and where does it stop?
4. Port/Service → is the specific port open and listening?
5. Application → is the service responding correctly at the protocol level?

Every "can't connect" problem lives at exactly one of these layers — the tools below map directly onto them, in the order you should actually check them.

`ping` — Is It Reachable At All?

```
ping google.com
ping -c 4 google.com      # send exactly 4 packets, then stop (default is infinite
                           until Ctrl+C)
ping -i 0.5 google.com    # interval between packets, in seconds
ping -s 1000 google.com   # custom packet size (bytes) — useful for testing MTU
issues
ping -W 2 google.com      # timeout per packet, in seconds
```

Reading the output

```
64 bytes from 142.250.80.14: icmp_seq=1 ttl=118 time=14.2 ms
```

Field	Meaning
icmp_seq	sequence number — gaps in this sequence mean dropped packets
ttl	Time To Live remaining when it arrived — roughly indicates how many hops away the source is (starting TTL varies by OS, typically 64/128/255)
time	round-trip latency in milliseconds

```
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
```

Any non-zero packet loss here is worth investigating further — even 1-2% loss can cause real application-level flakiness.

What ping tells you — and what it doesn't

- **Tells you:** whether the host responds to ICMP at all, and rough latency/loss.
- **Doesn't tell you:** whether the actual service you care about (a web server, a database, SSH) is reachable — many firewalls block ICMP specifically while still allowing real traffic through, so **"ping fails" does not always mean "the service is down"**, and **"ping succeeds" does not mean the service is up**. Treat ping as a reachability hint, not a service check.

`traceroute` — Where Does the Path Actually Break?

How it works, briefly

It sends packets with progressively increasing TTL values (1, 2, 3...). Each router along the path decrements TTL by 1; when a packet's TTL hits 0, that router sends back an error instead of forwarding it — which is how traceroute discovers each hop, one at a time, without ever needing cooperation from the destination itself.

```
sudo apt install traceroute -y      # often not installed by default
traceroute google.com
traceroute -n google.com           # skip reverse-DNS lookups per hop — much faster,
IPs only
```

Reading the output

```
1  192.168.1.1  1.203 ms  1.150 ms  1.098 ms
2  10.10.0.1    8.552 ms  8.402 ms  8.310 ms
3  * * *
4  203.0.113.1  15.221 ms 14.998 ms 15.102 ms
```

- Three times per hop = three separate probe packets, for consistency.
- * * * means that hop didn't respond — **this is often normal**, not necessarily a failure. Many routers are configured to not respond to traceroute probes at all while still forwarding traffic perfectly fine. Only treat this as a real problem if the trace **never recovers** past that point (every subsequent hop also times out) or never reaches the destination.
- **Where the trace actually stops (and never resumes) is your strongest clue** for where the real problem is — a firewall, a dead link, or a misconfigured route right at that hop.

`mtr` — the better version for ongoing diagnosis

```
sudo apt install mtr -y
mtr google.com
```

Combines ping and traceroute into one continuously-updating view — instead of a single snapshot, you see live loss percentage and latency **per hop**, which makes intermittent problems (a flaky hop that only drops packets sometimes) far easier to spot than a one-shot [traceroute](#).

`dig` / `nslookup` — Is DNS the Problem?

DNS issues are one of the most common causes of "can't reach X" — and the easiest to misdiagnose as a routing/firewall problem if you don't check it first.

`dig` — the more detailed, modern tool


```
dig example.com                # full detailed query
dig +short example.com         # just the resolved IP – the fast version for quick
checks
dig example.com MX             # mail server records
dig example.com NS             # nameservers for the domain
dig example.com TXT            # TXT records (SPF, verification records, etc)
dig -x 8.8.8.8                 # reverse lookup – IP to hostname
dig @1.1.1.1 example.com       # query a SPECIFIC DNS server directly,
bypassing your configured resolver
```

TIP

Why querying a specific server (@1.1.1.1) matters for troubleshooting: it isolates whether the problem is your local/configured DNS resolver specifically, or the domain's DNS records themselves. If `dig @1.1.1.1 example.com` resolves fine but plain `dig example.com` doesn't, your configured resolver (often in `/etc/resolv.conf`) is the actual problem, not the domain.

Reading `dig`'s output

```
;; QUESTION SECTION:
;example.com.          IN  A

;; ANSWER SECTION:
example.com.          299 IN  A  93.184.216.34

;; Query time: 24 msec
;; SERVER: 127.0.0.53#53(127.0.0.53)
```

- **ANSWER SECTION empty** → the domain doesn't resolve at all, or you queried the wrong record type
- **SERVER line** → confirms exactly which DNS server actually answered — critical for the @ isolation trick above
- **Query time** → high values (hundreds of ms+) can indicate a slow/distant/overloaded resolver

`nslookup` — older, still common, simpler output

```
nslookup example.com
nslookup example.com 1.1.1.1    # query a specific server, same idea as dig's @ syntax
```

Checking your actual configured resolver

```
cat /etc/resolv.conf
resolvectl status    # on systemd-resolved systems – shows the real active
resolver, which can differ from resolv.conf
```

Checking a Specific Port — Is the Service Actually Listening?

DNS resolving and the host being pingable doesn't mean the specific port/service you care about is open.

```
nc -zv example.com 443    # -z = scan mode (no data sent), -v = verbose – tells you
open/closed instantly
telnet example.com 443    # older alternative, connects and leaves you in an
interactive session if it succeeds
```

TIP

On the local machine, check what's actually listening:

```
ss -tulnp
# -t tcp -u udp -l listening only -n numeric ports -p show the owning process
```

If a service isn't in this list at all, it's not the network's fault — the service itself isn't running or isn't bound to the port/interface you expect.

`tcpdump` — See the Actual Packets

When ping/traceroute/dig/port-checks all look fine but something's still wrong, `tcpdump` shows you the raw truth of what's actually going out and coming back.

```
sudo tcpdump -i eth0          # capture on a specific interface (find names
with ip a)
sudo tcpdump -i any           # capture on all interfaces
sudo tcpdump -i eth0 host 203.0.113.5    # filter to one specific IP
sudo tcpdump -i eth0 port 443          # filter to one specific port
sudo tcpdump -i eth0 host 203.0.113.5 and port 443  # combine filters
sudo tcpdump -i eth0 -n          # skip DNS resolution on captured IPs –
faster, cleaner output
sudo tcpdump -i eth0 -w capture.pcap      # save raw capture to a file (open later
in Wireshark for deep analysis)
sudo tcpdump -r capture.pcap             # read back a saved capture
```

A genuinely common diagnostic pattern

```
# On the SERVER, check if a client's connection attempt is even arriving:
sudo tcpdump -i eth0 port 22
# then attempt to SSH in from the client
```

If you see the SYN packet arrive but no response goes back → the OS-level firewall (ufw/iptables) is silently dropping it — the packet reaches the box, the service just never gets a chance to answer. **If you see nothing arrive at all** → the problem is upstream — router port forwarding, an ISP-level block, or the packet never left the client's own network correctly.

This single check is often the fastest way to definitively answer "is it a firewall problem or a routing problem" — something ping/traceroute alone can't always distinguish.

Reading basic tcpdump output

```
14:32:01.123456 IP 203.0.113.5.54321 > 10.0.0.5.22: Flags [S], seq 123456, win 64240
```

- `Flags [S]` = SYN (connection attempt)
- `Flags [S.]` = SYN-ACK (server accepting)
- `Flags [.]` = ACK
- `Flags [R]` = RST (connection actively refused/reset — different from a silent drop)
- A `[S]` with no `[S.]` reply within the capture = the target isn't responding, consistent with either a firewall drop or the service not listening

`curl -v` — When It's an Application-Layer Problem, Not a Network One

Once you've confirmed the port is open and packets are flowing, but something's still wrong (wrong response, redirect loop, cert issue), drop to the application layer:

```
curl -v https://example.com
```

Shows the full handshake: DNS resolution, TCP connect, TLS handshake details, request headers sent, response headers received — the layer above what tcpdump shows you, and often exactly where an "it connects but doesn't work right" problem actually lives (expired cert, wrong Host header, unexpected redirect).

The Full Diagnostic Workflow

"I can't reach `service.example.com` on port 443"

- | | |
|---|---|
| 1. <code>dig service.example.com</code>
here – it's DNS. | → does the name even resolve? If not, stop |
| 2. <code>ping service.example.com</code>
failure here isn't

treat as a hint) | → is the host reachable at all? (remember:

definitive if ICMP might be blocked – |
| 3. <code>nc -zv service.example.com 443</code>
often more reliable | → is the SPECIFIC port open? This is

than ping for a real answer. |
| 4. <code>tracert / mtr service.example.com</code>
actually stop? | → if unreachable, WHERE does the path |
| 5. <code>sudo tcpdump -i eth0 port 443</code>
it) does the

it get a response? | → (from the server side, if you control

connection attempt even arrive? Does |
| 6. <code>curl -v https://service.example.com</code>
still wrong, | → if the port IS open but something's

check the application-layer exchange |

Run these roughly in order — each one either confirms that layer is fine (move to the next) or points you straight at the actual problem, without guessing.

Quick Reference

REACHABILITY	<code>ping -c 4 <host></code>	
PATH/ROUTING	<code>tracert -n <host></code>	or <code>mtr <host></code>
DNS	<code>dig +short <host></code>	<code>dig @<server> <host></code> (isolate resolver vs domain)
PORT CHECK	<code>nc -zv <host> <port></code>	
LOCAL LISTENERS	<code>ss -tulnp</code>	
PACKET CAPTURE	<code>sudo tcpdump -i <iface> host <ip> and port <port></code>	
APPLICATION LAYER	<code>curl -v https://<host></code>	

FASTEST FIREWALL-VS-ROUTING TEST: `tcpdump` on the server while connecting from the client –
 packet arrives + no response = firewall drop. packet never arrives = routing/upstream problem.

SSH Remote Access From Outside Your Network — Complete Guide

Keys, agents, tunnels and ssh_config — remote access done securely without password fatigue.

Level: Intermediate • Words: 1,705 • Sections: 13

Commands: apt, curl, ip, journalctl, passwd, ssh +5 more

Introduction

How to reach a Linux machine over SSH from anywhere on the internet, not just your local network — plus the hardening steps that matter once that machine is actually exposed.

Two Ways to Do This

Approach	How it works	Best for
Port forwarding	Open a port on your router, forward it to the machine's local IP	Full control, no third party, but exposes your home IP directly to the internet
Tunneling service (Tailscale, Cloudflare Tunnel, reverse SSH via VPS)	No port forwarding at all — an outbound connection from your machine creates the path in	Far safer, easier, works behind CGNAT/no router access — recommended for most personal setups

This guide covers the direct port-forwarding method in full (since that's what you asked for), then the safer alternative at the end — genuinely worth reading before you commit to opening a port.

Install & Enable the SSH Server

On the machine you want to connect into (the "server"):

```
sudo apt update
sudo apt install openssh-server -y
sudo systemctl enable ssh --now      # start now, and on every boot
sudo systemctl status ssh            # confirm it's active (running)
```

Find Your IPs

You need two different IPs — don't confuse them.

Local (internal) IP — how devices on your own network see this machine:

```
ip a          # look under your active interface (eth0/wlan0), the inet line
hostname -I   # quicker, just prints the IP(s)
```

Public IP — how the internet sees your entire network (this is what you'll actually connect to from outside):

```
curl ifconfig.me
# or open https://whatismyip.com in a browser
```

Give the server machine a fixed local IP. If it's on DHCP, its local IP can change after a reboot, breaking your port forward. Either:

- Set a **static IP** on the machine itself, or
- Set a **DHCP reservation** in your router (binds a specific local IP to this machine's MAC address permanently) — this is the cleaner option since the machine's config stays untouched.

Forward the Port on Your Router

- Log into your router's admin panel — usually `192.168.1.1` or `192.168.0.1` in a browser (check the router's label, or run `ip route | grep default` to find your gateway IP).
- Find **Port Forwarding** (sometimes under "NAT," "Virtual Server," or "Firewall").
- Create a rule:
- **External port:** e.g. `2222` (see the security note below on why not `22`)
- **Internal port:** `22`
- **Internal IP:** the server's local IP from Step 2
- **Protocol:** TCP
- Save and apply — most routers apply this instantly, some need a reboot.

Open the Port on the Server's Own Firewall

Port forwarding at the router is useless if the machine's local firewall still blocks the connection.

```
sudo ufw allow 22/tcp          # or your custom port, e.g. sudo ufw allow 2222/tcp
sudo ufw enable
sudo ufw status
```

Connect From Outside

From another network entirely (mobile data is a good real test — your own network won't prove it works from outside):

```
ssh -p 2222 solo@<your_public_ip>
```

If you forwarded external `2222` → internal `22`, you must specify `-p 2222` on the client side — the external and internal ports don't have to match, and here they intentionally don't.

Connecting as Root vs. as a Regular User — Two Ways In

Once your port forward and firewall are set up, who you log in as matters a lot for security. Here's both approaches, step by step.

6.1 Method 1 — Connect as a Regular User, Then Elevate (Recommended)

This is the standard, safe pattern: SSH in as an unprivileged user, then become root only for the specific commands that need it.

TIP

Step 1 — SSH in as your normal user:

```
ssh -p 2222 solo@<public_ip_or_ddns_hostname>
```

TIP

Step 2 — Confirm who you are once connected:

```
whoami    # prints your regular username, e.g. "solo"
```

TIP

Step 3 — Run an individual admin command with `sudo`:

```
sudo apt update
```

You're prompted for **your own** password (the account you SSH'd in as), not root's.

TIP

Step 4 — If you need several root commands in a row, get a full root shell:

```
sudo -i
whoami    # now prints "root"
```

TIP

Step 5 — Exit root, then exit the SSH session entirely when done:

```
exit      # drops out of the root shell, back to your normal user
exit      # closes the SSH connection
```

6.2 Method 2 — Connect Directly as Root

Technically possible, but disabled by default on most distros, and actively discouraged for anything internet-facing — every login attempt against a machine's SSH port is now guessing the known username "root" instead of also having to guess a username. Shown here for completeness and for local/lab use.

Step 1 — Give root a password (it has none set by default on Ubuntu/Debian):

```
sudo passwd root
```

Step 2 — Temporarily allow root login over SSH — edit `/etc/ssh/sshd_config` on the server:

```
PermitRootLogin yes
```

TIP

Step 3 — Restart the SSH service to apply it:

```
sudo systemctl restart ssh
```

TIP

Step 4 — Connect directly as root from the client:

```
ssh -p 2222 root@<public_ip_or_ddns_hostname>
```

Step 5 — Revert it once you're done (this is the important part — don't leave this open):

```
# back to /etc/ssh/sshd_config
PermitRootLogin no
```

```
sudo systemctl restart ssh
```

6.3 Which One Should You Actually Use?

Situation	Use
Any machine reachable from the internet (your port-forwarded setup)	Method 1 — <code>PermitRootLogin no</code> , connect as a regular user, <code>sudo</code> when needed

Situation	Use
Local-only lab machine, never exposed outside your LAN	Either is fine, Method 1 is still better practice
You just need to run one root-level command remotely	<code>ssh -p 2222 solo@host "sudo <command>"</code> — skips a full interactive login entirely

TIP

Bottom line: treat direct root login the same way you'd treat leaving your front door key under the mat — it works, but it's the first thing anyone probing your server will try. Method 1 with `sudo` gives you the exact same power with a username an attacker actually has to guess first, plus a clean audit trail of every elevated command in `/var/log/auth.log`.

Your Public IP Keeps Changing — Fix It With Dynamic DNS

Most home internet connections have a **dynamic** public IP — it changes periodically, breaking anything hardcoded to today's address. Dynamic DNS (DDNS) gives you a stable hostname that always points at your current IP.

TIP

Free options: DuckDNS, No-IP, Cloudflare (if you own a domain).

TIP

DuckDNS example:

- Create a free subdomain at duckdns.org, e.g. `solo-home.duckdns.org`.
- Install their update script (cron job or systemd timer) on the server — it pings DuckDNS every few minutes with your current IP.
- From then on, connect with the hostname instead of a raw IP:

```
ssh -p 2222 solo@solo-home.duckdns.org
```

Security Hardening — Don't Skip This

The moment you forward SSH to the internet, automated bots **will** find and hammer it within hours — this isn't hypothetical, it's guaranteed. These steps aren't optional polish, they're the difference between "exposed but safe" and "compromised."

Switch to key-based authentication, then disable passwords entirely

```
# on your CLIENT machine (not the server):
ssh-keygen -t ed25519
ssh-copy-id -p 2222 solo@<server_ip>    # copies your public key to the server
```

Then on the **server**, edit `/etc/ssh/sshd_config`:

```
PasswordAuthentication no
PubkeyAuthentication yes
```

```
sudo systemctl restart ssh
```

This alone eliminates the entire brute-force-password attack surface — a key can't be guessed the way a password can.

Disable root login

```
# /etc/ssh/sshd_config
PermitRootLogin no
```

Change the default port

Set **Port 2222** (or any non-standard number) in `/etc/ssh/sshd_config`. This doesn't add real cryptographic security, but it silences the vast majority of dumb automated scanners that only ever check port 22 — your logs get dramatically quieter and easier to actually monitor.

Restrict which users can SSH in at all

```
# /etc/ssh/sshd_config
AllowUsers solo
```

Install fail2ban

Automatically bans IPs that fail login too many times:

```
sudo apt install fail2ban -y
sudo systemctl enable fail2ban --now
```

Rate-limit connections at the firewall too

```
sudo ufw limit 2222/tcp      # throttles repeated connection attempts from the same IP
```

Keep the system patched

```
sudo apt update && sudo apt upgrade -y    # do this regularly – SSH/OpenSSH CVEs do
happen
```

After all of the above, re-check `sudo systemctl restart ssh` and re-test your connection before closing your current session — if `sshd_config` has a typo, you don't want to be locked out with no way back in.

The Safer Alternative: Skip Port Forwarding Entirely

Worth strong consideration, especially for a personal/home setup:

Tailscale (recommended)

Creates a private mesh VPN between your devices — no router config, no open ports, works even behind CGNAT (which port forwarding can't solve at all).

```
curl -fsSL https://tailscale.com/install.sh | sh
sudo tailscale up
```

Every device you install it on gets a stable private IP (`100.x.x.x`) that only your devices can reach — then just `ssh solo@<tailscale-ip>` from anywhere, with zero exposure to the public internet.

Cloudflare Tunnel

Similar idea — an outbound-only tunnel from your server to Cloudflare, no inbound port ever opened.

Reverse SSH tunnel via a cheap VPS

If you already have a VPS, the server can dial out to it and hold a tunnel open, letting you reach it through the VPS without ever forwarding a port at home:

```
ssh -R 2222:localhost:22 you@your-vps-ip
```


TIP

Why this matters: port forwarding exposes your home router directly to internet-wide scanning. A mesh VPN like Tailscale gets you the exact same remote-access outcome with essentially none of that exposure — for a personal setup it's very hard to justify skipping it.

Troubleshooting

Symptom	Likely cause
Connection timed out	Port forward isn't actually applied, or your ISP uses CGNAT (your "public" IP isn't really public — common on mobile/some ISPs)
Connection refused	SSH service isn't running, or local firewall (ufw) is blocking the port
Works on local network, fails from outside	Router port-forward rule is wrong, or pointing at the wrong internal IP
Prompted for password even after key setup	PasswordAuthentication wasn't actually set to no , or the public key wasn't copied to the right user's <code>~/.ssh/authorized_keys</code>
Locked out after editing sshd_config	Keep your current session open while testing a second connection in a new terminal — never close your only working session until the new config is confirmed working

Check live logs while debugging:

```
sudo journalctl -u ssh -f
```

Quick Setup Checklist

- ☐ **openssh-server** installed and running
- ☐ Static local IP or DHCP reservation on the server
- ☐ Router port forward: external port → server's local IP:22
- ☐ **ufw allow <port>/tcp** on the server itself
- ☐ Tested from an actual outside network (mobile data)
- ☐ Dynamic DNS set up if your public IP isn't static
- ☐ SSH key auth working, then **PasswordAuthentication no**
- ☐ **PermitRootLogin no**
- ☐ Non-default port set
- ☐ **fail2ban** installed and running
- ☐ Considered Tailscale instead of all of the above

Firewall Deep Dive — ufw vs iptables vs nftables

firewalld and nftables in depth — zones, rules and the exact commands that open (or close) a port safely.

Level: Advanced • Words: 1,755 • Sections: 8
Commands: apt, dmesg, head/tail, ip, iptables, nano +5 more

Introduction

Three tools, one underlying job: deciding what network traffic gets in, out, or forwarded. Here's how they relate, and how to actually write rules beyond `ufw allow 22`.

How These Three Actually Relate

nftables → the modern kernel packet-filtering framework (replaces the old netfilter/iptables backend)
iptables → legacy tool – on current systems, often just translates its commands into nftables rules underneath
ufw → a friendly frontend that generates iptables (or nftables, depending on distro version) rules for you

They're not three unrelated options — `ufw` is built on top of `iptables/nftables`, and modern `iptables` itself is frequently just a compatibility shim over the real nftables engine now running in the kernel. Which one you interact with directly is a tradeoff between simplicity (`ufw`) and raw control (`iptables/nftables`).

	ufw	iptables	nftables
Learning curve	Lowest	Moderate	Moderate-high (but cleaner once learned)
Underlying engine	iptables or nftables (distro-dependent)	netfilter (legacy syntax)	netfilter (modern syntax)
Best for	Quick, common rules on a single host	Fine-grained control, huge existing knowledge base online	New setups, complex rule sets, performance at scale
Status	Actively used, wraps the others	Legacy but still everywhere	The actual current standard going forward

`ufw` — Beyond Just `allow`

The basics, quickly

```
sudo ufw status verbose      # see current rules and default policies
sudo ufw default deny incoming # deny everything inbound by default – the correct
starting posture
sudo ufw default allow outgoing # allow everything outbound by default (usually fine
for a normal host)
sudo ufw enable              # turn it on
```

Real rule-writing

TIP

Allow/deny a specific port and protocol:

```
sudo ufw allow 22/tcp
sudo ufw deny 23/tcp      # explicitly block, distinct from just not allowing
```

TIP**Allow only from a specific IP or subnet:**

```
sudo ufw allow from 203.0.113.5 to any port 22
sudo ufw allow from 192.168.1.0/24 to any port 3306    # a whole subnet, e.g. DB access
only from your LAN
```

TIP**Restrict to a specific network interface:**

```
sudo ufw allow in on eth0 to any port 80
```

TIP**Port ranges:**

```
sudo ufw allow 6000:6010/tcp
```

TIP**Rate limiting — throttles repeated connection attempts from the same IP (built-in brute-force mitigation):**

```
sudo ufw limit ssh
sudo ufw limit 2222/tcp
```

This denies an IP if it attempts more than 6 connections within 30 seconds — genuinely useful on any internet-facing SSH port, and something **allow** alone doesn't give you.

TIP**Deny a specific IP outright (e.g. after spotting it in fail2ban/logs):**

```
sudo ufw deny from 198.51.100.23
```

TIP**Numbered rules — needed to delete precisely:**

```
sudo ufw status numbered
sudo ufw delete 3          # deletes rule #3 as shown in the numbered list
```

Application profiles — some packages register a named profile instead of you remembering ports:

```
sudo ufw app list
sudo ufw allow 'Nginx Full'    # equivalent to allowing both 80/tcp and 443/tcp
```

TIP**Logging:**

```
sudo ufw logging on
sudo tail -f /var/log/ufw.log
```

`iptables` — The Legacy Standard, In Real Depth

Core concepts

Tables — different purposes:

Table	Purpose
filter	the default — basic allow/deny decisions (what most rules use)
nat	network address translation — port forwarding, masquerading
mangle	packet modification (rare, advanced use)

TIP

Chains within the `filter` table:

Chain	Applies to
INPUT	traffic destined FOR this machine
OUTPUT	traffic originating FROM this machine
FORWARD	traffic passing THROUGH this machine (routing/NAT scenarios)

Targets — what to do with a matched packet:

Target	Effect
ACCEPT	let it through
DROP	silently discard — sender gets no response at all
REJECT	discard, but send back an explicit rejection (e.g. connection refused)

Rule syntax anatomy

```
iptables -A INPUT -p tcp --dport 22 -j ACCEPT
#           ■      ■      ■      ■
#           ■      ■      ■      ■■ target: what to do
#           ■      ■      ■■■■■■■■■■■ match: destination port 22
#           ■      ■■■■■■■■■■■■■■■■■■■ match: protocol tcp
#           ■■■■■■■■■■■■■■■■■■■■■■■■■■■ append this rule to the INPUT chain
```

Real rule-writing

TIP

Set default policies (do this **FIRST**, before adding allow rules, or you can lock yourself out):

```
sudo iptables -P INPUT DROP
sudo iptables -P FORWARD DROP
sudo iptables -P OUTPUT ACCEPT
```

TIP

Always allow established/related connections — without this, replies to your own outbound traffic get blocked too:

```
sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
```

TIP**Allow loopback (localhost) traffic — many services depend on this internally:**

```
sudo iptables -A INPUT -i lo -j ACCEPT
```

TIP**Allow SSH, but only from one IP:**

```
sudo iptables -A INPUT -p tcp -s 203.0.113.5 --dport 22 -j ACCEPT
```

TIP**Allow a port range:**

```
sudo iptables -A INPUT -p tcp --dport 6000:6010 -j ACCEPT
```

TIP**Rate-limit connections (basic brute-force mitigation):**

```
sudo iptables -A INPUT -p tcp --dport 22 -m conntrack --ctstate NEW -m limit --limit 3/  
min --limit-burst 3 -j ACCEPT  
sudo iptables -A INPUT -p tcp --dport 22 -j DROP
```

This accepts up to 3 new SSH connection attempts per minute, then silently drops anything beyond that — a manual equivalent of what `ufw limit` does automatically.

TIP**Block a specific IP outright:**

```
sudo iptables -A INPUT -s 198.51.100.23 -j DROP
```

TIP**Port forwarding (NAT table) — forward external port 8080 to an internal service on port 3000:**

```
sudo iptables -t nat -A PREROUTING -p tcp --dport 8080 -j REDIRECT --to-port 3000
```

TIP**Logging before dropping — see what's being blocked:**

```
sudo iptables -A INPUT -p tcp --dport 22 -j LOG --log-prefix "SSH-DROP: "  
sudo iptables -A INPUT -p tcp --dport 22 -j DROP
```

```
dmesg | grep SSH-DROP    # view the logged attempts
```

Viewing & deleting rules

```
sudo iptables -L -v -n --line-numbers    # list all rules, numbered, with packet
counters
sudo iptables -D INPUT 3                  # delete rule #3 from the INPUT chain
sudo iptables -F                          # flush (delete) ALL rules – use with
caution, this can open everything up
```

Making rules persistent (they're wiped on reboot otherwise)

```
sudo apt install iptables-persistent -y
sudo netfilter-persistent save
```

`nftables` — The Modern Replacement

nftables unifies what used to require separate tools (`iptables`, `ip6tables`, `arptables`, `ebtables`) into one framework with cleaner syntax and better performance for large rule sets.

Core concepts — similar shape, different words

```
table    → a container for chains (you name it and pick a family: ip, ip6, inet, arp,
bridge)
chain     → a hook point (input, output, forward – you define these yourself, unlike
iptables' fixed chains)
rule      → the actual match + action
```

Basic setup

```
sudo apt install nftables -y
sudo systemctl enable nftables --now
```

Writing rules — interactive vs. file-based

TIP

Interactive (temporary, for testing):

```
sudo nft add table inet filter
sudo nft add chain inet filter input { type filter hook input priority 0 \; policy drop
\; }
sudo nft add rule inet filter input tcp dport 22 accept
sudo nft add rule inet filter input ct state established,related accept
sudo nft add rule inet filter input iif lo accept
```

TIP

File-based (the real, persistent way — this is how nftables is meant to be used):

```
sudo nano /etc/nftables.conf
```

```
#!/usr/sbin/nft -f

flush ruleset

table inet filter {
    chain input {
        type filter hook input priority 0; policy drop;

        ct state established,related accept
        iif lo accept

        tcp dport 22 accept
        tcp dport 80 accept
        tcp dport 443 accept

        # rate-limited SSH, same idea as the iptables example above
        tcp dport 2222 ct state new limit rate 3/minute accept

        # explicit block of a known-bad IP
        ip saddr 198.51.100.23 drop
    }

    chain forward {
        type filter hook forward priority 0; policy drop;
    }

    chain output {
        type filter hook output priority 0; policy accept;
    }
}
```

```
sudo nft -f /etc/nftables.conf      # load it
sudo systemctl restart nftables    # or apply via service restart
```

Sets — nftables' real advantage for larger rule sets

Instead of writing a separate rule per IP or port, group them into a named set:

```
set allowed_ips {
    type ipv4_addr
    elements = { 203.0.113.5, 203.0.113.6, 198.51.100.10 }
}

chain input {
    type filter hook input priority 0; policy drop;
    ip saddr @allowed_ips tcp dport 22 accept
}
```

This is dramatically more efficient (and readable) than 50 near-identical iptables lines when you're allow-listing many IPs or ports.

Viewing & managing rules

```
sudo nft list ruleset              # show everything currently loaded
sudo nft list table inet filter    # show just one table
sudo nft flush ruleset            # clear everything – same caution as iptables -F
sudo nft delete rule inet filter input handle <handle_number> # delete one specific
rule by its handle
```

Get handle numbers with:

```
sudo nft -a list ruleset          # -a shows the handle for each rule, needed for targeted
deletion
```

Same Scenario, All Three Tools — Direct Comparison

Goal: allow SSH only from 203.0.113.5, rate-limited, deny everything else inbound.

TIP

ufw:

```
sudo ufw default deny incoming
sudo ufw allow from 203.0.113.5 to any port 22
sudo ufw limit 22/tcp
sudo ufw enable
```

TIP

iptables:

```
sudo iptables -P INPUT DROP
sudo iptables -A INPUT -i lo -j ACCEPT
sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
sudo iptables -A INPUT -p tcp -s 203.0.113.5 --dport 22 -m limit --limit 3/min -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 22 -j DROP
```

TIP

nftables:

```
chain input {
    type filter hook input priority 0; policy drop;
    iif lo accept
    ct state established,related accept
    ip saddr 203.0.113.5 tcp dport 22 limit rate 3/minute accept
}
```

Same outcome, three different vocabularies — this is the fastest way to see how the concepts map across tools.

Which One Should You Actually Use?

Situation	Use
Single personal server, common rules (SSH, a web port, maybe a DB port)	ufw — fastest to get right, hardest to accidentally lock yourself out
You're following an existing guide/Stack Overflow answer, most of which are written for it	iptables — still the most commonly documented tool online
Setting up a new system today with no legacy constraints, or managing many IPs/ports via sets	nftables — the direction the ecosystem is actually moving
You need to understand what ufw generated under the hood	sudo iptables -L -v -n (or nft list ruleset on nftables-backed distros) shows you exactly what ufw actually wrote

TIP

Practical note for your setup specifically: since you're already running ufw from the SSH guide, there's no need to switch tools just to get the deeper rules above — everything in section 1 (rate limiting, per-IP rules, per-interface rules) covers real-world hardening without ever touching raw iptables/nftables syntax. Reach for the lower-level tools when you hit something ufw genuinely can't express (complex NAT, sets of hundreds of IPs, multi-chain custom logic).

Quick Reference

```
UFW          sudo ufw allow from <ip> to any port <port>
              sudo ufw limit <port>/tcp
              sudo ufw status numbered → sudo ufw delete <n>

IPTABLES     sudo iptables -A INPUT -p tcp -s <ip> --dport <port> -j ACCEPT
              sudo iptables -L -v -n --line-numbers
              sudo iptables -D INPUT <n>

NFTABLES     sudo nft add rule inet filter input tcp dport <port> accept
              sudo nft -a list ruleset
              sudo nft delete rule inet filter input handle <n>

ALWAYS DO FIRST allow loopback + established/related BEFORE setting a deny-by-default
policy,          or you can lock yourself out of your own remote session immediately.
```

General Server Hardening — Beyond SSH

A practical hardening checklist — users, sudo, SSH, updates and auditing for internet-facing servers.

Level: Advanced • Words: 1,436 • Sections: 7

Commands: apt, cat, nano, nano/vim, ss, systemctl

Introduction

SSH hardening gets one door. This covers the rest of the house: automated patching, intrusion detection, activity auditing, and cutting down what's even exposed to attack in the first place.

The Hardening Mental Model

1. Reduce attack surface → fewer running services/open ports = fewer things that can go wrong
2. Patch automatically → known vulnerabilities get fixed before they're exploited, without you remembering
3. Detect intrusion attempts → fail2ban reacts to abuse in real time
4. Audit what actually happened → auditd/logs give you a record if something DOES get through

These four layers are independent — losing one doesn't defeat the others, which is the whole point of "defense in depth."

Minimizing Attack Surface

The single highest-leverage hardening step, and the one most guides skip in favor of flashier tools: **run less stuff**.

Audit what's actually running and listening

```
sudo ss -tulnp          # every listening port, and the process behind it
sudo systemctl list-units --type=service --state=running    # every active service
```

For each result, ask: does this actually need to be running on this machine, and does it need to be reachable from outside (vs. **localhost**-only)?

Disable services you don't need

```
sudo systemctl stop <service>
sudo systemctl disable <service>    # prevents it from starting again on next boot
```

Common offenders on a fresh install that are rarely needed on a headless server: print spoolers (**cups**), Bluetooth (**bluetooth**), and various desktop-environment leftovers if you installed from a desktop ISO instead of server-minimal.

Bind services to localhost when they don't need external access

Many services (databases, internal APIs, admin panels) only need to be reached by other processes on the same machine, or through a reverse proxy — not directly from the internet. In the service's own config:

```
# example: PostgreSQL, MySQL, Redis, etc — bind to loopback only
listen_addresses = 'localhost'
bind 127.0.0.1
```

A service that's only listening on **127.0.0.1** simply isn't reachable from outside at all, regardless of firewall rules — this is a stronger guarantee than "I firewalled it," because there's no rule to misconfigure.

Remove unused packages, don't just disable them

```
sudo apt list --installed
sudo apt purge <package>          # removes it entirely, including config files
sudo apt autoremove                # cleans up now-orphaned dependencies
```

A disabled-but-installed service can still get re-enabled by an update, another package's dependency, or a future mistake. Removed is more permanent than disabled.

Close ports at the firewall too (defense in depth with the previous point)

Even after minimizing what's running, apply your `ufw`/firewall rules (from the firewall guide) as a second layer — belt and suspenders, not either/or.

`unattended-upgrades` — Automatic Security Patching

Most real-world compromises exploit **known, already-patched** vulnerabilities on systems that simply never got updated. This closes that gap without you needing to remember `apt upgrade` regularly.

Install and configure

```
sudo apt install unattended-upgrades apt-listchanges -y
sudo dpkg-reconfigure --priority=low unattended-upgrades # interactive setup,
choose "Yes"
```

Key config file: `/etc/apt/apt.conf.d/50unattended-upgrades`

```
Unattended-Upgrade::Allowed-Origins {
    "${distro_id}:${distro_codename}-security";
    // uncomment the next line too if you also want non-security updates applied
    automatically -
    // most hardened setups deliberately leave this OFF and patch non-security updates
    manually
    // "${distro_id}:${distro_codename}-updates";
};

Unattended-Upgrade::Remove-Unused-Dependencies "true";
Unattended-Upgrade::Automatic-Reboot "false";      // see note below before flipping
this to true
Unattended-Upgrade::Automatic-Reboot-Time "02:00";
```

Whether to enable automatic reboots

Some kernel/library updates need a reboot to fully take effect — until then, the running processes are still using the old, vulnerable code even though the files on disk are patched. `Automatic-Reboot "true"` closes that gap completely, but takes the machine down on a schedule with no human confirmation — genuinely fine for most personal/dev servers, riskier for something where unexpected downtime has real consequences. Know the tradeoff rather than defaulting either way blindly.

Confirm it's actually enabled and working

```
cat /etc/apt/apt.conf.d/20auto-upgrades
```

```
APT::Periodic::Update-Package-Lists "1";
APT::Periodic::Unattended-Upgrade "1";
```

Check what it's actually done

```
cat /var/log/unattended-upgrades/unattended-upgrades.log
```

Dry-run test (see what it WOULD do, right now, without applying anything)

```
sudo unattended-upgrade --dry-run --debug
```

Tuning `fail2ban` Properly

Basic `fail2ban` install/enable was covered in the SSH guide — this is the actual tuning most people skip.

Understanding the two files that matter

```
/etc/fail2ban/jail.conf    # the DEFAULT config – DO NOT edit this directly, it gets
                           # overwritten on package updates
/etc/fail2ban/jail.local   # YOUR overrides – create this, fail2ban merges it on top
                           # of jail.conf
```

A properly tuned `jail.local`

```
sudo nano /etc/fail2ban/jail.local
```

```
[DEFAULT]
bantime = 1h           # how long an IP stays banned
findtime = 10m         # the window within which maxretry failures must occur
maxretry = 5           # how many failures within findtime triggers a ban
ignoreip = 127.0.0.1/8 ::1 192.168.1.0/24 # NEVER let fail2ban ban your own trusted
                           # network by accident

[sshd]
enabled = true
port = 2222            # match whatever port you actually run SSH on (see your SSH
                           # guide)
bantime = 1d           # SSH specifically deserves a longer ban than the default
                           # – real attackers, not typos

[nginx-http-auth]
enabled = true

[nginx-limit-req]
enabled = true
```

TIP

Why `ignoreip` matters more than it looks: without it, a genuine typo in your own SSH client (wrong key, mistyped password if you still allow that) can get you banned from your own server. Always whitelist your home/office IP or trusted subnet explicitly.

TIP

Why per-jail `bantime` matters: an SSH brute-force attempt is a much stronger malicious signal than, say, a single failed login on a lower-stakes service — tuning `bantime` per jail lets you ban SSH attackers harder and longer while staying more lenient elsewhere.

Escalating bans for repeat offenders

```
[DEFAULT]
bantime.increment = true
bantime.factor = 2      # each subsequent ban for the same IP doubles in length
bantime.maxtime = 1w    # cap the escalation so it doesn't grow unbounded forever
```

Checking fail2ban's actual state

```
sudo fail2ban-client status          # list active jails
sudo fail2ban-client status sshd    # see currently banned IPs for a specific
jail
sudo fail2ban-client set sshd unbanip <ip>  # manually unban an IP (useful if you
accidentally ban yourself)
```

Testing a jail's regex against your actual logs before trusting it

```
sudo fail2ban-regex /var/log/auth.log /etc/fail2ban/filter.d/sshd.conf
```

Confirms the jail's pattern actually matches your real log format — worth doing once for any custom jail you write, since a filter that doesn't match anything provides zero protection while looking configured.

`auditd` — Knowing What Actually Happened

Where `journald/syslog` tell you what services logged, `auditd` tracks lower-level system events: specific file access, command execution, permission changes — a much finer-grained record, at the cost of more noise to manage.

Install and enable

```
sudo apt install auditd audispd-plugins -y
sudo systemctl enable auditd --now
```

Writing audit rules — `/etc/audit/rules.d/audit.rules`

TIP

Watch a critical file for any changes:

```
-w /etc/passwd -p wa -k passwd_changes
-w /etc/shadow -p wa -k shadow_changes
-w /etc/sudoers -p wa -k sudoers_changes
```

`-p wa` = watch for writes and attribute changes. `-k <name>` = a searchable tag for this rule's matches.

TIP

Watch for use of a specific command (e.g. track every use of `sudo`):

```
-w /usr/bin/sudo -p x -k sudo_usage
```

TIP

Watch a whole directory:

```
-w /etc/ssh/ -p wa -k sshd_config_changes
```

Apply new rules:

```
sudo augenrules --load
# or reboot, since some environments load rules only at boot
```

Actually querying the audit log — this is the part that makes it useful

```
sudo ausearch -k passwd_changes      # everything matching that tag
sudo ausearch -k sudo_usage -ts today # sudo usage, today only
sudo ausearch -ui 1000               # everything by a specific UID
sudo aureport -au                     # summary report of authentication events
sudo aureport -f                     # summary report of file access events
```

TIP

Practical example — "did anyone touch `/etc/sudoers` this week, and who":

```
sudo ausearch -k sudoers_changes -ts week-ago
```

This is the kind of question `auditd` answers cleanly that plain syslog genuinely can't — it ties the event to the exact process, UID, and command line responsible, not just a general log line.

TIP

Honest tradeoff to know: `auditd` produces a lot of log volume if you watch broad paths — start with a small, deliberate rule set (critical config files, sudo usage) rather than trying to audit everything, or the noise buries the signal you actually care about.

Putting It Together — A Baseline Hardening Checklist

- [] Audited running services (`ss -tulnp`), disabled/removed anything unnecessary
- [] Internal-only services (DB, cache, admin panels) bound to `127.0.0.1`, not `0.0.0.0`
- [] Firewall (`ufw`/`iptables`/`nftables`) default-deny inbound, explicit allows only
- [] `unattended-upgrades` installed and confirmed active, security updates at minimum
- [] `fail2ban` tuned with a real `jail.local` — proper `bantime`/`findtime`/`maxretry`, `ignoreip` set to your own trusted network
- [] `auditd` watching at minimum: `/etc/passwd`, `/etc/shadow`, `/etc/sudoers`, and sudo usage itself
- [] SSH hardening from the earlier guide applied (key-only auth, non-default port, no root login)
- [] A recurring calendar reminder (not just automation) to actually review `aureport`/`fail2ban-client status`/`journalctl -p err` periodically — automation reduces the work, it doesn't replace occasionally actually looking

Linux User & Group Management — Complete Guide

Users, groups, permissions, sudo and ACLs — create accounts safely, avoid the usermod -G trap, and control access properly.

Level: Beginner • Words: 2,057 • Sections: 8
Commands: chage, chgrp, chmod, chown, getfacl, groupadd +9 more

Introduction

Everything about creating, modifying, and managing users, groups, and the permissions tied to them. Structured to pair with the RHCSA-aligned course roadmap (Module 3: User & Group Administration, Module 4: File Permissions & Access Control).

PART 1: Core Concepts

How Linux Tracks Users

Every user has a **UID** (User ID) and belongs to at least one **GID** (Group ID). Linux stores this in plain text system files:

File	Purpose	Notes / Gotchas
<code>/etc/passwd</code>	User account info (username, UID, GID, home dir, shell)	Password field here is just <code>x</code> — real hash lives in <code>/etc/shadow</code> . World-readable by design.
<code>/etc/shadow</code>	Encrypted passwords + password aging info	Only root can read this. Never world-readable — a common security misconfig is loosening its permissions.
<code>/etc/group</code>	Group names, GIDs, and member lists	Supplementary group memberships are listed here.
<code>/etc/gshadow</code>	Group passwords (rarely used)	Mostly legacy; group admin/password features are uncommon in modern setups.
<code>/etc/login.defs</code>	Default policies (UID ranges, password aging defaults)	Where <code>useradd</code> gets its defaults from if you don't specify flags.

TIP

Reading a `/etc/passwd` line:

```
maradona:x:1001:1001:Maradona Dev:/home/maradona:/bin/bash
```

`username : password-placeholder : UID : GID : comment/full-name : home-dir : login-shell`

User Types

Type	UID Range (typical)	Example
Root	0	<code>root</code>
System users	1–999 (RHEL) / 1–999 or 1–99 (Debian varies)	<code>nginx</code> , <code>sshd</code> — used by services, not humans
Regular users	1000+	Your actual login accounts

Primary vs. Supplementary Groups

- Primary group** — every user has exactly one; owns files the user creates by default. Stored in `/etc/passwd` (GID field).
- Supplementary (secondary) groups** — a user can belong to many; used to grant extra access (e.g. `sudo`, `docker`, `wheel`). Stored in `/etc/group`.

PART 2: User Commands

Viewing User Info

Command	Description	Example	! Notes / Gotchas
<code>whoami</code>	Print current username	<code>whoami</code>	Simplest identity check.
<code>id</code>	Show UID, GID, and all group memberships	<code>id maradona</code>	Best single command to confirm exactly what groups a user is really in.
<code>groups</code>	List groups a user belongs to	<code>groups maradona</code>	Shorter output than <code>id</code> , names only.
<code>finger</code>	Detailed user info (if installed)	<code>finger maradona</code>	Not installed by default on most modern distros.
<code>getent passwd</code>	Query user database (works with LDAP/AD too)	<code>getent passwd maradona</code>	Preferred over <code>grep-ing /etc/passwd</code> directly — works with any configured user backend, not just local files.
<code>last</code>	Show login history	<code>last maradona</code>	Reads <code>/var/log/wtmp</code> ; useful for auditing.

Creating Users

```

useradd username                # create user with defaults from /etc/login.defs
useradd -m username            # create with a home directory (needed on some
distros — RHEL does this by default, Debian does not)
useradd -m -s /bin/bash username # set login shell explicitly
useradd -m -c "Full Name" username # add a comment/full name
useradd -m -u 1050 username      # set a specific UID
useradd -m -g developers username # set primary group
useradd -m -G sudo,docker username # set supplementary groups (comma-separated, no spaces)
useradd -e 2026-12-31 username  # set account expiration date

```

Flag	Meaning	! Notes
<code>-m</code>	Create home directory	Debian/Ubuntu: required if you want a home dir. RHEL/CentOS: created by default even without <code>-m</code> . Always safest to include it explicitly.
<code>-s</code>	Login shell	Omitting this often defaults to <code>/bin/sh</code> or no shell — set it explicitly for interactive accounts.
<code>-g</code>	Primary group (singular)	Must already exist, or <code>useradd</code> fails.
<code>-G</code>	Supplementary groups (plural)	Comma-separated. This is the flag people reach for to add someone to <code>sudo/wheel</code> .
<code>-u</code>	Specific UID	Useful for keeping UIDs consistent across servers (e.g. for NFS).
<code>-e</code>	Expiration date (YYYY-MM-DD)	Account is disabled automatically after this date — good for contractor/temp accounts.

After creating, always set a password:

```
passwd username    # prompts to set the password interactively
```

Modifying Users — `usermod` (Deep Dive)

`usermod` changes an **existing** account's settings. This is the command you'll use most often after initial account creation.


```

usermod -aG groupname username      # ADD to a supplementary group (see warning below)
usermod -g groupname username      # CHANGE primary group
usermod -s /bin/zsh username        # change login shell
usermod -c "New Full Name" username # change comment/full name
usermod -d /new/home -m username    # change home dir and MOVE existing contents
usermod -l newusername oldusername  # rename the login (username itself)
usermod -L username                 # lock the account (disable password
login)
usermod -U username                  # unlock the account
usermod -e 2027-01-01 username      # change expiration date
usermod -e "" username               # remove expiration date

```

WARN

The single most common usermod mistake: `bash usermod -G sudo username #` ■ **WRONG** — this REPLACES all supplementary groups with just "sudo" `usermod -aG sudo username #` ■ **CORRECT** — the `-a` (append) flag ADDS to existing groups. Forgetting `-a` silently **removes** the user from every other supplementary group they were in (Docker, developers, etc.) — a classic "why did their access break" incident. Always pair `-G` with `-a` unless you deliberately want to overwrite the group list.

Flag	Meaning	! Notes
<code>-aG</code>	Append to supplementary groups	Always use <code>-a</code> together with <code>-G</code> .
<code>-g</code>	Set primary group	Group must exist first.
<code>-L / -U</code>	Lock / unlock account	Locks by putting ! in front of the password hash in <code>/etc/shadow</code> — doesn't delete the password, just disables it.
<code>-l</code>	Rename username	Does not rename the home directory — use <code>-d</code> with <code>-m</code> separately if you want that too.
<code>-e</code>	Expiration date	Empty string clears it.

Deleting Users

```

userdel username      # delete user, LEAVE home directory intact
userdel -r username   # delete user AND remove home directory + mail spool

```

WARN

`-r` is permanent — the home directory and its contents are gone, same as `rm -rf`. Back up first if there's any doubt.

Password & Account Aging

```

passwd username      # set/change password
passwd -l username   # lock password (same effect as usermod -L)
passwd -u username   # unlock password
passwd -e username   # force password change at next login
chage -l username     # list current aging settings
chage -M 90 username  # max password age: 90 days
chage -m 7 username   # min days between password changes
chage -W 14 username  # warn user 14 days before expiration
chage -E 2026-12-31 username # set account expiration via chage instead of
usermod

```

PART 3: Group Commands**Creating & Managing Groups**

```
groupadd groupname          # create a new group
groupadd -g 2050 groupname  # create with a specific GID
groupmod -n newname oldname # rename a group
groupmod -g 2100 groupname  # change a group's GID
groupdel groupname          # delete a group
```

WARN

You can't delete a group that's still set as any user's **primary** group — reassign those users first, or you'll get an error.

Managing Group Membership

```
gpasswd -a username groupname # add a user to a group (alternative to usermod -aG)
gpasswd -d username groupname # remove a user from a group
gpasswd -M user1,user2 groupname # set the entire member list at once (overwrites existing members)
```

Task	Best Command	Why
Add one user to a group	<code>usermod -aG group user</code> or <code>gpasswd -a user group</code>	Either works; <code>usermod -aG</code> is more commonly taught/used.
Remove one user from a group	<code>gpasswd -d user group</code>	<code>usermod</code> has no clean single-user "remove from group" flag — <code>gpasswd -d</code> is the direct tool.
See who's in a group	<code>getent group groupname</code>	Shows the member list straight from the group database.

PART 4: File Permissions Tied to Users & Groups

This is where user/group setup actually pays off — permissions are enforced based on **owning user**, **owning group**, and **everyone else**.

The Permission Triad

```
-rwxr-xr-- 1 maradona developers 1234 Aug 22 file.sh
■■■■■■■■■■ ■■■■■■■■ ■■■■■■■■
■■■■■■■■■■ ■■■■■■■■ ■■■■■■■■ group owner
■■■■■■■■■■ ■■■■■■■■ ■■■■■■■■ user owner
■■■■■■■■■■ ■■■■■■■■ others (rwx → r--)
■■■■■■■■■■ ■■■■■■■■ group (rwx → r-x)
■■■■■■■■■■ ■■■■■■■■ owner (rwx → rwx)
■■■■■■■■■■ ■■■■■■■■ file type (- = file, d = directory, l = symlink)
```

```
chmod u+x file.sh # owner: add execute
chmod g-w file.sh # group: remove write
chmod o=r file.sh # others: set to read-only exactly
chmod 750 file.sh # numeric: owner=rwx(7), group=rx(5), other=none(0)
chown maradona file.sh # change owning user
chown maradona:developers file.sh # change owning user AND group in one command
chgrp developers file.sh # change owning group only
chown -R maradona:developers dir/ # recursive — apply to a whole directory tree
```

Special Permissions (Beyond rwx)

Permission	Symbol	Numeric	What it does	Typical use
SUID	<code>s</code> in owner's execute slot	<code>4xxx</code>	Program runs with the file owner's privileges, not the caller's	<code>passwd</code> runs as root so any user can update their own shadow entry

Permission	Symbol	Numeric	What it does	Typical use
SGID	s in group's execute slot	2xxx	On a file: runs with the group's privileges. On a directory : new files inherit the directory's group automatically	Shared team directories — everyone's files land in the same group
Sticky bit	t in others' execute slot	1xxx	In a shared directory, users can only delete/rename their own files, even with write access to the dir	/tmp uses this by default

```
chmod u+s file           # set SUID
chmod g+s directory/     # set SGID on a directory
chmod +t directory/      # set sticky bit
chmod 2775 directory/    # SGID + rwxrwxr-x, numeric form
```

WARN

SUID on a script (not a compiled binary) is ignored by the Linux kernel for security reasons — if you need "run as another user," use `sudo` rules instead of trying to SUID a shell script.

Access Control Lists (ACLs) — Beyond Owner/Group/Other

When the basic owner/group/other model isn't granular enough (e.g. "give this one specific extra user access without changing group ownership"):

```
getfacl file.txt          # view current ACLs
setfacl -m u:someuser:rwx file.txt  # grant a specific user rwx, without changing
normal ownership
setfacl -m g:someteam:rx file.txt    # grant a specific group access
setfacl -x u:someuser file.txt       # remove that user's specific ACL entry
setfacl -b file.txt                 # remove ALL ACL entries, back to basic
permissions
setfacl -R -m u:someuser:rwx directory/ # apply recursively
setfacl -d -m u:someuser:rwx directory/ # DEFAULT ACL – new files created
inside inherit this automatically
```

WARN

A file with ACLs set shows a `+` after its permission string in `ls -l` (e.g. `-rwxr-x---+`) — that `+` is your visual cue to run `getfacl` and check what's really going on, since `ls -l` alone won't show the extra entries.

PART 5: `sudo` — Privileged Access Without Full Root

```
visudo          # ALWAYS use this to edit sudoers – validates syntax
before saving, prevents lockout
```

Typical sudoers entries:

```
username    ALL=(ALL:ALL) ALL          # full sudo access for one user
%wheel      ALL=(ALL)    ALL          # anyone in the "wheel" group gets full sudo (
common on RHEL)
username    ALL=(ALL) NOPASSWD: /usr/bin/systemctl restart nginx  # passwordless, but
ONLY for this one command
```

WARN

Never edit `/etc/sudoers` directly with a text editor. A syntax error can lock out `sudo` entirely, and without another root session open, recovery means booting into single-user/rescue mode. `visudo` checks syntax before writing the file — there's no good reason to skip it.

Command	Description	! Notes
<code>sudo command</code>	Run one command as root	Prompts for your own password, not root's.
<code>sudo -l</code>	List what commands you're allowed to run	Good first check when troubleshooting permission denials.
<code>sudo -u otheruser command</code>	Run as a specific user (not just root)	Useful for service accounts.
<code>su - username</code>	Fully switch to another user's shell/environment	The <code>-</code> matters — it loads that user's full environment (PATH, home dir), a plain <code>su username</code> doesn't.
<code>su -</code>	Switch to root, full environment	Requires the root password, unlike <code>sudo</code> .

PART 6: Practical Walkthrough — Common Scenarios

Scenario 1: Onboard a new developer with sudo + Docker access

```
useradd -m -s /bin/bash -c "New Developer" jdoe
passwd jdoe
usermod -aG sudo,docker jdoe      # -a is critical here — see Part 2 warning
id jdoe                          # verify group membership immediately
```

Scenario 2: Shared team directory where everyone's files land in the same group automatically

```
groupadd devteam
mkdir /srv/shared
chown :devteam /srv/shared
chmod 2775 /srv/shared           # SGID so new files inherit "devteam" group
usermod -aG devteam alice
usermod -aG devteam bob
```

Scenario 3: Give one extra user read access to a file without changing its group

```
setfacl -m u:contractor:r-- report.pdf
getfacl report.pdf              # confirm
```

Scenario 4: Temporarily disable a departing employee's account without deleting anything

```
usermod -L jdoe                 # lock password login
usermod -e 2026-08-22 jdoe      # also set expiration to today, belt-and-suspenders
# later, once fully offboarded:
userdel -r jdoe
```

Quick Reference Card

```
# Users
useradd -m -s /bin/bash -G group1,group2 username
usermod -aG groupname username      # ADD to group (never forget -a)
usermod -L / -U username            # lock / unlock
userdel -r username                  # delete + home dir

# Groups
groupadd groupname
gpasswd -a user group               # add member
gpasswd -d user group               # remove member
getent group groupname              # list members

# Permissions
chmod 750 file | chmod u+x,g-w file
chown user:group file
setfacl -m u:user:rxw file
getfacl file

# Privilege
visudo
sudo -l
su - username
```

Linux Directory Permissions — Complete Guide

What `rwX` really means on directories, `SGID` + sticky bits, `umask`, and `namei -l` for tracing permission denied up the path.

Level: Intermediate • Words: 1,237 • Sections: 7

Commands: `cd`, `chgrp`, `chmod`, `find`, `ls`, `mkdir` +2 more

Introduction

Directories are files too, but `rwX` means something different on them than on regular files — this is the #1 source of "why can't I `cd` into this even though I have read access" confusion. This guide is directory-specific; pair it with [linux-user-group-management.md](#) for the general permission model.

PART 1: What `rwX` Actually Means on a Directory

This is the single most important table in this guide — the meaning shifts entirely from what it means on a file.

Permission	On a file	On a directory
<code>r</code> (read)	View the file's contents	List the directory's contents (<code>ls</code> works)
<code>w</code> (write)	Modify the file's contents	Create, delete, or rename entries inside the directory (even files you don't own!)
<code>x</code> (execute)	Run the file as a program/script	Enter/traverse the directory (<code>cd</code> works, and you can access files inside by exact path)

The classic gotcha

```
chmod 644 mydir/    # r + w, no x
cd mydir/           # ■ "Permission denied" – even though you can technically "read" it
ls mydir/           # ■ Also fails – ls needs to traverse to stat each entry
```

WARN

Read without execute on a directory is nearly useless. You need **execute** to enter/traverse a directory at all — without it, `cd`, `ls`, and any attempt to access a file inside by path (`cat mydir/file.txt`) fails, regardless of read permission. Read alone only lets tools list filenames in very limited cases and is not something to rely on.

The other classic gotcha — write on a directory is powerful

```
chmod 777 shared/   # anyone can write here
```

Anyone with **write** on the directory can delete or rename files inside it — **even files they don't own** — unless the sticky bit is set (see Part 3). This surprises people who think file permissions alone protect their files; the directory's write permission matters just as much.

PART 2: Directory Permission Recipes

Numeric	Symbolic	Meaning	Typical use
<code>755</code>	<code>rwXr-Xr-X</code>	Owner full control, everyone else can enter/list but not modify	Most personal/project directories
<code>750</code>	<code>rwXr-X---</code>	Owner full control, group can enter/list, others nothing	Directories restricted to a team
<code>700</code>	<code>rwX-----</code>	Owner-only, no one else can even enter	Private directories (e.g. <code>~/ .ssh</code>)

Numeric	Symbolic	Meaning	Typical use
775	<code>rwrxwrx-r-x</code>	Owner + group full read/write/enter, others read-only entry	Collaborative directories (often paired with SGID, see Part 3)
770	<code>rwrxwrx---</code>	Owner + group full access, others completely locked out	Team-only working directories
1777	<code>rwrxwrxrwt</code>	Everyone can create files, but only delete their own (sticky bit)	<code>/tmp</code> -style shared drop directories
2775	<code>rwrxrwsr-x</code>	Group full access + new files auto-inherit the directory's group (SGID)	Shared project directories with consistent group ownership

```

chmod 755 mydir/
chmod -R 750 project/          # recursive – apply to every file AND subdirectory inside
find project/ -type d -exec chmod 750 {} \;  # apply ONLY to directories, leave file
permissions alone
find project/ -type f -exec chmod 640 {} \;  # apply ONLY to files, separately

```

WARN

`chmod -R` applies the exact same numeric mode to files and directories alike — which usually isn't what you want, since files rarely need the execute bit but directories almost always do (to be entered). The two-step `find -type d / find -type f` pattern above is the safer approach for mixed trees.

PART 3: Special Bits on Directories

These behave differently on directories than the general special-permissions summary you'd see for files.

SGID on a Directory — Group Inheritance

Normally, a new file's group ownership comes from the **creating user's** primary group. SGID on a directory overrides that: new files/subdirectories inside automatically inherit **the directory's group**, not the creator's.

```

mkdir /srv/teamshare
chgrp devteam /srv/teamshare
chmod g+s /srv/teamshare          # or: chmod 2775 /srv/teamshare

```

Now, no matter who creates a file inside `/srv/teamshare`, it's automatically owned by group `devteam` — solving the classic "shared folder where everyone's uploads end up with the wrong group" problem.

WARN

SGID is **inherited recursively by subdirectories that get created afterward** — a new subfolder made inside an SGID directory is itself SGID automatically, continuing the inheritance chain. But this only applies going forward; it doesn't retroactively fix subdirectories that already existed before you set SGID on the parent.

Sticky Bit on a Directory — Restricted Deletion

Normally, anyone with **write** access to a directory can delete or rename any file inside it (Part 1). The sticky bit changes this: users can only delete/rename files **they personally own**, even in a directory that's otherwise world-writable.

```

chmod +t /srv/dropbox          # or: chmod 1777 /srv/dropbox
ls -ld /srv/dropbox
# drwxrwxrwt ... /srv/dropbox

```

This is exactly how `/tmp` works system-wide: everyone can create files there, but you can't delete someone else's temp file just because the directory is world-writable.

Combining Both

```
chmod 3775 /srv/teamshare    # SGID (2000) + sticky bit (1000) = 3000, plus rwxrwxr-x
```

Group-consistent ownership **and** "you can only delete your own stuff" in the same shared directory.

PART 4: `umask` — Default Permissions for New Directories & Files

`umask` doesn't set permissions directly — it **subtracts** from the maximum defaults whenever something new is created.

Item type	Maximum default before umask	Typical umask	Resulting permission
New directory	777 (rwxrwxrwx)	022	755 (rwxr-xr-x)
New file	666 (rw-rw-rw-, no execute by default)	022	644 (rw-r--r--)

```
umask          # view current umask
umask 027      # set: group loses write, others get nothing
umask -S       # view umask in symbolic form instead of octal
```

WARN

Note the asymmetry: directories start from a max of 777 but files start from 666 — new files never get execute permission automatically from `umask` alone, even with `umask 000`. That's deliberate; you have to `chmod +x` a new file explicitly (or it needs to already be a compiled/scripted executable you intend to run).

`umask` is usually set per-session in `~/.bashrc` / `/etc/profile` (per-user) or `/etc/login.defs` (system default for new accounts).

PART 5: Auditing & Troubleshooting Directory Permissions

```
ls -ld directory/          # view a directory's OWN permissions (note the -d
- without it, ls lists its CONTENTS instead)
namei -l /path/to/deep/directory # show permissions of EVERY component in a path
- great for tracing a "permission denied" up the chain
find / -type d -perm -002 2>/dev/null # find world-writable directories (a common
security audit check)
find /home -type d -perm 777 2>/dev/null # find suspiciously wide-open directories
stat directory/             # detailed metadata including octal
permission mode
```

WARN

A "permission denied" error **doesn't always mean the target directory itself is misconfigured** — it can be any parent directory in the path missing execute permission. `namei -l` is the fastest way to check every level at once instead of manually `ls -ld`-ing each parent directory one by one.

Common real-world directory permissions (for reference)

Directory	Typical permissions	Why
/tmp	1777	World-writable, sticky bit prevents cross-user deletion
/root	700	Only root can even enter
/home/username	700 or 750	Private by default on most modern distros
~/.ssh	700 (dir), keys inside 600	SSH refuses to work at all if these are too open
/var/log	755 (dir), individual logs often 640	World-readable directory listing, but log contents often restricted

Directory	Typical permissions	Why
<code>/etc</code>	<code>755</code>	World-readable/traversable (most configs aren't secret), but only root can write

Quick Reference Card

```
# What rwx means on a directory
r = list contents  w = create/delete/rename entries  x = enter/traverse

# Recipes
755  rwxr-xr-x  normal shared-read directory
750  rwxr-x---  team-restricted
700  rwx-----  private
2775 rwxrwsr-x  SGID – group inheritance
1777 rwxrwxrwt  sticky – /tmp style
3775 rwxrwsr-t  SGID + sticky combined

# Special bits
chmod g+s dir/  # SGID: new files inherit dir's group
chmod +t dir/   # sticky: users can only delete their own files

# umask
umask          # view
umask 022      # set (dirs→755, files→644)

# Troubleshooting
ls -ld dir/     # the directory's OWN permissions (not its contents)
namei -l /a/b/c # check every parent in a path at once
find / -perm -002 -type d 2>/dev/null # audit world-writable dirs
```



Part IV — Servers & Automation

Containers, automation & monitoring at scale. — 6 guides

Docker & Containers — Full Guide

Images, containers, volumes and networking — run services in Docker without cargo-culting Dockerfiles.

Level: Intermediate • Words: 2,146 • Sections: 12

Commands: curl, docker, docker build, docker compose, docker ps, docker run +1 more

Introduction

Images, volumes, networks, and Compose stacks — the four pillars that cover almost everything you'll actually do with Docker day to day.

What a Container Actually Is (Briefly)

A container isn't a lightweight VM — it's a regular process on your host, isolated using two Linux kernel features:

- **Namespaces** — give the process its own isolated view of the filesystem, network, process list, hostname, etc.
- **cgroups** — limit and account for the resources (CPU, memory) that process can use.

TIP

Image vs container — the distinction that trips people up: an **image** is a read-only template (built once, stored, reused). A **container** is a running (or stopped) instance created from an image, with its own writable layer on top. You can run many containers from the same image, exactly like many instances of a class from one class definition.

Installing Docker

```
curl -fsSL https://get.docker.com | sudo sh      # official convenience script, quickest
path
sudo usermod -aG docker $USER                    # let your user run docker without
sudo every time
newgrp docker                                    # apply the new group in your
current shell without re-logging in
docker --version
docker run hello-world                          # confirm it actually works end
to end
```

Images — Pulling, Building & Understanding Layers

Pulling existing images

```
docker pull nginx                                # latest tag, from Docker Hub by default
docker pull nginx:1.25-alpine                    # a specific tag – always prefer pinning a real
version over "latest" in anything you'll rely on
docker images                                    # list images stored locally
docker rmi nginx                                 # remove an image (fails if a container is
still using it)
```

TIP

Why pinning matters: `latest` is just a tag, not a guarantee — it moves as the maintainer pushes new builds. A `docker-compose.yml` that says `image: nginx:latest` can pull a genuinely different image next month than it did today, silently. Pin real versions (`nginx:1.25-alpine`) for anything beyond quick local testing.

Building your own image — Dockerfile anatomy

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

ENV PYTHONUNBUFFERED=1

CMD ["python3", "app.py"]
```

Instruction	Meaning
FROM	the base image everything else builds on top of
WORKDIR	sets the working directory inside the image for subsequent instructions
COPY	copies files from your build context into the image
RUN	executes a command at build time , its result gets baked into the image layer
EXPOSE	documents which port the container listens on (doesn't actually publish it — that's done at docker run , see networks section)
ENV	sets an environment variable inside the running container
CMD	the default command run when the container starts (not build time) — can be overridden at docker run
ENTRYPOINT	similar to CMD but harder to override — commonly paired with CMD supplying default arguments to it

Layers — why instruction order matters for build speed

Each instruction creates a new **layer**, and Docker caches layers — if a layer's inputs haven't changed, it's reused instead of rebuilt. This is why `COPY requirements.txt .` and `RUN pip install` happen **before** `COPY . .` in the example above: your dependency list changes far less often than your actual code, so ordering it first means most rebuilds skip the (usually slow) dependency install step entirely and only re-run the fast final copy.

```
docker build -t myapp:1.0 .           # build from the Dockerfile in the current
directory
docker build -t myapp:1.0 -f Dockerfile.prod . # use a specifically-named Dockerfile
docker history myapp:1.0              # see every layer and its size – useful for
hunting bloat
```

`.dockerignore` — keep the build context small

```
.git
node_modules
*.log
.env
```

Same role as `.gitignore` but for what gets sent to the Docker build process — a large, unfiltered context slows every build down.

Containers — Running & Managing

```

docker run nginx                # run in foreground, attached
docker run -d nginx             # detached – runs in background, returns
your shell immediately
docker run -d --name myweb nginx # give it a memorable name instead of a
random one
docker run -d -p 8080:80 nginx   # map host port 8080 to container port
80
docker run -d -e APP_ENV=production myapp # set an environment variable
docker run -d --restart unless-stopped myapp # auto-restart policy – see below

docker ps                       # list RUNNING containers
docker ps -a                   # list ALL containers,
including stopped ones
docker stop myweb
docker start myweb
docker restart myweb
docker rm myweb                # remove a stopped container
docker rm -f myweb             # force-remove, even if
running

```

Restart policies

Policy	Behavior
no (default)	never auto-restarts
on-failure	restarts only on a non-zero exit code
always	always restarts, including after a manual stop and a host reboot
unless-stopped	like always , but respects a manual docker stop — won't restart until you explicitly start it again

unless-stopped is the right default for most real services — it survives crashes and host reboots, but doesn't fight you when you deliberately stop it for maintenance.

Interacting with a running container

```

docker logs myweb                # see its stdout/stderr
docker logs -f myweb             # live-tail, like tail -f
docker exec -it myweb bash       # open an interactive shell INSIDE the running
container
docker exec -it myweb sh         # use sh instead if the image is minimal (
alpine-based, no bash)
docker inspect myweb             # full JSON detail – IPs, mounts, env vars,
everything
docker stats                     # live resource usage (CPU/mem) across all
running containers

```

Volumes — Persisting Data Beyond a Container's Lifetime

Without a volume, anything written inside a container disappears the moment that container is removed — the writable layer is tied to the container, not the image. Volumes solve this.

Three types

Type	Where it lives	Use for
Named volume	Managed by Docker, under <code>/var/lib/docker/volumes/</code>	The default choice for most persistent data (databases, uploaded files)
Bind mount	An exact path YOU choose on the host	When you need direct host access to the files (e.g. live-editing code during development)

Type	Where it lives	Use for
tmpfs mount	RAM only, never touches disk	Sensitive temp data, or a performance-critical scratch space

```
# Named volume — Docker manages the actual location
docker run -d -v mydata:/var/lib/mysql mysql
docker volume ls
docker volume inspect mydata

# Bind mount — you specify the exact host path
docker run -d -v /home/solo/project:/app myapp

# tmpfs — RAM-backed, gone on container stop
docker run -d --tmpfs /app/cache myapp
```

TIP

Practical rule of thumb: named volumes for anything you want Docker to manage and back up as a unit (databases especially); bind mounts specifically when you need to actively edit files from the host side while the container runs (classic dev workflow — edit code locally, container picks it up live).

```
docker volume rm mydata          # delete a named volume (fails if in use)
docker volume prune              # remove ALL unused volumes — be sure nothing
important is only referenced there
```

Networks — How Containers Talk to Each Other and the World

Default network types

```
docker network ls
```

Driver	Behavior
bridge (default)	isolated internal network, containers can reach each other by IP, and the host via port mapping
host	container shares the host's network stack directly — no isolation, no port mapping needed, but no isolation either
none	no networking at all

Publishing a port — making a container reachable from outside

```
docker run -d -p 8080:80 nginx
#      ■      ■
#      ■      ■ container's internal port
#      ■■■■■■ host's port, what you actually connect to
```

```
docker run -d -p 127.0.0.1:8080:80 nginx    # bind ONLY to localhost — not reachable
from outside the host at all
```

This mirrors the "bind to localhost" principle from the hardening guide — if a service genuinely doesn't need external access, don't expose it beyond the loopback interface.

Custom networks — the right way for multi-container communication

```
docker network create mynet
docker run -d --name db --network mynet postgres
docker run -d --name app --network mynet myapp
```

Containers on the same custom network can reach each other **by container name** as a hostname — `app` can connect to `db:5432` directly, no IP addresses or port mapping needed between them. This is the real reason to create custom networks instead of relying on the default bridge, where name-based resolution isn't guaranteed the same way.

Docker Compose — Multi-Container Stacks Declaratively

Compose lets you define an entire multi-service stack (app + database + cache, etc.) in one YAML file instead of a string of individual `docker run` commands.

Anatomy of a `docker-compose.yml`

```
version: "3.9"

services:
  app:
    build: .
    ports:
      - "8080:8000"
    environment:
      - DATABASE_URL=postgres://user:pass@db:5432/mydb
    depends_on:
      - db
    restart: unless-stopped
    volumes:
      - ./app:/app          # bind mount for live code editing during dev

  db:
    image: postgres:16
    environment:
      - POSTGRES_USER=user
      - POSTGRES_PASSWORD=pass
      - POSTGRES_DB=mydb
    volumes:
      - db_data:/var/lib/postgresql/data
    restart: unless-stopped

  redis:
    image: redis:7-alpine
    restart: unless-stopped

volumes:
  db_data:
```

TIP

Key things this file does automatically that raw `docker run` doesn't:

- Creates a dedicated network for the whole stack, so `app` can reach `db` and `redis` by service name — no manual `docker network create` needed.
- `depends_on` controls **start order** (though notably not "wait until actually ready" — see the health check note below).
- One command spins up or tears down the entire stack together, consistently.

Core commands

```

docker compose up -d          # start the whole stack, detached
docker compose down          # stop and remove containers (volumes persist
unless you add -v)
docker compose down -v       # stop AND remove volumes too – destructive,
deletes your data
docker compose ps            # status of every service in the stack
docker compose logs -f app    # live logs for one specific service
docker compose logs -f       # live logs for the WHOLE stack, interleaved
docker compose build          # rebuild images defined with a `build:`
key
docker compose restart app    # restart just one service
docker compose exec app bash  # shell into a running service

```

The `depends\_on` gotcha

`depends_on` only waits for the dependency container to **start**, not for the application inside it to actually be ready to accept connections — a database container can report "started" well before Postgres has finished initializing. For anything that genuinely needs to wait for real readiness, add a health check:

```

services:
  db:
    image: postgres:16
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U user"]
      interval: 5s
      timeout: 5s
      retries: 5

  app:
    depends_on:
      db:
        condition: service_healthy  # NOW it actually waits for db to be truly ready

```

Practical AIOps Example — A Monitoring Stack via Compose

Directly relevant to your important-files list from earlier — Prometheus + Grafana as an actual runnable stack:

```

version: "3.9"

services:
  prometheus:
    image: prom/prometheus:latest
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
      - prom_data:/prometheus
    ports:
      - "127.0.0.1:9090:9090"  # local-only – reach it through a reverse proxy or
SSH tunnel, not directly exposed
    restart: unless-stopped

  grafana:
    image: grafana/grafana:latest
    ports:
      - "127.0.0.1:3000:3000"
    volumes:
      - grafana_data:/var/lib/grafana
    depends_on:
      - prometheus
    restart: unless-stopped

volumes:
  prom_data:
  grafana_data:

```



```
docker compose up -d
docker compose logs -f grafana      # confirm it started cleanly
```

This gives you a persistent, restart-safe monitoring stack in about 20 lines — genuinely representative of how much of real AIOps tooling gets deployed in practice.

Debugging & Common Issues

```
docker logs <container>                # first stop, always – most failures
explain themselves here
docker inspect <container> --format '{{.State.ExitCode}}' # what exit code did it die
with
docker events                          # live stream of Docker daemon
events – useful for catching a container that keeps restarting
docker exec -it <container> sh          # get inside it and poke around
directly
```

	Likely cause	
r	The main process (CMD) finished/crashed instantly — check docker logs ; a common cause is running an interactive-only process without -it , or a missing dependency inside the image	
	Docker service isn't running (sudo systemctl status docker), or your user isn't in the docker group yet	
	Something else on the host (or another container) already bound that port — <code>sudo ss -tulnp \</code>	<code>grep <port></code> to find it
	They're not on the same custom network — check with docker network inspect <network>	
up	If using a bind mount, confirm the path is actually correct; if not using one, you need to rebuild the image (docker compose build) since the old code is baked into the image layer	

Cleanup & Maintenance

```
docker ps -a                # see what's actually accumulated
docker container prune      # remove all stopped containers
docker image prune          # remove dangling (untagged) images
docker image prune -a       # remove ALL unused images, not just
dangling ones – more aggressive
docker volume prune        # remove unused volumes – be careful,
this can delete real data
docker system prune -a --volumes # nuclear option – cleans everything
unused across the board
docker system df            # see how much disk Docker is
actually using, broken down by category
```

Worth running periodically on any dev machine — Docker images and build cache accumulate fast, and **docker system df** is the quickest way to see where your disk actually went before you go hunting manually.

Quick Reference

```
IMAGES      docker pull <image>:<tag>      docker build -t <name> .      docker
images
CONTAINERS  docker run -d -p <host>:<container> --name <name> <image>
            docker ps -a      docker logs -f <name>      docker exec -it <name> sh
VOLUMES     docker run -v <volume>:<path>      docker volume ls
NETWORKS    docker network create <name>      docker run --network <name>
COMPOSE     docker compose up -d      docker compose down      docker compose logs -
f
CLEANUP     docker system df      docker system prune -a --volumes

ALWAYS PIN VERSIONS, NEVER RELY ON :latest FOR ANYTHING YOU NEED TO TRUST.
```

Ansible Basics — Write Once, Apply to Every Server

Playbooks, inventories and idempotency — automate ten servers as easily as one.

Level: Intermediate • Words: 1,763 • Sections: 16
Commands: apt, ufw

Introduction

The core AIOps pattern: describe the state you want, run it against one server or a thousand, and Ansible figures out what actually needs to change.

What Makes Ansible Different

- **Agentless** — no software to install on target servers, just SSH access and Python. Compare this to tools that need a persistent agent daemon running everywhere.
- **Push-based** — you run Ansible from a control machine, it connects out to targets and applies changes. (Contrast with pull-based tools where each server checks in for its own config.)
- **Declarative** — you describe the desired end state ("nginx should be installed and running"), not a sequence of imperative steps ("run apt install, then run systemctl start").
- **Idempotent** — running the same playbook 100 times produces the same result as running it once. If nginx is already installed and running, Ansible recognizes that and does nothing on the second run — this is the property that makes it safe to re-run constantly, which is the entire point.

Installation

```
sudo apt install ansible -y      # on the CONTROL machine only – targets need nothing
installed
ansible --version
```

TIP

Prerequisite on every target server: SSH access (ideally key-based, per your SSH guide) and Python 3 present — which is true by default on virtually every mainstream distro already.

Inventory — Defining What "Every Server" Means

The inventory file lists the servers Ansible manages, optionally grouped.

Basic static inventory (INI-style)

```
# inventory.ini

[webservers]
web1 ansible_host=203.0.113.10
web2 ansible_host=203.0.113.11

[databases]
db1 ansible_host=203.0.113.20

[all:vars]
ansible_user=solo
ansible_ssh_private_key_file=~/.ssh/id_ed25519
```

YAML inventory (more common in real-world setups now)

```
# inventory.yml
all:
  children:
    webservers:
      hosts:
        web1:
          ansible_host: 203.0.113.10
        web2:
          ansible_host: 203.0.113.11
    databases:
      hosts:
        db1:
          ansible_host: 203.0.113.20
  vars:
    ansible_user: solo
    ansible_ssh_private_key_file: ~/.ssh/id_ed25519
```

Testing connectivity to your inventory

```
ansible all -i inventory.ini -m ping
```

This isn't ICMP ping — it's Ansible's own module that confirms it can actually SSH in and run Python on every host. This is the very first thing to run against any new inventory, before attempting anything real.

Ad-Hoc Commands — One-Off Tasks Without a Playbook

For quick, single actions across your whole fleet, without writing a file:

```
ansible webservers -i inventory.ini -m ping
ansible webservers -i inventory.ini -a "uptime" # -a runs a
raw shell command
ansible webservers -i inventory.ini -m apt -a "name=nginx state=present" --become #
install nginx everywhere in the group
ansible all -i inventory.ini -m command -a "df -h" # disk
usage across the whole fleet, one command
```

--become = escalate privileges (the Ansible equivalent of **sudo**) for that task.

Ad-hoc commands are for quick checks/fixes — anything you'd want to repeat, version-control, or reason about later belongs in a playbook instead.

Playbooks — The Real Unit of Work

A playbook is a YAML file describing a set of tasks to apply to a set of hosts.

Anatomy of a basic playbook

```
# webserver-setup.yml
---
- name: Configure web servers
  hosts: webservers
  become: true

  tasks:
    - name: Install nginx
      apt:
        name: nginx
        state: present
        update_cache: true

    - name: Ensure nginx is running and enabled
      service:
        name: nginx
        state: started
        enabled: true

    - name: Copy custom index page
      copy:
        src: files/index.html
        dest: /var/www/html/index.html
        owner: www-data
        group: www-data
        mode: '0644'
```

```
ansible-playbook -i inventory.ini webserver-setup.yml
```

Reading the structure

- **hosts**: — which inventory group (or host, or **all**) this play targets
- **become**: — whether tasks need elevated privileges
- **tasks**: — the ordered list of actions, each using a **module** (**apt**, **service**, **copy**, etc.) with parameters

The idempotency payoff, made concrete

Run the playbook above a second time immediately: Ansible checks nginx's actual installed state, actual running state, and the actual content/checksum of **index.html** — and reports **"ok"** (no change) for anything already matching the desired state, only reporting **"changed"** for things that genuinely needed to change. This is what makes it safe to run constantly (e.g. via a systemd timer, tying back to your timers guide) rather than something you have to remember to run carefully by hand.

Variables

Inline in a playbook

```
- name: Configure app
  hosts: webservers
  vars:
    app_port: 8080
    app_env: production
  tasks:
    - name: Print the port
      debug:
        msg: "App runs on port {{ app_port }}"
```

``group_vars`` and ``host_vars`` — the scalable pattern for real projects

```
inventory.ini
group_vars/
  webservers.yml      # applies to every host in the "webservers" group
  all.yml             # applies to EVERY host, regardless of group
host_vars/
  web1.yml            # applies ONLY to the specific host "web1"
```

```
# group_vars/webservers.yml
app_port: 8080
nginx_worker_processes: 4
```

Ansible loads these automatically based on filename matching group/host names — no explicit include needed. This is the real-world way variables are organized once an inventory grows beyond a handful of hosts.

Variable precedence (simplified, most to least specific wins)

```
host_vars > group_vars > playbook vars > role defaults
```

Templates — Dynamic Config Files with Jinja2

Instead of copying a static file, generate one per-host from a template using variables.

```
# templates/nginx.conf.j2
server {
    listen {{ app_port }};
    server_name {{ inventory_hostname }};

    location / {
        proxy_pass http://127.0.0.1:{{ backend_port }};
    }
}
```

```
- name: Deploy nginx config from template
  template:
    src: templates/nginx.conf.j2
    dest: /etc/nginx/sites-available/myapp.conf
  notify: Reload nginx
```

`{{ inventory_hostname }}` is a built-in variable — automatically the current host's name in the inventory, no need to define it yourself. Every host gets a config generated with **its own** values, from one shared template.

Handlers — Restart Only When Something Actually Changed

A handler is a task that only runs if explicitly **notified** by another task that reported a real change — avoiding unnecessary restarts on every single run.

```
tasks:
  - name: Deploy nginx config
    template:
      src: templates/nginx.conf.j2
      dest: /etc/nginx/sites-available/myapp.conf
    notify: Reload nginx

handlers:
  - name: Reload nginx
    service:
      name: nginx
      state: reloaded
```

TIP

Why this matters: without this pattern, you'd either reload nginx on every playbook run (wasteful, briefly disruptive) or forget to reload it at all when the config actually changes (config drift). The handler only fires when the **template** task above reports **changed**, and even if 5 different tasks notify the same handler, it only runs **once** at the end of the play — not once per notification.

Conditionals & Loops

Conditionals

```
- name: Install a package only on Debian-based systems
  apt:
    name: nginx
    state: present
  when: ansible_facts['os_family'] == "Debian"
```

Loops

```
- name: Create multiple directories
  file:
    path: "{{ item }}"
    state: directory
  loop:
    - /opt/myapp/logs
    - /opt/myapp/data
    - /opt/myapp/config
```

Combining both

```
- name: Install packages, only if not already listed as present
  apt:
    name: "{{ item }}"
    state: present
  loop:
    - nginx
    - fail2ban
    - unattended-upgrades
  when: ansible_facts['os_family'] == "Debian"
```

Roles — Reusable, Shareable Structure

Once a playbook grows past a handful of tasks, split it into a **role** — a standardized directory structure Ansible understands automatically.

```
roles/
  webserver/
    tasks/main.yml      # the actual task list
    handlers/main.yml  # handlers
    templates/          # Jinja2 templates
    files/              # static files to copy
    defaults/main.yml  # default variable values (lowest precedence, easily
overridden)
    vars/main.yml       # role-specific variables (higher precedence)
```

```
# site.yml – using the role
- name: Configure all web servers
  hosts: webservers
  become: true
  roles:
    - webserver
    - firewall
    - monitoring
```

This is what "write once, apply to every server" really means at scale — a **webserver** role written once can be applied across every project/inventory that needs a web server, versioned and shared like any other code.

Ansible Vault — Handling Secrets Properly

Never put real passwords/API keys/keys in plain playbook variables that get committed to git. Vault encrypts a file (or specific values) so secrets can live safely in version control.

```
ansible-vault create secrets.yml      # create a new encrypted file, prompts for a
password
ansible-vault edit secrets.yml        # edit it later – decrypts, opens editor,
re-encrypts on save
ansible-vault view secrets.yml        # view without editing
ansible-vault encrypt existing_vars.yml # encrypt a file that already exists
```

```
ansible-playbook -i inventory.ini site.yml --ask-vault-pass  # prompts for the vault
password at run time
ansible-playbook -i inventory.ini site.yml --vault-password-file ~/.vault_pass  # or
read it from a file (keep THAT file out of git too)
```

Practical Example — A Real Hardening Playbook

Ties together several earlier guides (firewall, fail2ban, unattended-upgrades) into one applicable playbook:


```
---
- name: Baseline server hardening
  hosts: all
  become: true

  tasks:
    - name: Install core security packages
      apt:
        name:
          - ufw
          - fail2ban
          - unattended-upgrades
        state: present
        update_cache: true

    - name: Set ufw default deny incoming
      ufw:
        direction: incoming
        policy: deny

    - name: Allow SSH through ufw
      ufw:
        rule: limit
        port: '2222'
        proto: tcp

    - name: Enable ufw
      ufw:
        state: enabled

    - name: Deploy fail2ban jail.local from template
      template:
        src: templates/jail.local.j2
        dest: /etc/fail2ban/jail.local
        notify: Restart fail2ban

    - name: Ensure fail2ban is enabled and running
      service:
        name: fail2ban
        state: started
        enabled: true

    - name: Enable unattended-upgrades
      copy:
        content: |
          APT::Periodic::Update-Package-Lists "1";
          APT::Periodic::Unattended-Upgrade "1";
        dest: /etc/apt/apt.conf.d/20auto-upgrades

  handlers:
    - name: Restart fail2ban
      service:
        name: fail2ban
        state: restarted
```

Run this against your entire fleet with one command, and every server ends up in the exact same hardened state — re-run it any time to confirm nothing's drifted.

Common Modules Reference

Module	Purpose
<code>apt / yum / dnf</code>	package management
<code>service / systemd</code>	manage services
<code>copy</code>	copy a static file to the target
<code>template</code>	render a Jinja2 template to the target

Module	Purpose
file	manage file/directory state, permissions
user / group	manage system users/groups
lineinfile	ensure a specific line exists (or doesn't) in a file — great for surgical config edits
command / shell	run an arbitrary command (prefer a proper module when one exists — it's what gives you idempotency)
debug	print a variable/message, useful for troubleshooting a playbook
git	clone/update a git repository
docker_container	manage Docker containers (needs the community.docker collection)

TIP

Important idiom: prefer a dedicated module (`apt`, `service`, `copy`) over `command/shell` whenever one exists — dedicated modules understand desired state and are properly idempotent; `command/shell` just blindly runs a command every single time, re-doing work that didn't need doing and potentially erroring on a second run.

Running, Checking & Debugging

```
ansible-playbook -i inventory.ini site.yml --check           # DRY RUN – shows what
WOULD change, changes nothing
ansible-playbook -i inventory.ini site.yml --diff           # show the actual before/
after diff for changed files
ansible-playbook -i inventory.ini site.yml -v               # verbose (stack -vv, -
vvv for more detail)
ansible-playbook -i inventory.ini site.yml --limit web1     # run against just
one host from the inventory
ansible-playbook -i inventory.ini site.yml --tags "firewall" # run only tasks
tagged "firewall"
```

`--check` is worth using as a habit before applying anything new or edited to a production fleet — it's the equivalent of `ufw`'s dry-run pattern from earlier guides, catching surprises before they actually happen.

Quick Reference

```
TEST CONNECTIVITY      ansible all -i inventory.ini -m ping
AD-HOC COMMAND          ansible <group> -i inventory.ini -a "<command>"
RUN A PLAYBOOK         ansible-playbook -i inventory.ini <playbook>.yaml
DRY RUN                 ansible-playbook -i inventory.ini <playbook>.yaml --check --
diff
LIMIT TO ONE HOST       ansible-playbook -i inventory.ini <playbook>.yaml --limit
<host>
VAULT A SECRETS FILE    ansible-vault create secrets.yaml

STRUCTURE
inventory.ini / inventory.yaml    → who
group_vars/ , host_vars/         → what values apply where
playbook.yaml                    → what to do
roles/<name>/tasks/main.yaml      → reusable, shareable "what to do"
templates/*.j2                   → dynamic config, rendered per-host

CORE PRINCIPLE: idempotent, declarative, agentless – describe the end state,
run it as often as you want, only real changes ever get applied.
```

Monitoring Stack Setup — Prometheus + Grafana From Zero

Prometheus + Grafana + alerting rules — see server health before your users do.

Level: Advanced • Words: 1,416 • Sections: 11

Commands: apt, cd, chown, cp, curl, docker +7 more

Introduction

The stack that's been sitting as a loose entry in your important-files list, actually built out: metrics collection, storage, querying, visualization, and alerting.

Architecture — How the Pieces Fit

```
Target servers
  ■■ node_exporter      → exposes hardware/OS metrics (CPU, mem, disk, network) on
port 9100

Prometheus
  ■■ scrapes node_exporter(s) on a schedule, stores the time-series data, evaluates
alert rules

Grafana
  ■■ queries Prometheus, renders dashboards

Alertmanager (optional but standard)
  ■■ receives firing alerts FROM Prometheus, handles routing/dedup/notification (Slack,
email, etc.)
```

TIP

Key distinction to keep straight: Prometheus itself can evaluate alert rules and know something's wrong — but Alertmanager is the separate component that actually decides how/where to notify someone about it (grouping, silencing, routing to different channels). Small setups often skip Alertmanager initially and just watch Grafana dashboards; it's worth adding once you actually want to be notified rather than having to go look.

Two Installation Paths

Path	Best for
Docker Compose	Fastest to get running, easiest to tear down/rebuild, good for a dev box or quick lab setup
Native systemd services	More "production," direct host-level metrics access without container networking layers, matches how most real ops teams actually run it long-term

Both are shown below — start with Compose to learn the pieces, move to native if/when you want it running as core infrastructure.

Path A: Docker Compose Setup

Step 1 — Directory structure

```
mkdir -p ~/monitoring/{prometheus,grafana}
cd ~/monitoring
```

Step 2 — Prometheus config

```
# ~/monitoring/prometheus/prometheus.yml
global:
  scrape_interval: 15s          # how often Prometheus pulls metrics from each target

scrape_configs:
  - job_name: 'prometheus'      # Prometheus monitoring itself
    static_configs:
      - targets: ['localhost:9090']

  - job_name: 'node'            # the host machine, via node_exporter
    static_configs:
      - targets: ['node-exporter:9100']
```

Step 3 — Compose file

```
# ~/monitoring/docker-compose.yml
version: "3.9"

services:
  node-exporter:
    image: prom/node-exporter:latest
    container_name: node-exporter
    restart: unless-stopped
    network_mode: host          # needs host networking to see REAL host metrics, not the
                                # container's own

  prometheus:
    image: prom/prometheus:latest
    container_name: prometheus
    restart: unless-stopped
    volumes:
      - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml
      - prom_data:/prometheus
    ports:
      - "127.0.0.1:9090:9090"    # localhost-only – see the security section before
                                # exposing this further

  grafana:
    image: grafana/grafana:latest
    container_name: grafana
    restart: unless-stopped
    ports:
      - "127.0.0.1:3000:3000"
    volumes:
      - grafana_data:/var/lib/grafana
    depends_on:
      - prometheus

volumes:
  prom_data:
  grafana_data:
```

Step 4 — Launch it

```
docker compose up -d
docker compose ps          # confirm all three are running
docker compose logs -f prometheus  # confirm it started without config errors
```

Path B: Native systemd Install (No Containers)

Step 1 — Create a dedicated user (services shouldn't run as root)

```
sudo useradd --no-create-home --shell /bin/false prometheus
```

Step 2 — Download and install Prometheus

```
cd /tmp
curl -LO https://github.com/prometheus/prometheus/releases/latest/download/prometheus-3.0.0.linux-amd64.tar.gz
tar xzf prometheus-3.0.0.linux-amd64.tar.gz
sudo mkdir -p /etc/prometheus /var/lib/prometheus
sudo cp prometheus-3.0.0.linux-amd64/prometheus /usr/local/bin/
sudo cp prometheus-3.0.0.linux-amd64/promtool /usr/local/bin/
sudo cp -r prometheus-3.0.0.linux-amd64/console /etc/prometheus/
sudo cp -r prometheus-3.0.0.linux-amd64/console_libraries /etc/prometheus/
sudo chown -R prometheus:prometheus /etc/prometheus /var/lib/prometheus
```

(Check the actual latest version at github.com/prometheus/prometheus/releases before running this — version numbers age fast.)

Step 3 — Same `prometheus.yml` as above, placed at `/etc/prometheus/prometheus.yml`

Step 4 — systemd unit (direct application of your systemd deep-dive guide)

```
# /etc/systemd/system/prometheus.service
[Unit]
Description=Prometheus
Wants=network-online.target
After=network-online.target

[Service]
Type=simple
User=prometheus
Group=prometheus
ExecStart=/usr/local/bin/prometheus \
  --config.file=/etc/prometheus/prometheus.yml \
  --storage.tsdb.path=/var/lib/prometheus \
  --web.listen-address=127.0.0.1:9090
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now prometheus
sudo systemctl status prometheus
journalctl -u prometheus -f
```

Step 5 — node_exporter, same pattern

```
cd /tmp
curl -LO https://github.com/prometheus/node_exporter/releases/latest/download/node_exporter-1.8.0.linux-amd64.tar.gz
tar xzf node_exporter-1.8.0.linux-amd64.tar.gz
sudo cp node_exporter-1.8.0.linux-amd64/node_exporter /usr/local/bin/
sudo useradd --no-create-home --shell /bin/false node_exporter
```

```
# /etc/systemd/system/node_exporter.service
[Unit]
Description=Node Exporter
After=network.target

[Service]
Type=simple
User=node_exporter
ExecStart=/usr/local/bin/node_exporter
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now node_exporter
```

Step 6 — Grafana (native)

```
sudo apt install -y software-properties-common
sudo add-apt-repository "deb https://packages.grafana.com/oss/deb stable main"
curl https://packages.grafana.com/gpg.key | sudo apt-key add -
sudo apt update
sudo apt install grafana -y
sudo systemctl enable --now grafana-server
```

Verifying Prometheus Is Actually Collecting Data

```
curl http://localhost:9090/api/v1/query?query=up
```

Or open <http://localhost:9090> in a browser (via SSH tunnel if it's bound to localhost only — see the security section) and go to **Status** → **Targets**: every scrape target should show **State: UP**. If a target shows **DOWN**, that's your first debugging stop — usually a firewall blocking port 9100, or node_exporter not actually running (`systemctl status node_exporter`).

Connecting Grafana to Prometheus

Step 1 — Access Grafana (default login `admin/admin`, forces a password change on first login):

```
http://localhost:3000
```

TIP

Step 2 — Add Prometheus as a data source: Configuration → Data Sources → Add data source → Prometheus → set URL to <http://localhost:9090> (or <http://prometheus:9090> if Grafana is running as a Docker container on the same Compose network, using the service name instead of localhost) → Save & Test.

TIP

Step 3 — Import a ready-made dashboard instead of building from scratch: Dashboards → Import → enter dashboard ID **1860** ("Node Exporter Full," the standard, most widely used community dashboard for exactly this stack) → select your Prometheus data source → Import.

This alone gets you CPU, memory, disk, and network graphs for every monitored host, without writing a single query by hand — a genuinely good starting point before customizing anything.

Basic PromQL — Enough to Be Dangerous

```

up                                # 1 = target is being scraped successfully,
0 = it's down

node_cpu_seconds_total            # raw counter – rarely useful directly,
needs rate()

rate(node_cpu_seconds_total{mode="idle"}[5m])    # CPU idle rate over a 5-min window –
the building block for CPU usage %

100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100) # CPU
USAGE %, the query you actually want

node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes * 100    # memory available
as a percentage

node_filesystem_avail_bytes{mountpoint="/" } / node_filesystem_size_bytes{mountpoint="/"
"} * 100    # disk free % on root

rate(node_disk_io_time_seconds_total[5m])        # disk I/O time – the Prometheus
equivalent of the iostat %util you already know

```

TIP

The one PromQL concept worth actually internalizing: most raw metrics are **counters** — they only ever go up (total CPU seconds spent, total bytes received). You almost never graph a counter directly; you wrap it in `rate()` to get "how fast is this increasing," which is what turns a meaningless ever-climbing number into an actual usage rate you can read.

Alerting — Prometheus Rules + Alertmanager

Step 1 — Define alert rules

```

# /etc/prometheus/alert_rules.yml
groups:
  - name: node_alerts
    rules:
      - alert: HighCPUUsage
        expr: 100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m]))
* 100) > 85
        for: 5m
        labels:
          severity: warning
        annotations:
          summary: "High CPU on {{ $labels.instance }}"
          description: "CPU usage above 85% for 5+ minutes"

      - alert: DiskSpaceLow
        expr: (node_filesystem_avail_bytes{mountpoint="/" } / node_filesystem_size_bytes{
mountpoint="/" }) * 100 < 10
        for: 10m
        labels:
          severity: critical
        annotations:
          summary: "Low disk space on {{ $labels.instance }}"
          description: "Root filesystem below 10% free – ties directly to the disk-full
failure scenario from your storage guide"

      - alert: HostDown
        expr: up == 0
        for: 2m
        labels:
          severity: critical
        annotations:
          summary: "{{ $labels.instance }} is down"
          description: "Prometheus has not been able to scrape this target for 2+
minutes"

```

TIP

Reference this in `prometheus.yml`:

```
rule_files:
  - "alert_rules.yml"
```

```
promtool check rules /etc/prometheus/alert_rules.yml    # validate syntax BEFORE
reloading – catches typos early
sudo systemctl reload prometheus
```

TIP

The `for:` field matters as much as the threshold itself: without it, a single momentary CPU spike fires an alert instantly — `for: 5m` requires the condition to stay true continuously for 5 minutes before it actually fires, filtering out normal noise and only alerting on sustained real problems.

Step 2 — Alertmanager, for actual notifications

```
# /etc/alertmanager/alertmanager.yml
route:
  receiver: 'default'
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 4h

receivers:
- name: 'default'
  # webhook, email, or Slack config goes here – e.g. for Slack:
  slack_configs:
    - api_url: 'https://hooks.slack.com/services/YOUR/WEBHOOK/URL'
      channel: '#alerts'
      send_resolved: true
```

Add to `prometheus.yml`:

```
alerting:
  alertmanagers:
    - static_configs:
      - targets: ['localhost:9093']
```

Securing the Stack

Directly applying your hardening guide's principles to this specific setup:

`bash ssh -L 3000:localhost:3000 -L 9090:localhost:9090 solo@yourserver`` Then open `http://localhost:3000` on your **local** machine — traffic tunnels securely over the existing SSH connection.

- **Bind Prometheus/Grafana to `127.0.0.1`**, exactly as done throughout this guide — neither needs to be directly internet-facing.
- **Reach them remotely via SSH tunnel or Tailscale** (from your SSH/remote-access guides), not by opening ports 9090/3000 to the world:
- **Change Grafana's default admin password immediately** on first login — an exposed default-credential Grafana instance is a genuinely common real-world compromise vector.
- **If you do need Grafana reachable more broadly**, put it behind a reverse proxy (nginx) with real TLS and its own auth, rather than exposing Grafana's own port directly.

Quick Reference


```
STACK          node_exporter (metrics source) → Prometheus (scrape+store+alert) →
Grafana (visualize)
                optionally: Alertmanager (routes firing alerts to Slack/email/etc)

DOCKER PATH    docker compose up -d    (see prometheus.yml + docker-compose.yml
above)
NATIVE PATH    systemctl enable --now prometheus node_exporter grafana-server

VERIFY SCRAPING    http://localhost:9090 → Status → Targets → all should show UP
CONNECT GRAFANA    Data Sources → Prometheus → http://localhost:9090
FAST DASHBOARD    Import dashboard ID 1860 ("Node Exporter Full")

KEY PROMQL      rate(node_cpu_seconds_total{mode="idle"}[5m])    → CPU idle
rate, the building block for usage %
                  node_filesystem_avail_bytes{mountpoint="/"}    → disk
free bytes

ALERT RULE TEST    promtool check rules /etc/prometheus/alert_rules.yml
ACCESS SECURELY    ssh -L 3000:localhost:3000 -L 9090:localhost:9090
<host>    (never expose directly)
```

tmux & screen — Sessions That Survive a Disconnect

Sessions that survive disconnects — tmux basics that pay off the first time your SSH drops mid-task.

Level: Beginner • Words: 1,324 • Sections: 7

Commands: apt, docker, docker compose, rsync, screen, tmux

Introduction

The single fix for the most annoying problem in remote server work: close your laptop, lose wifi, or get disconnected mid-SSH-session, and whatever was running just dies with it. Both tools solve this the same way — by running your actual work in a session on the **server**, independent of your SSH connection, that you can detach from and reattach to later.

Why This Matters, Concretely

Without tmux/screen, running something long on a remote box means the SSH connection itself has to survive the entire duration — a laptop going to sleep, a wifi drop, or a closed terminal kills the process along with it. This has directly mattered in every SSH-based guide so far in this series: a long-running `dd` copy, a Docker Compose stack you're watching logs on, an Ansible playbook against a large fleet, or `journalctl -f` output you want to leave running while you do something else.

TIP

The fix: start the work inside a tmux/screen session on the server. Detach whenever (deliberately, or by accident via disconnect) — the session and everything running in it keeps going. Reattach later, from the same machine or a different one entirely, and it's exactly where you left it.

tmux — The Modern Standard

```
sudo apt install tmux -y
```

Core concepts

```
Session → a full workspace, can contain multiple windows
■■ Window → like a tab, can contain multiple panes
■■ Pane → a split section of a window, each running its own shell
```

The prefix key

Every tmux command starts with a **prefix** — by default `Ctrl+b`, pressed and released, then followed by the actual command key. This guide writes it as **Prefix + <key>**.

Starting & basic session management

```
tmux                                # start a new, unnamed session
tmux new -s mysession               # start a new session with a specific name — DO THIS,
unnamed sessions are annoying to find later
tmux ls                             # list all sessions currently running on this machine
tmux attach -t mysession            # reattach to a specific named session
tmux attach                         # attach to the most recent session if only one
exists
tmux kill-session -t mysession      # terminate a session entirely
```

Detaching (the whole point)

```
Prefix + d → detach from the current session, leaving it running in the background
```

You're now back at your normal shell — the session (and everything running inside it) keeps going untouched. **Simply closing your terminal or losing your SSH connection has the same practical effect as `Prefix + d`** — tmux doesn't care how you left, the session survives either way.

Windows (tabs within a session)

Command	Effect
<code>Prefix + c</code>	create a new window
<code>Prefix + ,</code>	rename the current window (do this, it makes <code>tmux ls</code> /window-switching sane)
<code>Prefix + n</code>	next window
<code>Prefix + p</code>	previous window
<code>Prefix + 0-9</code>	jump directly to window number 0–9
<code>Prefix + w</code>	interactive window list/picker
<code>Prefix + &</code>	close the current window (asks for confirmation)

Panes (splits within a window)

Command	Effect
<code>Prefix + %</code>	split vertically (side by side)
<code>Prefix + "</code>	split horizontally (stacked)
<code>Prefix + arrow key</code>	move between panes
<code>Prefix + o</code>	cycle to the next pane
<code>Prefix + z</code>	zoom the current pane to fullscreen (toggle again to un-zoom) — great for temporarily focusing without losing the layout
<code>Prefix + x</code>	close the current pane (asks for confirmation)
<code>Prefix + Ctrl+arrow</code>	resize the current pane

TIP

Practical layout example: split a window three ways to watch a deploy — one pane running `journalctl -u myapp -f`, another running `docker stats`, a third free for actually running commands. All three stay visible simultaneously, and the whole layout survives a detach/reattach exactly as arranged.

Copy mode — scrolling back and copying text

```
Prefix + [      → enter copy mode (lets you scroll up with arrow keys / Page Up)
q              → exit copy mode
```

Inside copy mode: move the cursor, press `Space` to start a selection, move again, then `Enter` to copy it into tmux's own buffer.

```
Prefix + ]      → paste tmux's copied buffer
```

Configuration — `~/tmux.conf`

```
# ~/.tmux.conf

# Change prefix from Ctrl+b to Ctrl+a (more ergonomic for many people, matches old
screen muscle memory)
set -g prefix C-a
unbind C-b
bind C-a send-prefix

# Start window/pane numbering at 1, not 0 – matches keyboard number row order
set -g base-index 1
setw -g pane-base-index 1

# Mouse support – click to switch panes, drag to resize, scroll to scroll back
set -g mouse on

# Increase scrollback buffer size (default is quite small)
set -g history-limit 10000

# Faster escape time – noticeable input lag fix for vim users inside tmux
set -sg escape-time 0

# Status bar – show session name, host, and time
set -g status-left "[bold]#S "
set -g status-right "%Y-%m-%d %H:%M"
```

Reload a config change without restarting tmux:

```
Prefix + :source-file ~/.tmux.conf
```

screen — The Older Alternative, Still Worth Knowing

screen predates tmux and is simpler/more limited (no built-in status bar, weaker scripting), but it's genuinely still present by default on more minimal/older systems where tmux might not be installed — worth knowing the basics even if tmux is your daily driver.

```
sudo apt install screen -y
```

Basic session management

```
screen                # start a new, unnamed session
screen -S mysession   # start a new NAMED session
screen -ls            # list running sessions
screen -r mysession   # reattach to a named session
screen -r              # reattach if only one session exists
screen -d -r mysession # force-detach it from anywhere else it's
                        # attached, then reattach here (useful if you forgot you left it attached on another
                        # terminal)
```

Detaching

```
Ctrl+a d    → detach (screen's prefix is Ctrl+a, not Ctrl+b – note the difference
from tmux's default)
```

Windows

Command	Effect
Ctrl+a c	create a new window
Ctrl+a n	next window
Ctrl+a p	previous window

Command	Effect
<code>Ctrl+a "</code>	interactive window list
<code>Ctrl+a A</code>	rename the current window

Basic splits (screen's split support is more limited than tmux's)

```
Ctrl+a S    → split horizontally
Ctrl+a |    → split vertically (newer screen versions)
Ctrl+a Tab  → switch between splits
```

tmux vs screen — Direct Comparison

	tmux	screen
Default prefix	<code>Ctrl+b</code>	<code>Ctrl+a</code>
Pane splitting	Full support, flexible, resizable	Present but more limited/clunkier
Scripting/automation	Strong (<code>tmux send-keys</code> , scriptable session setup)	Weaker
Status bar	Built-in, customizable	Not built-in
Mouse support	Yes (<code>set -g mouse on</code>)	Limited
Active development	Yes	Largely maintenance-only at this point
Availability	Needs installing on most distros	Sometimes present by default already, especially on older/minimal systems

TIP

Practical takeaway: learn tmux as your daily driver — it's more capable and what most current documentation/dotfiles assume. Recognize screen's basics for the occasional older box where it's the only one already installed and you don't want to bother installing tmux for a two-minute task.

Practical Workflow — Tying This Into Your Existing Guides

TIP

Running a long Ansible playbook against your fleet without babysitting the SSH connection:

```
tmux new -s ansible-run
ansible-playbook -i inventory.ini site.yml
# Prefix + d to detach, close your laptop, walk away
# later, from anywhere:
tmux attach -t ansible-run
```

TIP

Watching a Docker Compose monitoring stack while working on something else:

```
tmux new -s monitoring
Prefix + %                # split the window
# left pane:
docker compose logs -f
# Prefix + Right arrow to move to the right pane, then:
docker stats
```

TIP

A **dd** disk copy or large **rsync** that would otherwise die on disconnect:

```
tmux new -s backup
sudo rsync -avh /home/ /mnt/backup-drive/
# Prefix + d – this now survives your SSH connection dropping entirely
```

Quick Reference

START NAMED SESSION	tmux new -s <name>		
LIST SESSIONS	tmux ls		
REATTACH	tmux attach -t <name>		
DETACH	Prefix + d	(tmux default prefix: Ctrl+b)	
WINDOWS	Prefix + c (new)	Prefix + n/p (next/prev)	Prefix +
, (rename)			
PANES	Prefix + % (vert split)	Prefix + " (horiz split)	
Prefix + arrow (move)			
ZOOM A PANE	Prefix + z		
COPY MODE	Prefix + [	then Space to select, Enter to copy	
Prefix +] to paste			
SCREEN EQUIVALENT	screen -S <name>	/	screen -r <name> /
Ctrl+a d to detach			

RULE OF THUMB: if it's going to run longer than you're willing to babysit an SSH connection for, start it inside a named tmux session first – always.

Linux for Data Engineers — Comprehensive Overview

The Linux concepts data pipelines actually touch — filesystems, processes, cron and resource limits.

Level: Intermediate • Words: 777 • Sections: 6

Introduction

This document is the first in a series of articles titled "Linux for Data Engineers | A Comprehensive Overview". It aims to provide a high-level overview of Linux concepts that are relevant to data engineering, focusing on filesystems, processes, cron, and resource limits. The series will delve deeper into these topics in subsequent articles.

1. Why Linux for Data Engineers?

- Open Source and Community:** Linux is an open-source operating system, meaning its source code is freely available and can be modified and distributed by anyone. This fosters a large, active community that continuously improves the system and provides support.
- Stability and Security:** Linux is known for its stability and security, making it a reliable choice for production environments. Its modular architecture allows for fine-grained control over system resources and permissions.
- Flexibility and Customization:** Linux can be tailored to fit specific hardware and software requirements. This flexibility is crucial for data engineering, where different workloads and data sources may require specialized configurations.

2. The History of Linux: From Kernel to Operating System

I. The Birth of Linux at Bell Labs

- 1980s:** The story begins with the development of the Linux kernel by Linus Torvalds in 1991. However, the roots of Linux trace back to the work of Ken Thompson and Dennis Ritchie at Bell Labs in the early 1980s. They developed the Unix operating system, which inspired the design of Linux.
- 1985:** The American Physical Society awarded the Volta Prize to Ken Thompson for his work on the development of the Unix operating system, valued at 100,000 dollars.
- 1989:** The AT&T Bell Laboratories awarded the Turing Award to Dennis Ritchie for his work on the development of the Unix operating system.
- 1991:** Linus Torvalds, a Finnish computer science student, began the development of the Linux kernel while working at the University of Helsinki. He was inspired by the Unix operating system and wanted to create a free, open-source alternative.
- 1995:** The Linux kernel was released under the GNU General Public License (GPL), making it a free and open-source project.
- 1996:** The Linux kernel was integrated with the GNU operating system, creating the Linux operating system.
- 1997:** The Linux kernel was released under the GNU General Public License (GPL), making it a free and open-source project.

II. The Evolution of Linux: From Kernel to Operating System

- 1998:** The Linux kernel was integrated with the GNU operating system, creating the Linux operating system. This marked the beginning of the Linux ecosystem.
- 2000:** The Linux kernel was released under the GNU General Public License (GPL), making it a free and open-source project.
- 2001:** The Linux kernel was released under the GNU General Public License (GPL), making it a free and open-source project.
- 2002:** The Linux kernel was released under the GNU General Public License (GPL), making it a free and open-source project.
- 2003:** The Linux kernel was released under the GNU General Public License (GPL), making it a free and open-source project.
- 2004:** The Linux kernel was released under the GNU General Public License (GPL), making it a free and open-source project.
- 2005:** The Linux kernel was released under the GNU General Public License (GPL), making it a free and open-source project.
- 2006:** The Linux kernel was released under the GNU General Public License (GPL), making it a free and open-source project.
- 2007:** The Linux kernel was released under the GNU General Public License (GPL), making it a free and open-source project.
- 2008:** The Linux kernel was released under the GNU General Public License (GPL), making it a free and open-source project.
- 2009:** The Linux kernel was released under the GNU General Public License (GPL), making it a free and open-source project.
- 2010:** The Linux kernel was released under the GNU General Public License (GPL), making it a free and open-source project.
- 2011:** The Linux kernel was released under the GNU General Public License (GPL), making it a free and open-source project.
- 2012:** The Linux kernel was released under the GNU General Public License (GPL), making it a free and open-source project.
- 2013:** The Linux kernel was released under the GNU General Public License (GPL), making it a free and open-source project.
- 2014:** The Linux kernel was released under the GNU General Public License (GPL), making it a free and open-source project.
- 2015:** The Linux kernel was released under the GNU General Public License (GPL), making it a free and open-source project.
- 2016:** The Linux kernel was released under the GNU General Public License (GPL), making it a free and open-source project.
- 2017:** The Linux kernel was released under the GNU General Public License (GPL), making it a free and open-source project.
- 2018:** The Linux kernel was released under the GNU General Public License (GPL), making it a free and open-source project.
- 2019:** The Linux kernel was released under the GNU General Public License (GPL), making it a free and open-source project.
- 2020:** The Linux kernel was released under the GNU General Public License (GPL), making it a free and open-source project.
- 2021:** The Linux kernel was released under the GNU General Public License (GPL), making it a free and open-source project.
- 2022:** The Linux kernel was released under the GNU General Public License (GPL), making it a free and open-source project.
- 2023:** The Linux kernel was released under the GNU General Public License (GPL), making it a free and open-source project.
- 2024:** The Linux kernel was released under the GNU General Public License (GPL), making it a free and open-source project.
- 2025:** The Linux kernel was released under the GNU General Public License (GPL), making it a free and open-source project.

■. ■■■■ ■■■■ (Linux Kernel)

- **Linux Kernel** (Linus Torvalds) **Open Source** (Kernel).
- **Operating System (Kernel):** **Manages** **Hardware** (CPU) **Memory** (RAM) **Storage** (Disk) **Network** (Network).
- **Open Source** (Open Source) **Community**.

3 ■■■ ■■■■■■■■■■ ■■■■■■■■■■ ■■■■■■ ■■■■■■■■■■ (Red Hat ■■■■■)

- **Red Hat Enterprise Linux (RHEL):** The most widely used Linux distribution in enterprise environments. It is known for its stability, security, and long-term support.
- **Ubuntu Server:** A popular Linux distribution known for its ease of use and frequent updates. It is often used for web servers and cloud environments.
- **CentOS:** A community-driven Linux distribution that is binary-compatible with RHEL. It is often used as a cost-effective alternative to RHEL.
- **Debian:** A stable and reliable Linux distribution that is often used as a base for other distributions. It is known for its strong security and privacy focus.
- **SUSE Linux Enterprise Server (SLES):** A Linux distribution that is often used in enterprise environments. It is known for its strong security and support for a wide range of hardware.
- **Fedora Server:** A Linux distribution that is often used for development and testing. It is known for its cutting-edge features and frequent updates.

4██ ██████████ ███ ████████ ██████████████ (Terminal)

- **Profiles:** (Terminal) - (New Tab) " (Default Profile) Profile
- (Hidden Files/Directories) (.)
- **ls** **-a** (**--a**)

5. [REDACTED]: [REDACTED] [REDACTED] [REDACTED] ([REDACTED] [REDACTED])

■■■■ ■■■■■■■■ ■■■■■■■■ ■■■■■■■■ ■■■■■■■■ ■■■■■■■■ ■■■■■■■■

- Media Sosial:** Banyak pengguna media sosial merasa tertekan untuk selalu menampilkan kehidupan mereka yang sempurna. Platform seperti Instagram, Facebook, Blog, Vlog, TikTok, dan YouTube.
- Perfectionism (Ketidaksempurnaan):** Banyak orang memiliki sifat ketidaksempurnaan yang berlebihan. Mereka merasa bahwa mereka harus selalu tampil sempurna di depan orang lain, dan ini dapat menyebabkan stres.
- Perbandingan Sosial:** Banyak orang merasa bahwa mereka harus selalu tampil sempurna seperti orang lain. Mereka merasa bahwa mereka harus selalu tampil sempurna seperti orang lain.
- Persepsi Masyarakat:** Banyak orang merasa bahwa mereka harus selalu tampil sempurna seperti orang lain. Mereka merasa bahwa mereka harus selalu tampil sempurna seperti orang lain.
- Persepsi Diri Sendiri:** Banyak orang merasa bahwa mereka harus selalu tampil sempurna seperti orang lain. Mereka merasa bahwa mereka harus selalu tampil sempurna seperti orang lain.

■ ■ ■ ■ ■ "■■■■■■■■■■" (Ubuntu)

[illegible]

"■■■■■■■■" (Ubuntu):

- [illegible]

Linux User & Group Management — Extended Topics

PAM, LDAP/AD via SSSD, /etc/skel, disk quotas, newgrp and login auditing — identity management at scale.

Level: Advanced • **Words:** 1,422 • **Sections:** 8

Commands: cat, echo, id, last, lastlog, ls +3 more

Introduction

Companion to [linux-user-group-management.md](#). Covers the deeper topics that sit beneath and around everyday user/group administration: how authentication actually gets enforced (PAM), how accounts scale across many machines (centralized auth), how new accounts get their starter files (`/etc/skel`), per-user/group disk limits (quotas), switching active groups mid-session (`newgrp`), and tracking login activity (auditing).

PART 1: PAM — Pluggable Authentication Modules

PAM is the layer that actually decides "is this login allowed?" — `passwd`, `sudo`, `su`, `sshd`, and login managers don't implement authentication themselves; they all call into PAM and follow whatever policy is configured there.

How it fits together

```
User runs `sudo` ███ sudo consults PAM config (/etc/pam.d/sudo) ███ PAM runs a stack of modules ███ allow / deny
```

Concept	What it means
Service	The application asking for auth (sudo , sshd , login , passwd) — each has its own config file in /etc/pam.d/
Module type	auth (who are you), account (is the account valid — expired? locked?), password (how passwords are set/changed), session (setup/teardown when a session starts/ends)
Control flag	required , requisite , sufficient , optional — determines how a module's pass/fail affects the overall stack result
Module path	The actual .so file doing the work, e.g. pam_unix.so , pam_faillock.so

Common Files & Commands

File / Command	Purpose	! Notes / Gotchas
<code>/etc/pam.d/system-auth</code> (or <code>common-auth</code> on Debian)	Shared baseline policy most services include	This is usually the file you actually want to edit, rather than every individual service file.
<code>/etc/pam.d/sudo</code>	PAM policy specifically for <code>sudo</code>	Broken syntax here can lock out <code>sudo</code> — always keep a root shell open while testing changes.
<code>pam_faillock.so</code>	Locks accounts after N failed login attempts	Configured via <code>/etc/security/faillock.conf</code> on modern RHEL.

File / Command	Purpose	! Notes / Gotchas
<code>pam_pwquality.so</code>	Enforces password complexity rules	Configured via <code>/etc/security/pwquality.conf</code> (min length, character classes, etc.).
<code>faillock --user username</code>	View failed login attempts for a user	Replaces the older <code>pam_tally2</code> on modern RHEL.
<code>faillock --user username --reset</code>	Clear failed attempt count / unlock	Use after legitimately unlocking someone who mistyped a password too many times.

WARN

PAM configs are read top-to-bottom, and order matters. A misplaced `sufficient` line can accidentally let a stack "pass" earlier than intended, and a syntax mistake in a core file can lock out **all** authentication on the system — test in a non-production environment first, and never edit these over a session you can't recover if it breaks.

PART 2: Centralized Authentication (LDAP / AD / SSSD)

For more than a handful of machines, managing local `/etc/passwd` entries on every box doesn't scale — centralized auth lets many machines share one user directory.

The Pieces

Component	Role
LDAP (e.g. FreeIPA, OpenLDAP)	A directory server holding user/group records, queried over the network
Active Directory (AD)	Microsoft's directory service — common in mixed Windows/Linux environments
SSSD (System Security Services Daemon)	The client-side service on each Linux machine that talks to LDAP/AD/FreeIPA/Kerberos and caches results locally
Kerberos	Ticket-based authentication protocol, often paired with LDAP/AD for actual credential verification

Why This Matters for Commands You Already Know

Commands like `id`, `getent passwd`, and `getent group` don't just read local files — they query **whatever's configured** in `/etc/nsswitch.conf`, which can point to SSSD (and therefore LDAP/AD) instead of, or in addition to, local files.

```
cat /etc/nsswitch.conf | grep passwd      # shows lookup order, e.g. "files sss"
getent passwd someuser                  # works whether the user is local OR in LDAP/
AD, transparently
id someuser                             # same – works across both
systemctl status sssd                   # check if the SSSD client is running
ssscctl user-checks someuser            # (FreeIPA/SSSD) diagnose why a lookup for
a user is succeeding/failing
```

WARN

If a user "exists in AD/LDAP" but `id username` fails locally, the problem is almost always SSSD (not running, misconfigured, or can't reach the directory server) — not the directory itself. Check `systemctl status sssd` and `journalctl -u sssd` before assuming the account itself is broken.

TIP

Where this typically shows up on the RHCSA track: FreeIPA/IdM integration and `realmd/sss` join commands (`realm join domain.example.com`) are common Red Hat Academy / real-world topics, though not always core RHCSA exam objectives — worth knowing conceptually even if your exam version doesn't test it directly.

PART 3: `/etc/skel` — Default Home Directory Contents

When you create a user with `useradd -m`, the new home directory isn't empty — it's a **copy of** `/etc/skel`.

```
ls -la /etc/skel/          # see what every new user currently gets by default
# typically: .bashrc  .bash_profile  .bash_logout
```

TIP

Customizing what new users get:

```
echo 'export EDITOR=vim' >> /etc/skel/.bashrc      # every NEW user created from now on
inherits this
mkdir /etc/skel/Projects                          # every new user gets a starter
"Projects" folder
```

WARN

`/etc/skel` only affects **future** account creation — editing it does nothing for users who already exist. To retroactively apply a change to existing users, you'd need to copy the relevant file into each existing home directory yourself.

PART 4: Disk Quotas — Limiting Usage Per User/Group

Prevents one user or group from filling an entire filesystem.

Setup Overview

```
# 1. Filesystem must be mounted with quota support (in /etc/fstab):
#   /dev/sdb1 /home xfs defaults,uquota,gquota 0 0

mount -o remount /home          # apply after editing fstab
quotacheck -cug /home          # create initial quota files (u=user, g=group)
quotaon /home                   # turn quota enforcement on
```

Setting & Viewing Limits

Command	Description	! Notes
<code>edquota -u username</code>	Edit a user's quota limits interactively (opens \$EDITOR)	Sets both a soft limit (warning) and hard limit (hard stop).
<code>edquota -g groupname</code>	Edit a group's quota limits	Same soft/hard concept, applied to the group total.
<code>quota -u username</code>	View a user's current usage vs. limits	Run as the user themselves, or root to check anyone.
<code>repquota /home</code>	Summary report of everyone's usage on a filesystem	Good for an at-a-glance audit of a shared volume.
<code>edquota -t</code>	Set the grace period — how long a soft limit can be exceeded before it's enforced as hard	E.g. "7 days over soft limit before writes are blocked."

WARN

Soft vs. hard limit, the distinction that actually matters: soft limits can be exceeded temporarily (user gets a warning, has a grace period to clean up); hard limits cannot be exceeded at all — further writes simply fail once hit. Setting only a hard limit with no soft limit/grace period gives users no warning before things start failing.

TIP

RHEL note: on XFS (the RHEL default filesystem), quotas are managed slightly differently at mount time (`uquota/gquota/pquota` mount options) compared to the traditional `quotacheck`-based ext4 workflow — check which filesystem you're on before following older ext-focused tutorials verbatim.

PART 5: `newgrp` — Switching Active Primary Group Mid-Session

If a user belongs to multiple groups but wants files they create right now to be owned by a different group than their default primary — without logging out and back in.

```
newgrp developers      # starts a new shell with "developers" as the active primary
group
# ... create files here, they'll be group-owned by "developers" ...
exit                  # return to the previous shell/group context
```

vs.	Behavior
<code>newgrp groupname</code>	Temporary, only affects the current shell session, reverts on <code>exit</code>
<code>usermod -g groupname user</code>	Permanent change to the account's actual primary group

WARN

`newgrp` only works for groups the user is **already a member of** (primary or supplementary) — it doesn't grant new access, it just changes which existing group is "active" for newly created files in that session.

PART 6: Auditing — Tracking Logins & Failures

Command	Description	! Notes / Gotchas
<code>last</code>	Successful login history (who, when, from where)	Reads <code>/var/log/wtmp</code> . Add a username to filter: <code>last jdoe</code> .
<code>lastb</code>	Failed login attempts	Reads <code>/var/log/btmp</code> — requires root to read. Good first stop when investigating brute-force attempts.
<code>lastlog</code>	Most recent login per user (all accounts, one line each)	Shows "Never logged in" for unused/system accounts — useful for spotting dormant accounts.
<code>faillock --user username</code>	PAM-based failed attempt count (modern RHEL)	Pairs with <code>pam_faillock.so</code> from Part 1 — this is what actually triggers auto-lockouts.
<code>journalctl -u sshd</code>	Full systemd journal for the SSH service	More verbose than <code>last/lastb</code> — useful for seeing the reason a login failed, not just that it failed.
<code>who / w</code>	Who's logged in right now	<code>w</code> additionally shows what each session is currently doing.

TIP

A basic audit routine:

```
lastlog | grep -v "Never logged in"  # who's actually used their account recently
lastb -n 20                          # last 20 failed login attempts
faillock --user jdoe                  # check if a specific account is close to
auto-lockout
```

Quick Reference Card

```
# PAM
faillock --user NAME          # check failed attempts
faillock --user NAME --reset  # clear/unlock

# Centralized auth
getent passwd NAME            # works for local or LDAP/AD users
systemctl status sssd        # check the client daemon

# Skeleton files
ls -la /etc/skel/             # what new users inherit

# Quotas
edquota -u NAME               |   edquota -g NAME
quota -u NAME                  |   repquota /home

# Active group switch
newgrp GROUPNAME              # temporary, this session only

# Auditing
last NAME | lastb | lastlog | w
```

Part V — NTI Linux Course (RH124)

*Red Hat Enterprise Linux 9 — slides + curated notes + labs. Mirrors the site's **NTI Linux** tab.*

Day 1 — Get Started with Red Hat Enterprise Linux

An introduction to Linux, RHEL, the command line, and file management fundamentals.

Learning Objectives

- Explain the purpose of open source, Linux, Linux distributions, and Red Hat Enterprise Linux.
- Log in to a Linux system and run simple commands from the shell.
- Describe how Linux organizes files and manage them from the command line.

Understanding Linux and Its Origins

A Brief History

Linux traces its lineage back to the UNIX operating system, developed through decades of incremental innovation:

- **1969** — Thompson and Ritchie create AT&T UNIX.
- **1972** — UNIX version 2 is rewritten in C.
- **1977** — BSD (Berkeley Software Distribution) is released.
- **1983** — Richard Stallman launches the GNU Project.
- **1987** — Andy Tannenbaum creates Minix.
- **1991** — Linus Torvalds creates i386 Linux.
- **1992** — Linux is licensed under the GPL (GNU General Public License).

Key Takeaway: Linux emerged from decades of UNIX development, combining Torvalds' kernel with the GNU Project's tools and the GPL's open licensing model.

Linux Distributions

A **Linux distribution** (distro) packages the Linux kernel together with system tools, libraries, and software into a complete, installable operating system. Distributions descend from UNIX-inspired lineages and split into distinct families:

- **Debian family** → Ubuntu → Linux Mint
- **Fedora family** → RHEL → CentOS, Oracle Linux
- **SUSE family** → SLES → OpenSUSE
- Other distributions include Red Hat Linux, Fedora, and Kali Linux

All distributions share the same Linux kernel at their core, but differ in package management, target use case, and support model.

Why Linux?

Linux's popularity across servers, desktops, and embedded systems comes from a combination of practical advantages:

- **Open Source** — source code is freely available and modifiable.
- **It is Free** — no licensing cost for most distributions.
- **High Security** — a strong permissions model and active security community.
- **High Stability** — long uptimes and reliable performance.
- **Ease of Maintenance** — centralized package management and scripting support.

- **Runs on Any Hardware** — from embedded devices to enterprise servers.
- **Customization** — deep configurability at every layer.
- **Community Support** — a large global base of contributors and users.
- **Education and Support** — extensive documentation and learning resources.

TIP

Quick check: Run `uname -a` and `cat /etc/os-release` to confirm your distro family — Debian shows `ID=ubuntu`, RHEL shows `ID=rhel`.

```
uname -r
cat /etc/os-release | grep -E "(NAME|VERSION|ID)"
lsb_release -a 2>/dev/null || echo "lsb_release not installed"
```

Linux Components

Linux systems are built from three core layers, each with a distinct role.

Kernel

The **kernel** is the core of the operating system.

- Contains components such as device drivers.
- Loads into RAM when the machine boots.
- Stays resident in RAM until the machine powers off.

Shell

The **shell** is the interface through which a user communicates with the kernel.

- Common shells include C shell, ksh, and Bash.
- **Bash** is the most commonly used shell on Linux.
- The shell parses commands entered by the user and translates them into logical segments executed by the kernel or other utilities.

INFO

Key Takeaway: The kernel manages hardware and system resources, while the shell provides the interface users rely on to issue commands to that kernel.

INFO

Mental model: CLI → Shell (bash) → Kernel → Hardware. Verify your shell with `echo $SHELL` and kernel with `uname -r`.

```
echo $SHELL
echo $BASH_VERSION
uname -r
ps -p $$ -o comm=
```

Virtual Machines & Network Setup

- **Creating a VM:** `File -> New VM -> Installing Redhat on VMware` (Recommended: 2 processors, 2 cores).
- **Network Types:** Bridged, NAT, Host-only (chosen according to the IP range).
- Choose **NAT** when setting up VMware.
- **Redhat Developer Account:** You can use developer account credentials to log in to a Redhat server with a GUI on VMware.

Getting IP & SSH

```
ifconfig
```

Returns the IP address (Skip this step for Ubuntu).

```
ssh root@ipaddress
```

Connects to the server.

Minimum Requirements for RHEL 9

Before installing RHEL 9, ensure the target system meets these minimums:

- **CPU:** Dual or quad core processor
- **RAM:** 2 GB or more
- **Disk space:** 20 GB or more
- 10 GB for root (/)
- 1 GB for swap
- 4 GB for /home
- 512 MB for /boot
- **Network:** Working network connection
- **Installation media** required

Access the Command Line

There are two primary ways to reach a Linux shell:

- **From a Graphical User Interface (GUI)** — opening a terminal application within a desktop environment such as GNOME.
- **Remote access over Secure Shell (SSH)** — connecting to the system from another machine.

Running Commands

Commands in Linux follow a consistent syntax:

```
command [options] [arguments]
```

- Each item is separated by a space.
- **Options** modify the command's behavior.
- **Arguments** are file names or other information the command needs.
- Multiple commands can be separated with a semicolon (;).

Example

```
usermod -L omar
```

Command Syntax: Right vs. Wrong

Spacing matters. Options must be separated from the command and from each other correctly.

Right	Wrong
\$ ls -l /dev	\$ ls - l /dev
\$ ls -a /dev	\$ ls-a /dev
\$ mail -s test root	\$ mail test root -s
\$ who -u	\$ -u who
\$ ls -l -d	\$ ls -l-d

Right	Wrong
\$ ls -ld	\$ ls -l d

INFO

Important: Options must be attached directly to their dash (no space between `-` and the letter), and the order of arguments relative to options matters for correct parsing.

Shell Shortcuts

Bash provides keyboard shortcuts that save time when editing commands at the prompt:

Shortcut	Action
Ctrl+A	Jump to the beginning of the command line.
Ctrl+E	Jump to the end of the command line.
Ctrl+U	Clear from the cursor to the beginning of the command line.
Ctrl+K	Clear from the cursor to the end of the command line.
Ctrl+LeftArrow	Jump to the beginning of the previous word on the command line.
Ctrl+RightArrow	Jump to the end of the next word on the command line.
Ctrl+R	Search the history list of commands for a pattern.

NOTE

Spacing trap: `ls -l` (space) $\neq$ `ls -l`. The shell splits on spaces — `-` must hug its letter. Also `mail test root -s` fails because `-s` must precede the address.

```
whoami; pwd; echo $HOME
ls -l /dev | head
# try history search
# Ctrl+R then type "ssh" — press Ctrl+R again to cycle
```

Manage Files from the Command Line

Linux organizes all data in a single hierarchical tree of files and directories, rooted at `/`.

The File System Hierarchy

[Diagram — see online version]

Every directory branches from the single root (`/`), and each serves a specific, standardized purpose.

Important Directories

Location	Purpose
<code>/usr</code>	Installed software, shared libraries, include files, and read-only program data.
<code>/usr/bin</code>	User commands.
<code>/usr/sbin</code>	System administration commands.
<code>/usr/local</code>	Locally customized software.
<code>/etc</code>	Configuration files specific to this system.
<code>/var</code>	Variable data that persists between boots — databases, cache directories, log files, printer-spoiled documents, and website content.
<code>/run</code>	Runtime data for processes started since the last boot, including process ID and lock files. Contents are recreated on reboot; this directory consolidates <code>/var/run</code> and <code>/var/lock</code> from earlier RHEL versions.

Location	Purpose
/home	Personal data and configuration files for regular users.
/root	Home directory for the administrative superuser, root.
/tmp	World-writable space for temporary files. Files untouched for 10 days are deleted automatically.
/boot	Files needed to start the boot process.
/dev	Special *device files* used by the system to access hardware.

INFO

Note: A second temporary directory, `/var/tmp`, also exists — files there are deleted automatically only after 30 days of inactivity, giving them a longer lifespan than files in `/tmp`.

File Types

Every file in Linux has a type, indicated by a leading symbol in a long directory listing (`ls -l`):

Symbol	Meaning
-	Regular file
d	Directory
l	Link
c	Special File
s	Socket
p	Named Pipe
b	Block Device

Rules for Naming Files

Should:

- Be descriptive.
- Use only alphanumeric characters: uppercase, lowercase, numbers, `@`, `_`.

Should Not:

- Include embedded blanks (spaces).
- Contain shell metacharacters: `* ? > < / ; & ! [ ] | \ ' " ( ) { }`

Additional rules:

- Filenames are case sensitive.
- Filenames starting with a `.` are hidden.
- Maximum filename length is 255 characters.

Absolute Path vs. Relative Path

Every file and directory can be referenced two ways:

- **Absolute path** — the full path starting from the root `/`, unambiguous regardless of the current location (e.g., `/home/alice/document.txt` or `/var/log/messages`).
- **Relative path** — a path expressed relative to the current working directory, which changes depending on where you currently are in the file system.

INFO

Key Takeaway: An absolute path always starts at `/` and works from anywhere; a relative path is shorter but only valid from your current location.

DANGER

Spaces & dashes: Quote names with spaces: `touch "my notes.txt"` or escape `touch my\ notes.txt`. For leading dash: `touch -- -weird` and `rm -- -weird --` ends option parsing.

```
touch "my file.txt" && ls -b
touch -- -dash && ls -l -- -dash
rm -- -dash
mkdir -p "My Projects/src" && tree -L 2 2>/dev/null || find . -maxdepth 2 | sort
```

Inodes, Hard Links, and Soft Links

What Is an Inode?

Linux allocates an **index node (inode)** for every file and directory in the filesystem. Inodes do not store the actual file data — instead, they store the metadata that points to where the file's data blocks live on disk.

Metadata Stored in an Inode

- File type
- Permissions
- Hard links count
- Owner ID
- Group ID
- Soft/Hard Links
- Access Control List (ACLs)
- Size of file
- Timestamp (access time)
- Timestamp (modification time)

A **hard link** is a direct reference to a file's inode, while a **file** itself is also just a name pointing to an inode. A **soft link** does not reference the inode directly — it points to the file name/path instead, and both hard links and soft links are maintained by the file system.

Soft Link vs. Hard Link

Soft Link	Hard Link
An alias to the original file, similar to the shortcut feature in Windows OS.	The exact replica of the original file it is pointing to.
Contains only the location to the original file, not the actual data.	Contains the actual content of the file.
Has a different inode value pointing to the original value.	Shares the same inode value pointing to the same file location.
Can be created across filesystems.	Cannot be created outside the filesystem.
Becomes inaccessible when the original file is removed.	Changes in the hard-linked file will reflect in the other files.
Can link both to a file or a directory.	Can only link to a file, not a directory.

INFO

Key Takeaway: A hard link is another name for the exact same file data, while a soft link is a pointer to a file's location that breaks if the original is deleted.

TIP

See it live: `ls -i file; stat file; df -i` — if `df -i` shows 100% IUse, you can be “out of space” even with free GB.

```
echo hello > a.txt
ln a.txt hard.txt
ln -s a.txt soft.txt
ls -li a.txt hard.txt soft.txt
cat soft.txt
echo world >> a.txt
cat hard.txt # shows world too (same inode)
cat soft.txt
```

Pattern Matching (Regex)

Bash supports pattern matching (globbing) to efficiently reference multiple files at once.

Shell Pattern-Matching Symbols

Pattern	Matches
*	Any string of zero or more characters.
?	Any single character.
[abc...]	Any one character in the enclosed class (between the square brackets).
[!abc...]	Any one character *not* in the enclosed class.
[^abc...]	Any one character *not* in the enclosed class.
[[alpha:]]	Any alphabetic character.
[[lower:]]	Any lowercase character.
[[upper:]]	Any uppercase character.
[[alnum:]]	Any alphabetic character or digit.
[[punct:]]	Any printable character not a space or alphanumeric.
[[digit:]]	Any single digit from 0 to 9.
[[space:]]	Any single white space character. This may include tabs, newlines, carriage returns, form feeds, or spaces.

Regular Expressions

Regular expressions (regex) extend pattern matching with more powerful syntax, commonly used with tools like `grep` and `sed`:

Symbol	Description
.	Replaces any character.
^	Matches start of string.
\$	Matches end of string.
*	Matches zero or more times the preceding character.
\	Represents special characters.
()	Groups regular expressions.
?	Matches exactly one character.

INFO

Key Takeaway: Shell pattern matching (globbing) and regular expressions are related but distinct — globbing operates on filenames at the shell level, while regex is a more expressive matching language used inside text-processing tools.

INFO

Glob vs regex: Shell glob `*.txt` is not regex `.*\.``txt`. Globbs expand before the command runs — try `echo *.txt` to see.

```
touch a.txt B.TXT file1.log
ls *.txt          # glob
ls [a-z]*.txt     # range
ls [!a]*          # not a
# regex with grep (different!)
grep -E "^\a.*\.
```

Key Takeaways

- Linux is an open-source operating system descended from UNIX, licensed under the GPL, with distributions (distros) built around a shared kernel but differing in packaging and tooling.
- The **kernel** manages hardware and resources; the **shell** (commonly Bash) is the interface for issuing commands.
- Commands follow the syntax `command [options] [arguments]`, and correct spacing between options is essential.
- Linux organizes everything into a single hierarchical filesystem rooted at `/`, with standardized directories like `/etc`, `/var`, `/home`, and `/tmp` serving specific purposes.
- Every file and directory has an **inode** storing its metadata; hard links share an inode with the original file, while soft links merely point to its path.
- Bash pattern matching and regular expressions allow efficient, flexible operations across many files at once.

Users & Privileges (Preview)

Symbol	Meaning
\$	Regular user
#	Superuser

Day 2 — Get Help, Manage Text Files, and Administer Local Users

Resolving problems with local help systems, working with text files and shell redirection, and managing local users, groups, and password policies.

Learning Objectives

- Find information in local Linux system manual pages.
- Redirect command output and errors to files, and chain commands with pipes.
- Create and edit text files from the command line with the vim editor.
- Set shell and environment variables to modify shell behavior.
- Create, manage, and delete local users and groups, and administer password policies.

Get Help in Red Hat Enterprise Linux

Linux systems ship with an extensive built-in reference: the manual pages (**man** pages). Knowing how to navigate them turns any unfamiliar command into a solvable problem.

Manual Page Sections

The manual is divided into numbered sections, each covering a different category of documentation:

Section	Content Type
1	User commands (both executable and shell programs)
2	System calls (kernel routines invoked from user space)
3	Library functions (provided by program libraries)
4	Special files (such as device files)
5	File formats (for many configuration files and structures)
6	Games (historical section for amusing programs)
7	Conventions, standards, and miscellaneous (protocols, file systems)
8	System administration and privileged commands (maintenance tasks)
9	Linux kernel API (internal kernel calls)

Structure of a Manual Page

Each man page follows a consistent internal structure, making it easy to jump straight to the information you need:

Heading	Description
NAME	Subject name — usually a command or file name — with a very brief description.
SYNOPSIS	Summary of the command syntax.
DESCRIPTION	In-depth description to provide a basic understanding of the topic.
OPTIONS	Explanation of the command execution options.
EXAMPLES	Examples of how to use the command, function, or file.
FILES	A list of files and directories related to the man page.
SEE ALSO	Related information, normally other man page topics.
BUGS	Known bugs in the software.
AUTHOR	Information about who has contributed to the development of the topic.

INFO

Key Takeaway: Manual pages are organized by section and follow a predictable internal layout, so once you know the structure, finding the right information becomes fast and consistent.

TIP

When stuck: `man -k "copy file"` (apropos) finds commands by description. If empty, run `sudo mandb` once to build the index.

```
whatis passwd
man -k "print files"
man 1 passwd | head -n 20
man 5 passwd | head -n 20
passwd --help 2>&1 | head
```

Create, View, and Edit Text Files

Working efficiently at the command line means controlling where output goes and how commands connect to each other.

Input-Output Redirection

Every running program or command has three standard communication channels, each identified by a file descriptor (FD) number:

- **Standard Input (stdin)** — FD 0, typically the keyboard.
- **Standard Output (stdout)** — FD 1, typically the screen.
- **Standard Error (stderr)** — FD 2, typically the screen, but separately from stdout.

These channels can be redirected to files instead of the terminal, letting you save output, discard errors, or feed input from a file.

Pipelines

A **pipeline** connects the standard output (`stdout`) of one process directly to the standard input (`stdin`) of another, allowing commands to be chained together to process data in stages.

[Diagram — see online version]

Redirection Operators

Operator	Behavior
<code>2>/dev/null</code>	Discard stderr error messages by redirecting them to /dev/null.
<code>>file 2>&1</code> or <code>&>file</code>	Redirect stdout and stderr to overwrite the same file.
<code>>>file 2>&1</code> or <code>&>>file</code>	Redirect stdout and stderr to append to the same file.

Example

```
command &> output.log
```

This redirects both stdout and stderr into `output.log`, overwriting any existing content.

The vim Text Editor

vim is a modal command-line text editor — its behavior changes depending on which mode you're in.

#Modes of vi/vim

[Diagram — see online version]

- **Command mode** — the default mode for navigation and issuing editing commands.
- **Insert mode** — entered with `i` or `a`, used for typing text directly; return to command mode with `Esc`.
- **Visual mode** — entered with `v`, `V`, or `Ctrl+V`, used for selecting text.
- **Extended command mode (Ex mode)** — entered with `:`, used for saving, quitting, and advanced operations.

#Common Ex Mode Commands

Command	Action
<code>:wq</code>	Save and quit the current file.
<code>:x</code>	Save the current file if there are unsaved changes, then quit.
<code>:w</code>	Save the current file and remain in editor.
<code>:q</code>	Quit the current file (only if there are no unsaved changes).
<code>:q!</code>	Quit the current file, ignoring any unsaved changes.
<code>:10</code>	Jump to line number 10.

#Vim Cheat Sheet

Quick reference for moving between modes and performing common editing actions:

- **To edit mode:** `I`, `i`, `a`, `A` (variants of insert, positioned at line start, cursor, after cursor, or line end).
- **Cut, copy, paste line:** `dd` (cut/delete line), `yy` (yank/copy line), `p` (paste).
- **Delete char/word:** `x`, `X`, `dw`.
- **Join lines:** `J`.
- **Search, repeat:** `/` (search), `n` (repeat search).
- **Cursor movement:** `h` (left), `j` (down), `k` (up), `l` (right), `1G` (go to first line), `G` (go to last line), `$` (end of line).
- **Undo, redo:** `u` (undo), `.` (repeat last change).
- **Save & exit:** `ZZ`.
- **Ex mode:** `:` — includes search and replace (`:%s /old/new/g`), change settings (`:set ...`), and save/exit commands (`:w`, `:w!`, `:q`, `:q!`, `:wq`, `:x`).
- From Edit mode, or returns to Command mode.

INFO

Key Takeaway: vim's power comes from its distinct modes — navigate and issue commands in Command mode, type freely in Insert mode, and save/quit/search through Ex mode.

Variables in Linux

Linux shells support two categories of variables that control behavior and store data:

- **User-defined variables** — created and set by the user for their own use in scripts or commands.
- **Shell variables** — variables used or set by the shell itself to control its behavior and environment.

NOTE

Redirection order matters: `cmd > file 2>&1` works, but `cmd 2>&1 > file` does NOT — stderr goes to old stdout (screen). Use `&> file` as shorthand or `> file 2>&1` at the end.


```
echo hello > /tmp/demo.txt
cat /tmp/demo.txt
echo world >> /tmp/demo.txt
cat /tmp/demo.txt
# mix stdout+stderr
ls /tmp/nope /tmp/demo.txt > /tmp/out.txt 2>&1; cat /tmp/out.txt
ls /tmp/nope /tmp/demo.txt &> /tmp/both.txt; cat /tmp/both.txt
# pipes + status
echo "a b c" | tr " " "\n" | sort | uniq -c
echo ${PIPESTATUS[@]} # shows each pipe exit code
```

Word Count & Pipelines Deep Dive

What it does

wc stands for word count, and despite the name, it actually counts three things at once: lines, words, and characters.

```
echo "nti aiops" | wc
```

Here, **echo "nti aiops"** simply prints the text **nti aiops**, and that text is piped into **wc**, which analyzes it. The output has three numbers, always in this order:

Lines	Words	Characters
1	2	10

- 1 line — because **echo** only produced one line of text
- 2 words — "nti" and "aiops" are two separate words
- 10 characters — counting every letter and the space between them

Getting just ONE of these numbers

Often you don't need all three — just one specific count. You can add a flag to **wc** to isolate exactly what you want:

```
echo "nti aiops" | wc -w
```

This returns only the word count (2), with nothing else cluttering the output.

Flag	Returns
wc -l	Line count only
wc -w	Word count only
wc -c	Character count only

Combining wc with other commands

```
ls -R /etc | wc -l
```

- **ls -R /etc** lists everything inside **/etc**, and the **-R** flag makes it recursive — meaning it doesn't just list the top-level folder, it digs into every subfolder inside it too, listing everything at every depth
- Piping that into **wc -l** counts how many total lines were produced — effectively telling you how many files and folders exist inside **/etc** and all its subfolders combined

This shows how small commands combine into powerful one-liners: instead of manually counting hundreds of files, one line does it instantly.

Manage Local Users and Groups

What Is a User?

A **user account** provides security boundaries between different people and programs that can run commands. The system distinguishes user accounts by a unique identification number, the **user ID (UID)**.

There are three main types of user accounts:

- **Superuser** — the administrative account with full system privileges.
- **System users** — accounts used internally by system processes.
- **Regular users** — accounts for people who log in interactively.

UID Ranges

UID ranges follow a standard convention on Red Hat systems:

- **UID 0** — always assigned to the superuser account, `root`.
- **UID 1–200** — a range of "system users" assigned statically to system processes by Red Hat.
- **UID 201–999** — a range of "system users" used by system processes that do not own files on the file system; typically assigned dynamically from the available pool when the software needing them is installed.
- **UID 1000+** — the range available for assignment to regular users.

The `/etc/passwd` File

User account information is stored in `/etc/passwd`, with each line following a fixed colon-separated format:

```
user01:x:1000:1000:User One:/home/user01:/bin/bash
```

Each field carries specific meaning:

- **Username** for this user (`user01`).
- **Password placeholder** — the encrypted password used to be stored here, but has moved to `/etc/shadow`. This field should always be `x`.
- **UID number** for this user account (`1000`).
- **GID number** for this user account's primary group (`1000`).
- **Real name** for this user (`User One`).
- **Home directory** for this user (`/home/user01`) — the initial working directory when the shell starts, containing the user's data and configuration settings.
- **Default shell program** for this user, which runs on login (`/bin/bash`). For a regular user, this is normally the program providing the command-line prompt; a system user might use `/sbin/nologin` if interactive logins are not allowed.

The `/etc/group` File

Group information is stored in `/etc/group`, also using a fixed colon-separated format:

```
group01:x:10000:user01,user02,user03
```

- **Group name** for this group (`group01`).
- **Obsolete group password field** — should always be `x`.
- **GID number** for this group (`10000`).
- **Member list** — users who are members of this group as a supplementary group (`user01`, `user02`, `user03`).

Primary Group and Supplementary Group

- When a new regular user is created, a new group with the same name as that user is created.
- That group becomes the **primary group** for the new user, and the user is the only member of this **User Private Group**.
- Users may also belong to **supplementary groups**.

- Membership in supplementary groups is determined by the `/etc/group` file.
- Users are granted access to files based on whether *any* of their groups — primary or supplementary — have access.

INFO

Key Takeaway: A user's file access is the union of permissions granted to their primary group and every supplementary group they belong to.

Superuser Privilege: su vs. sudo

Two commands grant superuser privilege, with different models of access:

- `su` — switches the current session directly to another user account (typically root), usually requiring that account's password.
- `sudo` — allows a permitted user to run a single command with superuser privileges, typically using their own password, without fully switching accounts.

The /etc/shadow File

Password security information is stored separately in `/etc/shadow`, using this field structure:

```
name:password:lastchange:minage:maxage:warning:inactive:expire:blank
```

- **Login name**
- **Encrypted password**
- **Days since Jan 1, 1970** that the password was last changed
- **Minimum days** before the password may be changed
- **Maximum days** after which the password must be changed
- **Days before expiration** that the user is warned
- **Days after expiration** that the account is disabled
- **Account expiration date**, represented as the number of days since 1970-01-01
- **Blank field**, reserved for future use

Configuring Password Aging with chage

The `chage` command manages password aging policy per user, controlling the timeline between password changes, warnings, and account deactivation:

- `-d` — sets the last change date.
- `-m` — sets minimum days between changes.
- `-M` — sets maximum days before a change is required.
- `-W` — sets the number of warning days before expiration.
- `-I` — sets the number of inactive days after expiration before the account is disabled.

Example

```
chage -m 0 -M 90 -W 7 -I 14 user03
```

This sets `user03`'s password policy so it can be changed at any time (`-m 0`), must be changed at least every 90 days (`-M 90`), warns the user 7 days before expiration (`-W 7`), and disables the account after 14 days of inactivity past expiration (`-I 14`).

INFO

Key Takeaway: Password aging policy is enforced through `/etc/shadow` fields and configured with `chage`, giving administrators fine-grained control over how often users must update their credentials.

DANGER

Shadow perms: `/etc/shadow` should be `640` and owned `root:shadow` — never `chmod 777 /etc/shadow`. Check with `ls -l /etc/shadow; stat -c %a /etc/shadow`.

```
id
whoami; groups
sudo useradd -m demo1 2>&1 | head
sudo passwd demo1 # set, then
chage -l demo1
# lock vs disable
sudo passwd -l demo1 # locks password with !
sudo usermod -L demo1 # alternative
sudo userdel -r demo1 2>/dev/null; echo cleaned
# safe sudoers check
sudo visudo -c && echo "sudoers OK"
sudo grep -vE '^[[:space:]]*(#|$)' /etc/sudoers | head
```

Key Takeaways

- Manual pages are organized into numbered sections and follow a predictable structure, making the `man` command a reliable first stop for troubleshooting.
- Shell redirection and pipes let commands read from and write to files or each other, using standard input, output, and error streams.
- vim is a modal editor — mastering the transitions between Command, Insert, Visual, and Ex modes is the key to using it efficiently.
- User and group information lives in `/etc/passwd` and `/etc/group`, while sensitive password data and aging policy live in `/etc/shadow`.
- Every regular user gets a private primary group at creation, and file access is determined by the combination of primary and supplementary group memberships.
- `su` switches users entirely, while `sudo` grants privileged access to individual commands — both provide paths to superuser privilege with different security trade-offs.

Switching Users — su

`su - username` (with the dash)

```
su - omar
```

This switches you to being the user `omar`. The dash `(-)` is crucial: it tells Linux to also load `omar`'s entire environment, exactly as if `omar` had logged in directly himself — his home directory becomes your working directory, his shell configuration and variables load, his PATH becomes active. It will also display when `omar` last logged in, similar to what you'd see on a real login screen.

`su username` (without the dash)

```
su omar
```

This still switches you to being `omar`, but it's a lighter switch — you keep your own current environment and variables rather than fully adopting his, and you typically remain in whatever directory you were already in rather than jumping to his home folder.

Comparison

Behavior	su - username	su username
Environment loaded	Full environment, exactly as if that user logged in directly	Keeps your own current environment and variables

Behavior	su - username	su username
Working directory	Jumps to that user's home directory	Typically remains in whatever directory you were already in
Last login display	Shown, similar to a real login screen	Not part of this lighter switch
Best for	A true, full simulation of that user logging in (the safer, more predictable choice)	A quick, temporary permission switch without changing your whole environment

visudo — Safely Managing Sudo Permissions

The file behind sudo

Every time someone runs a command with `sudo`, Linux checks a specific configuration file — `/etc/sudoers` — to decide whether that user is allowed to do that, and what exactly they're permitted to run (sometimes limited to specific commands, sometimes full access).

Why not just edit it with vim directly?

You technically could open `/etc/sudoers` with `vim /etc/sudoers`, but this is considered dangerous, because a single typo in this file could silently break the entire permissions system — potentially locking every single administrator out of sudo access at once, with no easy way back in.

```
visudo
```

DANGER

Editing `/etc/sudoers` directly with a regular editor is dangerous — a single typo can silently break the entire permissions system, potentially locking every administrator out of sudo access at once, with no easy way back in.

`visudo` solves this by opening the file in a protected editing session that automatically checks your syntax for errors the moment you try to save. If it detects a mistake, it stops you, shows a warning, and refuses to save the broken version — giving you the chance to fix it or discard your changes safely. This built-in safety check is the entire reason `visudo` exists as a dedicated command instead of people just using a regular text editor.

Vim Scripting & External Commands

Vim Script: Find and Replace

```
:%s/login/nologin/g
```

`g` means Global (to replace the words).
(*new name*)

Execute External Command in Vim

```
:.! date
```

(`■■■■ ■■■■ ■■■■ ■■■■■■ ■■■ ■■■■■`)

```
:* Number ! Command
```

Will appear in the number of line that matches that number.

Vim Options

Command	Description
<code>:set number</code>	Will index the lines

Command	Description
:set arabic	Will make in arabic way
:set noarabic	Turns off arabic mode

Variables in Shell

User-defined Variable / Shell Variable — that you created / that already exists in the system or the user created it.

```
X=15  
echo $X
```

Mathematics in Shell

```
$(x+y)  
echo $(x+y)
```

Day 3 — Coming Soon

Wait for it — new content is on the way!

Coming Soon

Day 3 is being prepared. It will cover **Permissions & Processes**, **SSH**, **Logs & Networking**, plus **Archiving & Packages**. Check back soon — the café is still brewing the slides ■.

INFO

What to expect: File permissions (chmod/chown), process control (ps/kill/systemd), SSH hardening, log analysis (journalctl), and archiving (tar/rsync).

In the meantime, review **Day 1 & Day 2** labs or explore the **Topic Index** and **Flashcards**.

Part VI — Hands-On Labs & Exercises

Solved labs from the site's Practice Lab / content/Task folders — try first, then check solutions.

Lab 1 — Solved

Source: content\Day1\Task\linux_lab1_solved.md

Linux Lab 1 — Solved Exercises

INFO

Topic: Linux commands, filesystem navigation, files, paths, and basic text processing
Purpose: Practical reference for the corresponding lab exercises.

1. less vs more

Both commands display text one screen at a time, but `less` provides more flexible navigation.

Command	Main behavior
<code>more</code>	Displays text page by page, primarily moving forward
<code>less</code>	Displays text page by page with richer navigation, including forward/backward movement

Example

```
more /etc/passwd
less /etc/passwd
```

With `less`, you can use `Space` to move forward, `b` to move backward, and `q` to quit.

2. cat vs more

`cat` normally prints the entire file directly to standard output, while `more` pauses after each screen.

```
cat /etc/passwd
more /etc/passwd
```

Key idea: Use `cat` for short files or when you want the contents passed directly through a pipeline. Use `more` when you need page-by-page viewing.

3. rm vs rmdir

The manual pages show that the commands have different purposes:

```
man rm
man rmdir
```

- `rm` removes files and can remove directories when used with the appropriate recursive option.
- `rmdir` removes **empty directories**.
- `rmdir` is therefore more restrictive than `rm`.

Examples

```
rm file1
rmdir dir1
rm -r dir1
```

4. Create and manipulate the hierarchy

The intended hierarchy can be created under the home directory with:

```
~/dir1/
├── docs/
│   ├── dir11/
│   └── dir12/
└── mycv
```

Create it

```
cd ~
mkdir -p dir1/docs/dir11 dir1/docs/dir12
touch dir1/mycv
```

4a. Remove `dir11` in one step

```
rm -r ~/dir1/docs/dir11
```

What happened?

`dir11` is a directory, so ordinary `rm dir11` fails because `rm` does not remove directories without the recursive option.

```
rm dir1/docs/dir11
# rm: cannot remove './dir11': Is a directory
```

Use recursive removal:

```
rm -r dir1/docs/dir11
```

INFO

Caution: `rm -r` recursively removes the directory and its contents. Use it carefully.

4b. Remove `dir12` using `rmdir -p`

```
rmdir -p ~/dir1/docs/dir12
```

Because `rmdir -p` removes the specified empty directory and then attempts to remove its empty parent directories, `docs` is also removed if it has become empty.

The resulting hierarchy is:

```
~/dir1/
└── mycv
```

`dir1` remains because it contains `mycv`.

4c. Absolute and relative paths for `mycv`

Given:

```
pwd
/home/user
```

The file is located at:

```
/home/user/dir1/mycv
```

Absolute path:

```
/home/user/dir1/mycv
```

Relative path from `/home/user`:

```
dir1/mycv
```

5. Copy `/etc/passwd` to the home directory as `mypasswd`

```
cp /etc/passwd ~/mypasswd
```

Verify:

```
ls -l ~/mypasswd
```

6. Rename `mypasswd` to `oldpasswd`

Use `mv`:

```
mv ~/mypasswd ~/oldpasswd
```

Verify:

```
ls -l ~/oldpasswd
```

7. Four ways to go from `/usr/bin` to the home directory

Assuming the home directory is `/home/user`:

Method 1 — `cd` with no argument

```
cd
```

Method 2 — Use `~`

```
cd ~
```

Method 3 — Use `$HOME`

```
cd "$HOME"
```

Method 4 — Use the absolute path

```
cd /home/user
```

INFO

`cd -` is also useful, but it switches to the previous working directory rather than specifically meaning "go home."

8. List commands in `/usr/bin` beginning with `w`

```
ls /usr/bin/w*
```

For a cleaner command-only list:

```
find /usr/bin -maxdepth 1 -type f -name 'w*' -printf '%f\n'
```

The first command may also include matching directories or symlinks depending on the system.

9. Display the first 4 lines of `/etc/passwd`

```
head -n 4 /etc/passwd
```

Equivalent short form:

```
head -4 /etc/passwd
```

10. Display the last 7 lines of `/etc/passwd`

```
tail -n 7 /etc/passwd
```

Equivalent short form:

```
tail -7 /etc/passwd
```

Quick Command Reference

Task	Command
View file interactively	<code>less file</code>
View file page by page	<code>more file</code>
Print file	<code>cat file</code>
Remove file	<code>rm file</code>
Remove empty directory	<code>rmdir dir</code>
Remove directory recursively	<code>rm -r dir</code>
Create directories	<code>mkdir -p path</code>
Create empty file	<code>touch file</code>
Copy	<code>cp source destination</code>
Rename/move	<code>mv old new</code>
Go home	<code>cd ~</code>

Task	Command
First lines	<code>head -n N file</code>
Last lines	<code>tail -n N file</code>

Key Takeaways

- `less` provides more flexible navigation than `more`.
- `cat` prints a file directly, while pagers such as `more` and `less` display it interactively.
- `rm` and `rmdir` have different directory-removal behavior.
- `rmdir -p` can remove empty parent directories.
- `~` and `$HOME` are convenient ways to refer to the user's home directory.
- `head` and `tail` are useful for extracting the beginning and end of files.

Lab 2 — Solved

Source: content\Day2\Task\linux_lab2_solved.md

Linux Lab 2 — Solved Exercises

INFO

Topic: Manual pages, user management, Vim, redirection, and administrative access

Purpose: Practical reference for the corresponding lab exercises.

1. Display the `passwd` command and file manual pages sequentially

The `passwd` command has a section 1 manual entry, while the `/etc/passwd` file is documented in section 5.

Display them sequentially with:

```
man passwd
man 5 passwd
```

Exit the first manual page with `q`, then the second opens.

You can also invoke them as separate commands in a shell sequence:

```
man passwd; man 5 passwd
```

2. Display the manual page for the `passwd` file

The password database format is documented in section 5:

```
man 5 passwd
```

The `5` explicitly selects the **file formats** section rather than the `passwd` command's section.

3. Find commands whose manual pages contain `passwd`

Use `man -k`, which searches manual-page descriptions and keywords:

```
man -k passwd
```

On systems where `apropos` is available, the equivalent is:

```
apropos passwd
```

4. Create a user account

The requested account is:

```
Username: Islam
Password: Islam
```

Creating a user generally requires root or `sudo` privileges.

Create the account

```
sudo useradd -m Islam
```

Set the password

```
sudo passwd Islam
```

When prompted, enter:

```
Islam
```

INFO

Security note: A real system should not use a predictable password such as `Islam`. The value is used here only because it is explicitly specified by the lab.

Verify

```
id Islam
```

5. Search manual pages mentioning `password`

Use:

```
man -k password
```

This searches the manual database for entries associated with the keyword.

6. Create `notes.txt` using Vim

Open the file:

```
vim ~/notes.txt
```

Inside Vim:

1. Press `i` to enter **Insert mode**.
1. Type your full name.
1. Press `Esc` to return to **Command mode**.
1. Type:

```
:wq
```

1. Press `Enter`.

This saves the file and exits Vim.

7. Display `notes.txt`

Use:

```
cat ~/notes.txt
```

This prints the contents directly to the terminal.

8. Append `AIOPS` without opening an editor

Use `echo` with the append operator:

```
echo "AIOPS" >> ~/notes.txt
```

Verify:

```
cat ~/notes.txt
```

Why `>>`?

- `>` overwrites the file.
- `>>` appends to the existing content.

So using `>` here would be the classic tiny command-line disaster where your previous content quietly ceases to exist.

9. Redirect `ls -l /etc` into `etc_listing.txt`

Run:

```
ls -l /etc > ~/etc_listing.txt
```

Verify that the file exists:

```
ls -l ~/etc_listing.txt
```

Then inspect its contents:

```
cat ~/etc_listing.txt
```

What happened?

The `>` operator redirects standard output from the command into a file.

Conceptually:

```
ls -l /etc
  ■
  ▼
etc_listing.txt
```

10. Safely inspect which users have sudo privileges

The lab asks you to determine which user is configured to run commands through `sudo`.

First inspect the main sudoers configuration safely with:

```
sudo visudo -c
```

This checks the configuration syntax without editing it.

To inspect the configuration:

```
sudo grep -vE '^[[:space:]]*(#|$)' /etc/sudoers
```

Also inspect included configuration files:

```
sudo grep -R -vE '^[[:space:]]*(#|$)' /etc/sudoers.d/
```

Common configurations grant sudo access to a group such as:

```
%sudo ALL=(ALL:ALL) ALL
```

Check the current user's groups:

```
groups
```

On many Debian/Ubuntu systems, membership in the `sudo` group indicates administrative sudo access.

You can also check a specific user with:

```
sudo -l -U username
```

INFO

Important: Do not edit `/etc/sudoers` directly with a normal text editor. `visudo` performs syntax checking and is designed for safely editing sudoers configuration.

11. Switch to root with `su -`

Run:

```
su -
```

Enter the root password when prompted.

Confirm the current identity:

```
whoami
```

Expected result:

```
root
```

Return to your normal user:

```
exit
```

Verify if necessary:

```
whoami
```

INFO

On systems where direct root login is disabled, `su -` may fail even when `sudo` works. In that case, the system's configured administrative method should be used instead.

12. Create `steps.txt` in Vim and save without exiting

Open the file:

```
vim ~/steps.txt
```

Press:

```
i
```


Enter Insert mode and type:

```
Welcome to AIOPS Training
This course covers Vim
```

Press:

```
Esc
```

You are now in Command mode.

Save without exiting:

```
:w
```

Press **Enter**.

The file remains open in Vim.

13. Search for the Vim line and replace **raining** with **Course**

The lab wording appears to contain a typo:

```
search for Vim return raining
```

Based on the text entered in the previous exercise:

```
This course covers Vim
```

the intended operation appears to be searching for the line containing **Vim**, then replacing a word.

If the intended replacement is to replace **raining** with **Course**, use:

```
:%s/raining/Course/g
```

However, **raining** does not exist in the text specified by exercise 12, so this substitution would make no change.

To replace **Vim** with **Course**, which is consistent with the exercise's apparent intent:

```
:%s/Vim/Course/g
```

If the exact lab instruction is meant literally, first search:

```
/raining
```

Then press **Enter**.

INFO

Source ambiguity: The supplied lab text says "**Vim return raining**" but the preceding content contains "**This course covers Vim**". The exercise therefore contains an apparent inconsistency. The commands above preserve that distinction rather than silently pretending the typo never happened.

14. Copy the current line, paste it below, then save and exit

In Vim Command mode:

```
yy
```

This copies (yanks) the current line.

Then:

```
p
```

This pastes it below the current line.

Finally:

```
:wq
```

Save and exit.

Combined command sequence

```
Esc → yy → p → :wq → Enter
```

Vim Quick Reference

Action	Vim command
Enter Insert mode	i
Return to Command mode	Esc
Save	:w
Save and exit	:wq
Exit without saving	:q!
Search	/pattern
Copy current line	yy
Paste below	p
Delete current line	dd
Substitute text	:%s/old/new/g

Linux Command Reference

Purpose	Command
Search manual database	man -k keyword
Command manual	man command
File-format manual	man 5 passwd
Create user	sudo useradd -m username
Set password	sudo passwd username
Show user identity	whoami
Show user information	id username
Append text	echo "text" >> file
Redirect output	command > file
Switch to root	su -
Return from su	exit
Check sudoers syntax	sudo visudo -c

Key Takeaways

- Manual pages are divided into sections, and `man 5 passwd` refers to the **passwd file format**, not the `passwd` command.
- `man -k` searches the manual database by keyword.
- `>>` appends data, while `>` redirects output and overwrites the destination file.
- Vim separates **Insert mode** from **Command mode**.
- `yy` copies the current line and `p` pastes it below.
- `visudo` should be used when modifying sudoers configuration because syntax errors can break administrative access.

Part VII — Appendix: Command Reference Bank

Quick-reference for every command in the site's bank — categories, descriptions & keywords. Not a substitute for `man`, but a fast lookup.

Categories at a Glance

Category	Count
File & Directory Navigation	4 commands
File Manipulation	5 commands
Viewing & Editing Text	5 commands
System & Process Management	6 commands
Networking & Permissions	5 commands
Archiving & Compression	4 commands
Version Control (Git)	5 commands
Containers (Docker)	4 commands

File & Directory Navigation

Command	Description	Example
<code>pwd</code>	Print working directory Useful in scripts to get absolute paths — combine with <code>\$(pwd)</code> . Tip: in scripts prefer <code>\$(pwd)</code> — <code>`echo "Running in \$(pwd)"`</code> — and <code>`pwd -P`</code> resolves symlinks vs <code>`pwd -L`</code> .	<code>pwd</code>
<code>ls</code>	List directory contents <code>-a</code> shows dotfiles (<code>.bashrc</code> etc.); <code>-h</code> with <code>-l</code> gives human-readable sizes. Try <code>`ls -lhtr`</code> (human, time, reverse) and <code>`ls -li`</code> to see inodes. For spaces: <code>`ls -b`</code> escapes them.	<code>ls -la</code>
<code>cd</code>	Change directory <code>cd</code> - jumps back to previous directory; <code>cd</code> alone goes home. <code>`cd -`</code> is a stack of 1 — use <code>`pushd`/`popd`</code> or <code>`cd \$OLDPWD`</code> for more. <code>`cd`</code> with no arg is <code>`cd \$HOME`</code> .	<code>cd /var/log</code>
<code>tree</code>	List contents in a tree-like format Not installed by default — <code>sudo apt install tree</code> . If missing: <code>`sudo apt install tree`</code> or fallback <code>`find . -maxdepth 2 -type d   sort`</code> (faster, no color).	<code>tree -L 2</code>

File Manipulation

Command	Description	Example
<code>touch</code>	Create an empty file / update timestamp If the file exists, it only updates the modified timestamp — no data is lost. For spaces: <code>`touch "my file.txt" —</code> and <code>`touch --dashfile`</code> creates a file starting with dash. <code>`touch</code>	<code>touch newfile.txt</code>
<code>mkdir</code>	Make a directory Without <code>-p</code> , it fails if parent directories don't already exist. Quote parents with spaces: <code>`mkdir -p "My Projects/src" —</code> without <code>-p</code> fails if parent missing.	<code>mkdir -p dir/subdir</code>
<code>cp</code>	Copy files/directories <code>-r</code> is required for directories; use <code>-i</code> to get a prompt before overwriting. Use <code>`cp -a`</code> to preserve perms/symlinks (archive). For safety: <code>`cp -i`</code> prompts, <code>`cp -n`</code> no-clobber. Never	<code>cp -r source_dir/dest_dir/</code>
<code>mv</code>	Move or rename files No <code>-i</code> prompt by default — silently overwrites an existing destination file. Add <code>`-i`</code> to prompt before overwrite. Across filesystems <code>`mv`</code> copies+deletes — use <code>`rsync --remove-sourc</code>	<code>mv oldname.txt newname.txt</code>
<code>rm</code>	Remove files/directories There is no Recycle Bin. <code>rm -rf</code> is permanent and irreversible — double-check the path. Danger: <code>`rm -rf /tmp/*`</code> OK, but <code>`rm -rf / tmp/*`</code> (space) = disaster. Use <code>`rm --file`</code> for d	<code>rm -rf folder/</code>

Viewing & Editing Text

Command	Description	Example
<code>cat</code>	Concatenate and print file content Not ideal for huge files — it dumps everything at once; use <code>less</code> instead. For numbers: <code>`cat -n file`</code> or better <code>`nl -ba`</code> . For large: <code>`less`</code> or <code>`bat`</code> (syntax). <code>`cat file1 file2 > co</code>	<code>cat file.txt</code>
<code>less</code>	View file content page by page <code>/searchterm</code> searches forward, <code>n</code> repeats the search — much better than <code>more</code> . Inside <code>less`</code> : <code>`g`</code> top, <code>`G`</code> bottom, <code>`error`</code> + <code>`n`</code> next, <code>`N`</code> prev, <code>`F`</code> follow (like <code>tail -F</code>). Quieter tha	<code>less largefile.log</code>
<code>head/tail</code>	View first/last lines of a file <code>-f</code> is the go-to for watching live logs; <code>Ctrl+C</code> to stop following. <code>`tail -n +5 file`</code> from line 5 onward. <code>`head -c 100`</code> first bytes. For live: <code>`tail -F`</code> follows across rotations (be	<code>tail -f app.log</code>
<code>grep</code>	Search text within files <code>-r</code> for recursive search, <code>-n</code> for line numbers, <code>-v</code> to invert (exclude matches). Speed: <code>`grep -F "literal" (fixed) 10× faster than regex. Limit scope: `grep -R --include="*.conf"`.</code>	<code>grep -i "error" /var/log/syslog</code>
<code>nano/vim</code>	Terminal text editors <code>nano</code> is beginner-friendly; <code>vim</code> has a steeper curve — <code>:q!</code> quits without saving. <code>nano`</code> : <code>Ctrl+O`</code> write, <code>Ctrl+X`</code> exit. <code>vim`</code> : <code>vim -R`</code> read-only, <code>:%s/old/new/gc`</code> with confirm. For qui	<code>nano config.yaml</code>

System & Process Management

Command	Description	Example
top/htop	Interactive process viewer htop is friendlier and color-coded but usually needs installing separately. Press <code>`1`</code> in top/htop to see per-CPU. <code>`htop`</code> → <code>`F6`</code> sort, <code>`k`</code> kill, <code>`u`</code> filter user. For batch: <code>`ps aux`</code>	htop
ps	Snapshot of current processes aux shows all processes for all users, not just yours — combine with grep to filter. <code>`ps -ef --forest`</code> or <code>`ps auxf`</code> shows tree. Combine: <code>`ps aux --sort=-%mem   head -n 10`</code> for mem	<code>ps aux grep python</code>
kill	Terminate a process by PID -9 force-kills without letting the process clean up — try plain kill (SIGTERM) first. Try <code>`kill -TERM 1234`</code> (15) first, then <code>`kill -KILL`</code> (9) only if needed — -9 skips cleanup. Fi	<code>kill -9 1234</code>
systemctl	Manage system services Use <code>systemctl status service_name</code> to check if something actually started correctly. Check first: <code>`systemctl status nginx`</code> — <code>`enabled`</code> vs <code>`active`</code> . <code>`systemctl is-enabled`</code> vs <code>`is-ac`</code>	<code>sudo systemctl restart nginx</code>
df	Disk space usage Checks whole filesystems/partitions — good first check when you get 'no space left on device'. Inodes matter too: <code>`df -i`</code> — "No space left" can be inode exhaustion, not bytes. Bin	<code>df -h</code>
du	Directory space usage Without <code>-s</code> it recursively lists every subfolder's size — can be slow on large trees. <code>`du -sh --max-depth=1 /var`</code> limits depth (fast). <code>`du -sh *   sort -rh`</code> finds biggest. For inte	<code>du -sh /var/log</code>

Networking & Permissions

Command	Description	Example
chmod	Change file permissions Never use <code>chmod 777</code> casually — it grants everyone full read/write/execute access. Numeric vs symbolic: <code>`chmod 644 file`</code> (= <code>rw-r--r--</code>) vs <code>`chmod u+x,go-w`</code> . Check <code>`umask 022`</code> — defa	<code>chmod +x script.sh</code>
chown	Change file owner Usually requires sudo; changing ownership on system files can break services. Recursive danger: <code>`chown -R user:group /`</code> breaks system. Always verify with <code>`ls -ld`</code> first. For group	<code>sudo chown user:group file.txt</code>
ip	Show/manipulate IP addresses Replaces the older, deprecated ifconfig on most modern distros. Modern: <code>`ip -br a`</code> brief, <code>`ip route`</code> default gw, <code>`ss -tulpn`</code> replaces <code>`netstat`</code> . <code>`ifconfig`</code> is deprecated — use <code>`ip`</code>	<code>ip a</code>
curl	Transfer data from URLs -O saves with the remote filename; add -L to follow redirects. Add <code>`-L`</code> to follow redirects (<code>`curl -L -O https://.../file.zip`</code>). For headers: <code>`-I`</code> . Never <code>`curl http://x   bash`</code> wi	<code>curl -O https://example.com/file.zip</code>
ssh	Secure shell remote login Add -p PORT for a non-default port; key-based auth (ssh-keygen) is safer than passwords. Generate ed25519: <code>`ssh-keygen -t ed25519 -C "host"`</code> — safer than RSA. Disable root: <code>`Permi</code>	<code>ssh username@192.168.1.10</code>

Archiving & Compression

Command	Description	Example
tar	Archive files into a tarball -c create, -z gzip, -v verbose, -f file. Use -x to extract: <code>tar -xvf archive.tar.gz</code> . Preserve perms: <code>`tar -p -czf`</code> . Danger absolute: <code>`tar -czf backup.tar.gz -P /etc`</code> stores leadi	<code>tar -czvf archive.tar.gz folder/</code>
gzip/gunzip	Compress or decompress files gzip replaces the file with file.log.gz; use -k to keep the original, gunzip to reverse. Keep original: <code>`gzip -k file.log`</code> (keep) vs default replaces. Test: <code>`gzip -t file.gz`</code> veri	<code>gzip file.log</code>
zip/unzip	Package files into a .zip archive Cross-platform archives; -r is required to include subdirectories. For permission preserve on Linux, prefer <code>`tar.gz`</code> . <code>`zip -r9`</code> best compress, <code>`-e`</code> encrypt (weak).	<code>zip -r app.zip app/</code>
rsync	Efficiently sync files/directories -a preserves permissions, -v verbose, -z compresses. Great for backups and deploys. Dry-run first: <code>`rsync -avzn src/ dst/ (-n)`</code> . Show progress: <code>`--progress`</code> or <code>`--info=progress2`</code> .	<code>rsync -avz src/ user@host:/dest</code>

Version Control (Git)

Command	Description	Example
<code>git clone</code>	Copy a remote repository locally Add a folder name at the end to clone into a specific directory. Shallow: <code>`git clone --depth 1`</code> for huge repos. Use <code>`--branch main`</code> to clone single branch.	<code>git clone https://github.com/user/repo.git</code>
<code>git status</code>	Show working tree state <code>-s</code> gives a short, concise output of changed files. Short: <code>`git status -sb`</code> (branch + short). For ignore check: <code>`git check-ignore -v file`</code> .	<code>git status -s</code>
<code>git commit</code>	Record staged changes Use <code>git add</code> first to stage; <code>-a</code> commits already-tracked modified files. Amend last: <code>`git commit --amend`</code> . For co-authors: add <code>`Co-authored-by:`</code> . Use <code>`commit.template`</code> for consistency	<code>git commit -m "fix: typo"</code>
<code>git push/pull</code>	Upload or download commits <code>pull</code> = <code>fetch</code> + <code>merge</code> . Use <code>--set-upstream (-u)</code> the first time you push a branch. Safer <code>pull</code> : <code>`git pull --rebase`</code> avoids merge commits. First push: <code>`git push -u origin feature`</code> sets	<code>git push origin main</code>
<code>git branch</code>	List or create branches <code>checkout -b</code> creates and switches; <code>git merge feature</code> integrates changes. Delete merged: <code>`git branch -d feature`</code> . For remote: <code>`git push origin --delete feature`</code> .	<code>git checkout -b feature</code>

Containers (Docker)

Command	Description	Example
<code>docker run</code>	Run a container from an image <code>-d</code> detached, <code>-p</code> host:container port mapping. Image is pulled if missing. Never <code>`--privileged`</code> unless needed — breaks isolation. Clean after: <code>`docker run --rm`</code> auto-remove.	<code>docker run -d -p 8080:80 nginx</code>
<code>docker ps</code>	List running containers <code>-a</code> shows all containers including stopped ones. Format: <code>`docker ps --format "table {{.Names}} {{.Status}}"`</code> for scripts. Watch: <code>`watch -n1 docker ps`</code> .	<code>docker ps -a</code>
<code>docker build</code>	Build an image from a Dockerfile <code>-t</code> tags the image; the <code>.</code> is the build context (current dir). Layer cache: order Dockerfile from least to most changing. Use <code>`.dockerignore`</code> to skip. Try <code>`docker build --no-cache`</code>	<code>docker build -t myapp .</code>
<code>docker compose</code>	Manage multi-container apps Reads <code>docker-compose.yml</code> ; <code>-d</code> starts in the background. Check config: <code>`docker compose config`</code> . For logs: <code>`docker compose logs -f --tail 50`</code> .	<code>docker compose up -d</code>

Alphabetical Bank (excerpt)

Total bank entries: 164 — below is a compact index grouped by category.

Command	Category	What it does
<code>7z</code>	Archiving	7-Zip archiver (high compression).
<code>alias</code>	System Info	Creates a shortcut command for a longer one.
<code>apt</code>	Package Management	Installs, updates, and removes Debian packages.
<code>awk</code>	Text Processing	Pattern scanning and processing language.
<code>bg</code>	Process Management	Resumes job in background.
<code>blkid</code>	Disk Management	Shows block device UUID, TYPE and attributes.
<code>bzip2</code>	Archiving	Compresses with bzip2.
<code>cal</code>	System Info	Shows calendar.
<code>cat</code>	Text Processing	Reads and outputs file contents.
<code>cd</code>	File System	Changes the current working directory.
<code>chage</code>	User Management	Changes password expiry info.
<code>chgrp</code>	Permissions	Changes group ownership.
<code>chmod</code>	Permissions	Changes file access permissions.

Command	Category	What it does
<code>chown</code>	Permissions	Changes file owner and group.
<code>chsh</code>	User Management	Changes login shell.
<code>clear</code>	System Info	Clears the terminal screen.
<code>cmp</code>	Text Processing	Compares files byte by byte.
<code>column</code>	Text Processing	Formats input into columns.
<code>cp</code>	File System	Copies files or directories.
<code>crontab</code>	Process Management	Schedules commands to run on a timer.
<code>curl</code>	Networking	Transfers data to or from a server.
<code>date</code>	System Info	Shows or sets date/time.
<code>dd</code>	Disk Management	Copies and converts raw data.
<code>df</code>	System Info	Reports file system disk space usage.
<code>diff</code>	Text Processing	Compares files line by line.
<code>dig</code>	Networking	DNS lookup utility.
<code>dmesg</code>	System Info	Kernel ring buffer messages.
<code>dnf</code>	Package Management	Modern version of yum for Fedora/RHEL.
<code>docker</code>	Containers	Platform for building and running containers.
<code>docker compose</code>	Containers	Manages multi-container applications via compose file.
<code>docker ps</code>	Containers	Lists running (or all) containers.
<code>docker run</code>	Containers	Runs a new container from an image.
<code>du</code>	System Info	Estimates file and directory space usage.
<code>echo</code>	Text Processing	Prints text or variables to standard output.
<code>env</code>	Shell & Scripting	Runs program with modified environment.
<code>export</code>	Shell & Scripting	Exports variable to child processes.
<code>fdisk</code>	Disk Management	Partitions a disk (MBR/GPT).
<code>fg</code>	Process Management	Brings job to foreground.
<code>file</code>	File System	Determines file type by content.
<code>find</code>	File System	Searches for files in a directory hierarchy.
<code>findmnt</code>	Disk Management	Finds and lists mounted filesystems.
<code>firewall-cmd</code>	Networking	Firewalld control.
<code>free</code>	System Info	Displays amount of free and used memory.
<code>free</code>	System Info	Already in bank but enriched — displays RAM/swap.
<code>getfacl</code>	Permissions	Gets file ACLs.
<code>git</code>	Version Control	Distributed version control system for tracking changes.
<code>git clone</code>	Version Control	Downloads a remote repository to your machine.
<code>git commit</code>	Version Control	Saves a snapshot of your staged changes.
<code>git pull</code>	Version Control	Downloads and merges remote changes locally.
<code>git push</code>	Version Control	Uploads local commits to a remote repository.
<code>grep</code>	Text Processing	Searches for patterns within text.
<code>groupadd</code>	Permissions	Creates a new user group.
<code>groupdel</code>	User Management	Deletes a group.

Command	Category	What it does
groups	User Management	Shows groups for a user.
gzip	Archiving	Compresses a file using the gzip algorithm.
head	Text Processing	Outputs the first lines of a file.
help	Shell & Scripting	Shows help for shell builtin.
hexdump	Text Processing	Hex dump of file.
history	System Info	Shows the command line history.
host	Networking	Simple DNS lookup.
hostnamectl	System Info	Controls system hostname.
htop	Process Management	Interactive process viewer.
id	User Management	Prints user and group IDs.
ifconfig	Networking	Configures network interfaces (legacy).
iostat	System Monitoring	CPU and I/O statistics.
ip	Networking	Shows or manipulates routing and network devices.
iptables	Networking	Netfilter firewall rules.
jobs	Process Management	Lists active jobs.
journalctl	System Info	Queries the systemd journal (logs).
kill	Process Management	Terminates a process by PID.
killall	Process Management	Terminates processes by name.
last	User Management	Shows last logins from wtmp.
lastlog	User Management	Reports most recent login per user.
less	Text Processing	Views file contents one page at a time.
ln	File System	Creates hard or symbolic (soft) links.
ls	File System	Lists directory contents.
lsblk	Disk Management	Lists block devices and partitions in a tree view.
lsmod	System Info	Lists loaded kernel modules.
lspci	System Info	Lists PCI devices.
lsusb	System Info	Lists USB devices.
man	Shell & Scripting	Displays manual page.
mkdir	File System	Creates a new directory.
mkfs.ext4	Disk Management	Creates an ext4 filesystem.
modprobe	System Info	Adds/removes kernel modules.
more	Text Processing	Views file contents page by page (forward only).
mount	Disk Management	Mounts a filesystem.
mpstat	System Monitoring	Per-CPU statistics.
mtr	Networking	Combines ping + traceroute live.
mv	File System	Moves or renames files and directories.
nano	Text Processing	A simple terminal text editor.
nc	Networking	Netcat — reads/writes TCP/UDP.
ncdu	Disk Management	Interactive disk usage analyzer (ncurses).
netstat	Networking	Displays network connections and ports.

Command	Category	What it does
nice	Process Management	Runs command with priority.
nmap	Networking	Network scanner.
nohup	Process Management	Runs command immune to hangup.
nslookup	Networking	Interactive DNS query (legacy).
od	Text Processing	Octal dump of file.
pacman	Package Management	Package manager for Arch Linux.
parted	Disk Management	Advanced partition editor (supports GPT).
passwd	Permissions	Changes a user's password.
patch	Text Processing	Applies diff patch to file.
pgrep	Process Management	Finds PIDs by name.
pidof	Process Management	Finds PID of program.
ping	Networking	Tests network connectivity to a host.
pkill	Process Management	Signals processes by name.
ps	Process Management	Reports a snapshot of current processes.
pstree	Process Management	Shows processes as a tree.
pwd	File System	Prints the current working directory path.
renice	Process Management	Alters priority of running process.
rm	File System	Removes files or directories.
rmdir	File System	Removes an empty directory.
rsync	Archiving	Efficiently copies and syncs files locally or remote.
scp	Networking	Securely copies files between hosts.
screen	Process Management	Terminal multiplexer (legacy).
sed	Text Processing	Stream editor for filtering and transforming text.
setfacl	Permissions	Sets file ACLs.
sftp	Networking	Interactive SFTP session.
sort	Text Processing	Sorts lines of text files.
source	Shell & Scripting	Executes file in current shell.

... and 44 more — see the live site search for the full bank.

You made it — that's the whole site, distilled into one document. Keep this on your desk, in your bag, or on a second monitor. When something breaks at 2 a.m., you'll thank past-you for printing it.

Built with ReportLab • Source data from [assets/data/content-library.json](#) & [nti.json](#) • No ads, no trackers, no navigation chrome — just content.

Tip: Print double-sided, flip on long edge, and use the bookmarks pane for navigation.